Patrones de diseño en Java

17 patrones del GoF, con diagramas UML, construyendo Emberdeep, un juego de mazmorras por turnos

Fábrica de Cursos
24 de octubre de 2026

Índice general

Cómo usar este libro **iv**

I Punto de partida y primer patrón **1**

1. Del dolor al patrón: el núcleo de Emberdeep **2**

1.1. Qué es un patrón (y qué no) . . . 2
1.2. Cómo se lee cada capítulo . . . 3
1.3. Leer un diagrama de clases UML . . . 3
1.4. Emberdeep y su estructura de carpetas . . . 5
1.4.1. Capas arriba, funcionalidad dentro del motor . . . 5
1.4.2. Composition root . . . 5
1.4.3. Un proyecto Gradle . . . 5
1.5. En el proyecto: H1, el núcleo sin patrones . . . 7
1.5.1. Las clases de datos y Fighter . . . 7
1.5.2. El resto del núcleo . . . 8
1.6. Seis olores que ya se ven en el núcleo . . . 10
1.7. Ejercicios . . . 11
1.8. Resumen . . . 14
Soluciones de los ejercicios . . . 15

2. Strategy: IA del enemigo y fórmula de daño **27**

2.1. El olor que pide Strategy: los `if` por nombre . . . 27
2.2. Piezas del patrón Strategy . . . 28
2.3. El ejemplo de Strategy, pieza a pieza: la IA del enemigo . . . 29
2.3.1. La estrategia: `EnemyAi` . . . 29
2.3.2. Una estrategia concreta: `FirstAliveAi` . . . 29
2.3.3. El portador y el contexto: `Enemy` y `Fighter` . . . 30
2.3.4. El resultado en Combat . . . 31
2.3.5. Aleatoriedad repetible: Random con semilla inyectada . . . 31
2.4. Strategy con lambdas: la fórmula de daño . . . 32
2.4.1. Una interfaz funcional . . . 32
2.4.2. Una versión de clase y otra de lambda . . . 32
2.4.3. Clase o lambda . . . 33

2.5. Strategy: cuándo sí y cuándo no . . . 34
2.6. En el proyecto: H2 con Strategy . . . 34
2.7. Ejercicios . . . 36
2.8. Resumen . . . 39
Soluciones de los ejercicios . . . 40

II Crear y compartir objetos 55

3. Factory Method y Abstract Factory: enemigos y objetos por bioma 56
3.1. El olor que pide Factory Method: `new` por todas partes . . . 56
3.2. Piezas del patrón Factory Method . . . 57
3.3. El ejemplo de Factory Method, pieza a pieza: los spawners . . . 58
3.3.1. El producto: `Enemy` gana un `kind` . . . 58
3.3.2. El creador: `EnemySpawner` . . . 58
3.3.3. Los creadores concretos . . . 59
3.3.4. El cliente: `GameSession` . . . 60
3.4. El olor que pide Abstract Factory: un esqueleto con una poción de caverna . . . 61
3.5. Piezas del patrón Abstract Factory . . . 61
3.6. El ejemplo de Abstract Factory, pieza a pieza: los biomas . . . 62
3.6.1. Los productos: `Item` e `ItemKind` . . . 62
3.6.2. La fábrica abstracta y las dos fábricas concretas . . . 63
3.6.3. El cliente con una sola fábrica . . . 65
3.6.4. Por qué los spawners se retiran . . . 65
3.7. Si ya lo conoces: lo nuevo aquí es... . . . 66
3.8. Factory Method y Abstract Factory: cuándo sí y cuándo no . . . 67
3.9. En el proyecto: H3 con Factory Method y Abstract Factory . . . 68
3.9.1. El inventario: un record inmutable . . . 68
3.10. Ejercicios . . . 70
3.11. Resumen . . . 75
Soluciones de los ejercicios . . . 76

4. Builder y Singleton: héroes y mazmorra paso a paso 90
4.1. El olor que pide Builder: un constructor con ocho parámetros . . . 90
4.2. Piezas del patrón Builder . . . 91
4.3. El ejemplo de Builder, pieza a pieza: `HeroBuilder` . . . 91
4.4. Si ya lo conoces: Builder y `record` . . . 92
4.5. Builder anidado: `DungeonBuilder` y `FloorBuilder` . . . 93
4.6. El olor que pide Singleton: la configuración que todos leen . . . 94
4.7. Piezas del patrón Singleton . . . 95

4.8. El ejemplo de Singleton, pieza a pieza: holder y `enum` . . . 95
4.8.1. Primer intento: perezoso, y con un fallo . . . 95
4.8.2. Solución 1: el holder . . . 96
4.8.3. Solución 2: el `enum` . . . 97
4.9. Si ya lo conoces: lo nuevo en Singleton es medir su coste . . . 97
4.10. Singleton: cuándo sí y cuándo no; la alternativa de `Main` . . . 97
4.11. En el proyecto: H4 con Builder y Singleton . . . 98
4.12. Ejercicios . . . 99
4.13. Resumen . . . 105
Soluciones de los ejercicios . . . 106

5. Prototype y Flyweight: copiar o compartir 120
5.1. El olor que pide Prototype: montar la misma plantilla cien veces . . . 120
5.2. Piezas del patrón Prototype . . . 121
5.3. El ejemplo de Prototype, pieza a pieza: `EnemyTemplate.copy()` . . . 122
5.4. Prototype: el bug de la copia superficial . . . 125
5.5. Si ya lo conoces: lo nuevo en Prototype es la copia profunda y el registro . . . 126
5.6. El olor que pide Flyweight: un `TileType` por cada casilla . . . 126
5.7. Piezas del patrón Flyweight . . . 127
5.8. El ejemplo de Flyweight, pieza a pieza: `TileTypes` . . . 128
5.9. Prototype, Factory y Flyweight: copiar, construir o compartir . . . 130
5.10. Prototype y Flyweight: cuándo sí y cuándo no . . . 130
5.11. En el proyecto: H5 y H6 con Prototype y Flyweight . . . 131
5.12. Ejercicios . . . 132
5.13. Resumen . . . 138
Soluciones de los ejercicios . . . 139

III Ampliar el comportamiento 164

IV Comunicar 165

V Estados, estructura y cierre 166

Referencias 167

Cómo usar este libro

A quién va dirigido

Este libro es para quien ya programa en Java moderno y quiere aprender los patrones de diseño del catálogo GoF[7] para reconocerlos y aplicarlos en su trabajo, y poder defender la decisión en una *code review*.

Damos por sabido lo siguiente:

- Java 21: *records*, interfaces `sealed`, *pattern matching*, `switch` con patrones y lambdas. No hay repaso.
- Los principios SOLID. Tampoco se repasan.
- Singleton, Builder y Factory: seguramente los conoces. Aun así tienen su capítulo, con un enfoque nuevo, y una sección «Si ya lo conoces: lo nuevo aquí es...» que te dice qué repasas y qué es nuevo.

No necesitas conocer ningún framework. Este libro no usa Spring ni ningún otro, ni inyección de dependencias automática, ni Lombok: las dependencias se pasan a mano por el constructor.

Qué vas a aprender

Aprenderás 17 patrones del GoF con profundidad: cuándo aparecen (el «olor» en el código), qué piezas tienen, cómo se escriben y cuándo *no* conviene usarlos. Los capítulos se ordenan por el problema del juego, no por familia del catálogo, en cinco partes:

- **Parte I. Punto de partida y primer patrón** (caps. 1-2): el núcleo de Emberdeep sin patrones y Strategy.
- **Parte II. Crear y compartir objetos** (caps. 3-5): Factory Method, Abstract Factory, Builder, Singleton, Prototype y Flyweight.
- **Parte III. Ampliar el comportamiento** (caps. 6-8): Decorator, Chain of Responsibility, Command y Memento.
- **Parte IV. Comunicar** (caps. 9-11): Observer, Bridge y Mediator.
- **Parte V. Estados, estructura y cierre** (caps. 12-14): State, Composite y Facade, con guardado de la partida en fichero.

Cuenta con unas 25-28 horas, a razón de hora y media a dos horas por capítulo. Los capítulos 5 y 6 tratan dos patrones cada uno y son los más largos.

Al terminar sabrás reconocer el patrón adecuado, aplicarlo y explicar por qué es mejor que la alternativa más simple. El libro sirve también como guía de consulta: el apéndice final reúne una tabla con los 17 patrones para recordar de un vistazo qué hace cada uno y cómo se usa.

Cómo se lee cada patrón

Cada patrón sigue siempre la misma ficha, para que sepas dónde buscar:

1. **El olor que pide el patrón.** El problema, visto en el código de Emberdeep antes de aplicar nada.
2. **Piezas del patrón.** Una tabla y un diagrama UML.
3. **El ejemplo, pieza a pieza.** El código, explicado parte por parte.
4. **Cuándo sí y cuándo no.** Cuándo compensa y cuándo es sobrediseño.
5. **Contraste**, cuando el patrón se confunde con otro (por ejemplo, Decorator frente a Chain of Responsibility).
6. **En el proyecto.** Cómo avanza Emberdeep con ese patrón.

El nombre del patrón aparece en el título de cada sección que trata de él. Los patrones que quizá ya conoces (Factory, Builder, Singleton, Prototype, Bridge y Facade) llevan además una sección «Si ya lo conoces: lo nuevo aquí es…».

La tabla de piezas tiene cuatro columnas. Así se lee, con el patrón Strategy como ejemplo:

Rol	Clase en el ejemplo	Propósito	Paquete
Estrategia (interfaz)	`EnemyAi`	Elige objetivo	`engine.combat.ai`
Contexto (clase)	`Fighter`	Usa la estrategia	`engine.character`

- **Rol**: el nombre que le da el patrón a la pieza y su tipo en Java (interfaz, clase, *record*…).
- **Clase en el ejemplo**: cómo se llama esa pieza en el código de Emberdeep.
- **Propósito**: para qué sirve, en pocas palabras.
- **Paquete**: dónde vive en el proyecto.

Las piezas que construyes tú en un ejercicio de proyecto, y no el texto, se marcan con «(ejercicio)».

Cómo leer los diagramas UML

Cada patrón lleva un diagrama de clases UML para entenderlo de un vistazo. El capítulo 1 trae la leyenda completa: aquí tienes solo el mapa de símbolos, que usarás en todo el libro. Los diagramas muestran solo lo relevante para el patrón, no todos los atributos y métodos.

Símbolo	Significa
Caja con tres compartimentos	Clase: nombre, atributos y métodos.
Caja con «interface»	Interfaz: solo métodos.
Caja amarilla	Nota con una aclaración.
Línea con triángulo hueco	Herencia (`extends`).
Línea discontinua con triángulo hueco	Realización (`implements`).
Flecha abierta	Asociación: una clase guarda una referencia a otra.
Rombo hueco	Agregación: el todo agrupa partes que pueden vivir solas.
Rombo lleno	Composición: las partes no existen sin el todo.
Flecha discontinua	Dependencia: una clase usa a otra sin guardarla.

Cómo se dibujan los diagramas

Los diagramas están dibujados con TikZ[17], que ya forma parte del libro, y no con una herramienta externa. A ti no te hace falta saber TikZ: solo leerlos.

Los diagramas de contraste se colocan lado a lado, para comparar a la vez la forma de dos patrones parecidos.

El proyecto del curso

Construirás **Emberdeep**, un juego de mazmorras por turnos para la consola. Un grupo de unos tres héroes baja por una mazmorra de unas pocas plantas y lucha contra enemigos con cuatro acciones: atacar, defender, usar un objeto y lanzar una habilidad. Al terminar el curso tendrás un juego completo, con equipo y mejoras temporales, una tubería de daño, deshacer, repetición de partidas, logros, estados, oleadas de enemigos y guardado de la partida en un fichero.

El juego es el pretexto: cada patrón entra porque Emberdeep lo necesita, no porque toque. Las reglas son mínimas a propósito, para que lo difícil sean los patrones y no el dominio. En el capítulo 1 escribes un núcleo jugable sin ningún patrón; después, cada capítulo avanza el proyecto un hito y parte exactamente del estado que dejó el anterior. El orden de los hitos no es siempre el de su número: por ejemplo, los puntos de control de Memento (hito H15a) llegan en el capítulo 8, antes del hito H10.

Emberdeep vive en una carpeta `emberdeep/`, aparte del libro, y se organiza en tres paquetes (`app`, `engine` y `ui`). El motor no imprime nada; la capa `ui` pinta lo que el motor emite. Cada vez que la estructura de carpetas cambia, el libro te muestra el árbol actualizado.

Cómo ejecutar Emberdeep

El proyecto usa Gradle y no tiene dependencias en tiempo de ejecución; la única librería que aparecerá es JUnit, a partir del capítulo 8. El capítulo 1 crea los archivos de Gradle y genera el *wrapper*. Desde la carpeta `emberdeep/`, el juego se arranca así:

```
./gradlew run
```

Más adelante, el modo «guion» lee las acciones de un archivo en lugar del teclado, y se lanza con `./gradlew run -args="scripts/demo.txt"`. El capítulo donde aparece lo explica con detalle.

El código Java del libro no se compila al escribirlo: lo escribes tú y lo ejecutas en tu máquina. Por eso conviene que, al seguir cada capítulo, ejecutes el proyecto con frecuencia y compares tu resultado con el que se indica.

Los ejercicios

Cada capítulo termina con ejercicios. Todos se pueden resolver con lo que has visto hasta ese punto del libro: si un ejercicio necesita una herramienta o una dependencia, el libro te habrá explicado antes cómo instalarla y usarla. Cada ejercicio indica su dificultad:

Dificultad	Qué significa
★☆☆☆☆	Reproducir: aplicar lo que acabas de ver siguiendo los pasos que se indican.
★★☆☆☆	Aplicar con guía: usar un concepto nuevo en un caso parecido, con pistas.
★★★☆☆	Aplicar sin guía: resolver un problema nuevo combinando lo visto en el capítulo.
★★★★☆	Integrar: combinar conceptos de varios capítulos y tomar decisiones de diseño.
★★★★★	Diseñar: problema abierto con varias soluciones válidas; se da una de referencia y criterios para valorar la tuya.

y su tipo:

Tipo	En qué consiste
Guiado	Pasos numerados que te llevan de la mano.
Práctica	Resolver un problema concreto con lo aprendido.
Proyecto	Avanzar el proyecto del curso un hito más.
Diseño	Decidir cómo estructurar algo y justificarlo.
Depuración	Encontrar y corregir el fallo de un código dado.
Lectura de código	Leer código y responder preguntas sobre él.
Test	Escribir o completar pruebas automáticas.

Cada capítulo incluye un ejercicio de tipo proyecto, que avanza Emberdeep. Si tu código del capítulo anterior difiere del que usa el libro, compáralo con la solución del ejercicio de proyecto anterior antes de seguir.

Las soluciones completas están al final de cada capítulo, en la sección *Soluciones de los ejercicios*. Cada enunciado enlaza con su solución y cada solución con su enunciado. Intenta resolver el ejercicio antes de mirarla: la solución es más útil cuando la comparas con la tuya.

Qué necesitas instalar para empezar

Para empezar solo necesitas lo que ya tienes: JDK 21, Gradle, Git y un editor con soporte para Java (en este curso, NeoVim). El libro no explica esas instalaciones. Antes del capítulo 1 comprueba que funcionan:

```
java -version
gradle --version
git --version
```

`java -version` debe indicar la versión 21. Cualquier otra herramienta que hiciera falta se instala en el capítulo donde se usa por primera vez (la única prevista es JUnit, en el capítulo 8).

Referencias

El libro se apoya en libros, artículos y documentación que puedes consultar para profundizar. Aparecen citados entre corchetes, como [7], y la lista completa está al final, en *Referencias*.

Parte I

Punto de partida y primer patrón

CAPÍTULO 1

Del dolor al patrón: el núcleo de Emberdeep

Objetivos del capítulo

- Explicar qué es un patrón de diseño y qué no es, y cómo se lee la ficha de cada capítulo.
- Leer un diagrama de clases UML: clases, interfaces y las seis relaciones que usa este libro.
- Crear un proyecto Gradle con Java 21, wrapper y entrada por teclado.
- Organizar el código en tres paquetes (`app`, `engine`, `ui`) con un motor que no imprime.
- Dar el primer paso del proyecto: un combate por turnos jugable, sin patrones, y localizar sus olores.

Un patrón de diseño no se aprende como se aprende una librería: se aprende a reconocer el dolor que lo pide. Por eso este capítulo no enseña ningún patrón. Primero construyes Emberdeep a la manera más directa, con `if` y listas, y después miras con calma dónde empieza a doler. Cada capítulo siguiente cura uno de esos dolores con un patrón. Por el camino aprendes lo que necesitas para leer el resto del libro: la ficha de cada patrón, los diagramas UML y la estructura del proyecto.

1.1 Qué es un patrón (y qué no)

Un *patrón de diseño* es una solución con nombre a un problema que se repite al diseñar software orientado a objetos. Tiene tres partes: el problema (cuándo aparece), la solución (qué piezas colaboran y cómo) y las consecuencias (qué ganas y qué pagas). La idea viene de la arquitectura de edificios, y Beck y Cunningham la llevaron a la programación orientada a objetos en 1987 [1]. Los cuatro autores del catálogo que seguimos en este libro lo presentaron primero en un artículo sobre abstracción y reutilización del diseño [6] y después en el libro que lo hizo famoso [7], con 23 patrones. Aquí estudiamos 17, los que Emberdeep necesita de verdad.

El nombre es la parte más útil en el trabajo diario. En una *code review*, decir «esto pide un Strategy» comunica en tres palabras lo que de otro modo llevaría un párrafo, y obliga a defender la propuesta con los términos del propio patrón.

Un patrón **no** es:

- **Una librería ni código para copiar.** Cada patrón se escribe de forma distinta según el lenguaje y el problema. En Java 21 muchos son más cortos que hace veinte años gracias a los *records*, las interfaces `sealed` y las lambdas.
- **Un objetivo.** Un programa no es mejor por llevar más patrones. Cada uno añade clases e indirección, y esa indirección solo compensa si resuelve un problema que ya tienes.
- **Un sustituto de entender el problema.** El patrón es la respuesta; si no sabes cuál es la pregunta, lo aplicarás donde no toca.

Patrón sin dolor es sobrediseño

La regla de este libro: primero el olor (el síntoma en el código), después el patrón. Si no puedes señalar la línea que duele, todavía no necesitas el patrón.

Para una primera lectura más ligera

Si quieres una explicación muy visual y motivada por problemas, *Head First Design Patterns* [5] cubre buena parte del catálogo con otro estilo. Este libro es más breve por patrón y busca que puedas volver a él para consultar.

1.2 Cómo se lee cada capítulo

El capítulo «Cómo usar este libro» describe la ficha de cada patrón. Aquí la tienes como mapa de consulta: qué pregunta contesta cada apartado y cuándo lo usarás.

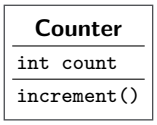
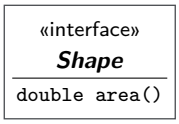
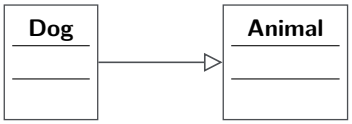
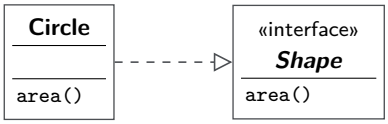
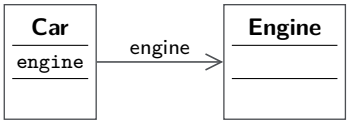
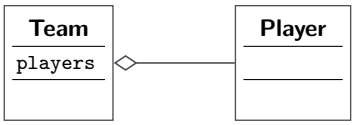
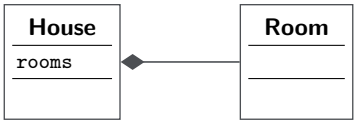
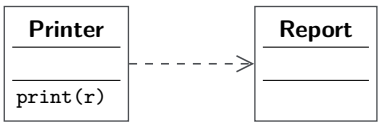
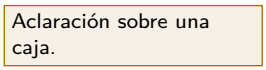
Apartado	Pregunta que contesta	Lo usas para
El olor que pide X	¿Qué síntoma del código lo pide?	Decidir si lo necesitas.
Piezas del patrón X	¿Qué roles hay y cómo se relacionan?	Recordarlo de un vistazo (tabla y UML).
El ejemplo de X, pieza a pieza	¿Cómo se escribe en Java 21?	Copiar la forma, no el código.
X: cuándo sí y cuándo no	¿Compensa aquí?	Defenderlo en una code review.
Contraste	¿En qué se diferencia del patrón parecido?	No confundirlos.
En el proyecto: Hn con X	¿Cómo avanza Emberdeep?	Practicar con el ejercicio de proyecto.

Este capítulo es la excepción: no tiene patrón, así que solo tiene el proyecto y el anticipo de los olores. Cada ejercicio indica con qué se parte. Los que no son de tipo *proyecto* usan clases propias y no tocan Emberdeep; solo el ejercicio de proyecto cambia el código del juego, y su solución es el punto de partida del capítulo siguiente.

1.3 Leer un diagrama de clases UML

Cada patrón lleva un diagrama de clases para verlo de un vistazo. La notación completa está en la especificación de UML [13]; para leer los diagramas de este libro basta una guía breve [4], y aquí tienes la tuya. Los diagramas se dibujan con TikZ [17] y tú solo los lees. Muestran lo relevante para el patrón, no todos los miembros de cada clase.

La tabla recoge todos los símbolos, con su equivalente en Java. Las cajas de ejemplo no son de Emberdeep: son clases mínimas para ver el símbolo solo.

Dibujo	Significa	En Java
	**Clase**: nombre, atributos y métodos. Un compartimento vacío no tiene miembros.	`class Counter { int count; void increment() }`
	**Interfaz**: «interface» y nombre en cursiva; solo métodos.	`interface Shape { double area(); }`
	**Herencia**: línea continua con triángulo hueco hacia la clase base.	`class Dog extends Animal`
	**Realización**: línea discontinua con triángulo hueco hacia la interfaz.	`class Circle implements Shape`
	**Asociación**: flecha abierta; la clase de origen guarda una referencia.	`private final Engine engine;`
	**Agregación**: rombo hueco en el todo; las partes vienen de fuera y viven solas.	`Team(List<Player> players)`: el equipo recibe jugadores ya creados.
	**Composición**: rombo lleno en el todo; las partes nacen y mueren con él.	`rooms.add(new Room())` dentro de `House`.
	**Dependencia**: flecha discontinua; se usa un tipo sin guardarlo.	`void print(Report r)`: `Report` solo es un parámetro.
	**Nota**: caja amarilla con un comentario.	Un comentario `//` en el código.

Cuatro reglas para leerlos:

1. Lo abstracto (interfaces y clases base) suele ir arriba; sus implementaciones, debajo.
2. La punta de la flecha señala lo que se conoce o se usa. Quien está en la cola *depende* de quien está en la punta: si cambia la punta, puede romperse la cola.
3. En agregación y composición el rombo está en el todo, y el todo conoce a sus partes.
4. Cuando una caja de interfaz aparece con varias flechas de realización, esas clases son intercambiables para quien solo conoce la interfaz. Esa es la razón de ser de casi todos los patrones.

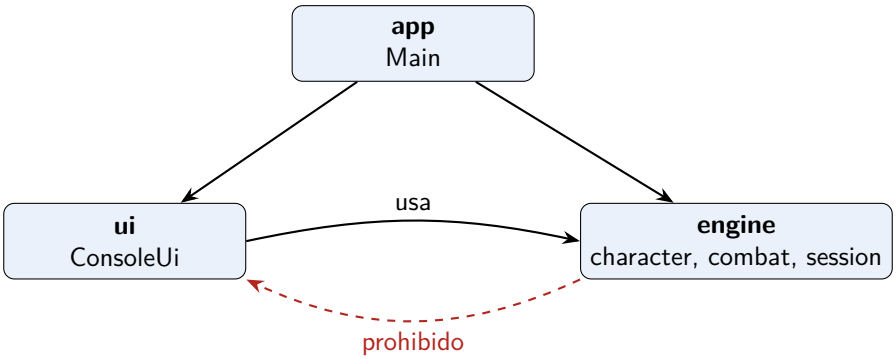
Lo practicarás con el diagrama del núcleo de Emberdeep (sección 1.5) y en el ejercicio 1.1.

1.4 Emberdeep y su estructura de carpetas

1.4.1 Capas arriba, funcionalidad dentro del motor

El código de Emberdeep se organiza con un único principio: **capas arriba, funcionalidad dentro del motor**. Hay tres paquetes de primer nivel:

- `engine`: las reglas del juego (héroes, combate, sesión). El motor no imprime, no lee de la consola y no importa nada de `ui`.
- `ui`: lo que el jugador ve y teclea. Depende del motor.
- `app`: el punto de arranque. Conoce a los dos y los une.



Dependencias entre paquetes: la flecha va de quien depende a quien es usado.

Dentro de `engine` se agrupa por funcionalidad del juego (`character`, `combat`, `session`...), nunca por patrón: el patrón vive en el paquete del problema que resuelve. Así se encuentra el código sabiendo qué hace, no cómo está hecho. La razón de que el motor no imprima es práctica: devuelve mensajes y datos, y quien quiera pintarlos (la consola hoy, otra salida mañana) lo hace desde `ui`.

1.4.2 Composition root

Emberdeep no usa ningún framework que cree objetos por ti. Las dependencias se pasan a mano por el constructor, y hay un único lugar donde se montan todas: el método `main`. Ese lugar se llama *composition root* (raíz de composición): crea los objetos, los conecta y arranca el juego. Ninguna otra clase hace `new` de sus colaboradores principales. Es una convención, no un patrón del catálogo, y te será útil en todos los capítulos.

1.4.3 Un proyecto Gradle

Emberdeep se construye con Gradle (DSL Groovy). Ya tienes Gradle instalado; lo que haces aquí es crear el proyecto: dos archivos de configuración a mano y el *wrapper*, un pequeño script que descarga y usa la versión de Gradle que fija el proyecto, para que cualquiera lo compile igual.

☞ Proyecto Gradle con Java 21 y wrapper

1. Carpetas. Crea la raíz del proyecto y los paquetes de la sección anterior:

```
$ mkdir -p emberdeep/src/main/java/emberdeep/{app,engine/character,engine/
    ↪ combat,engine/session,ui}
$ cd emberdeep
```

2. `settings.gradle`. Solo da nombre al proyecto:

settings.gradle

```
rootProject.name = 'emberdeep'
```

3. `build.gradle`. Aplica el plugin `application`, fija Java 21 y conecta el teclado:

build.gradle

```
plugins {
    id 'application'
}

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

application {
    mainClass = 'emberdeep.app.Main'
}

run {
    standardInput = System.in
}
```

4. Wrapper. Con el Gradle instalado, genera el wrapper fijando la versión:

```
$ gradle wrapper --gradle-version 9.8.2
$ ./gradlew --version
```

La segunda orden debe mostrar `Gradle 9.8.2`. La primera vez, `./gradlew` descarga esa versión (hace falta conexión). Se crean `gradlew`, `gradlew.bat` y la carpeta `gradle/wrapper/`.

Qué hace cada pieza:

- `id 'application'` añade la tarea `run` y el bloque `application`, donde `mainClass` indica la clase con el método `main`. Sin él, Gradle no sabe qué ejecutar.
- `java { toolchain { ...} }` pide compilar y ejecutar con Java 21 aunque el JDK por defecto de tu máquina sea otro. Gradle lo recomienda para evitar que el proyecto se compile con versiones distintas [8]. Con JDK 21 instalado, Gradle lo encuentra solo.
- `run { standardInput = System.in }`: por defecto Gradle ejecuta el programa con una entrada vacía (lo dice la referencia de la tarea `JavaExec` en su documentación). Sin esta línea, el programa no puede leer lo que escribes. Es imprescindible en un juego de consola.
- El wrapper (`gradlew`) es la forma recomendada de ejecutar Gradle en un proyecto [9].

Para generarlo necesitas un Gradle instalado que pueda ejecutarse con Java 21 (a partir de la 8.5, según el manual).

Carpetas que crea Gradle

Al ejecutar, Gradle crea `.gradle/` y `build/` dentro del proyecto. Son archivos generados: si usas Git, no los añadas al repositorio. No aparecen en los árboles de este libro.

Leer del teclado con Scanner. Para leer lo que escribe el jugador usarás `java.util.Scanner`, que ya viene en el JDK. `new Scanner(System.in)` crea un lector sobre la entrada estándar, y su método `nextLine()` devuelve la siguiente línea escrita, sin el salto final. Si no hay más entrada (por ejemplo, la entrada vacía de Gradle sin `standardInput`), lanza `NoSuchElementException`. Lo probarás en el ejercicio 1.2.

1.5 En el proyecto: H1, el núcleo sin patrones

El hito H1 es un combate por turnos jugable en consola, escrito a la manera más directa. Estas son las reglas, deliberadamente mínimas:

- Tres héroes contra los tres enemigos de la planta 1. Cada combatiente tiene vida máxima, ataque y defensa.
- Una ronda: cada héroe vivo actúa en orden (hoy solo puede *atacar* a un enemigo vivo que elige el jugador) y después actúan los enemigos vivos.
- Daño = ataque del atacante − defensa del defensor, con un mínimo de 1.
- Gana quien deje sin vida a todos los del otro bando.

1.5.1 Las clases de datos y Fighter

Las definiciones de héroe y enemigo son *records*: datos inmutables. Lo que cambia durante el combate (la vida actual) vive en `Fighter`, una clase mutable que envuelve a un héroe o a un enemigo. Los cuatro archivos van en `engine/character/`:

`Stats.java`

```
1 package emberdeep.engine.character;
2
3 public record Stats(int maxHp, int attack, int defense) {}
```

`Hero.java`

```
1 package emberdeep.engine.character;
2
3 public record Hero(String name, Stats stats) {}
```

`Enemy.java`

```
1 package emberdeep.engine.character;
2
3 public record Enemy(String name, Stats stats) {}
```

Fighter.java

```java
package emberdeep.engine.character;

public class Fighter {
    private final String name;
    private final Stats stats;
    private int hp;

    private Fighter(String name, Stats stats) {
        this.name = name;
        this.stats = stats;
        this.hp = stats.maxHp();
    }

    public static Fighter ofHero(Hero hero) {
        return new Fighter(hero.name(), hero.stats());
    }

    public static Fighter ofEnemy(Enemy enemy) {
        return new Fighter(enemy.name(), enemy.stats());
    }

    public String name() { return name; }
    public Stats stats() { return stats; }
    public int hp() { return hp; }
    public boolean isAlive() { return hp > 0; }

    public void takeDamage(int amount) {
        hp = Math.max(0, hp - amount);
    }
}
```

El constructor es privado y hay dos fábricas estáticas con nombre, `ofHero` y `ofEnemy`, para que se vea desde dónde se crea cada combatiente. La vida nunca baja de 0, y `isAlive()` es lo único que el resto del motor necesita preguntar.

1.5.2 El resto del núcleo

El diagrama muestra cómo se relacionan las siete clases del hito. Léelo con las reglas de la sección anterior: `ConsoleUi` usa a `GameSession` sin guardarla (dependencia); `GameSession` guarda su `Combat` (asociación); `Combat` agrupa a los `Fighter` que recibe ya creados (agregación), y cada uno de ellos, igual que `Hero` y `Enemy`, guarda unas `Stats` (asociación).

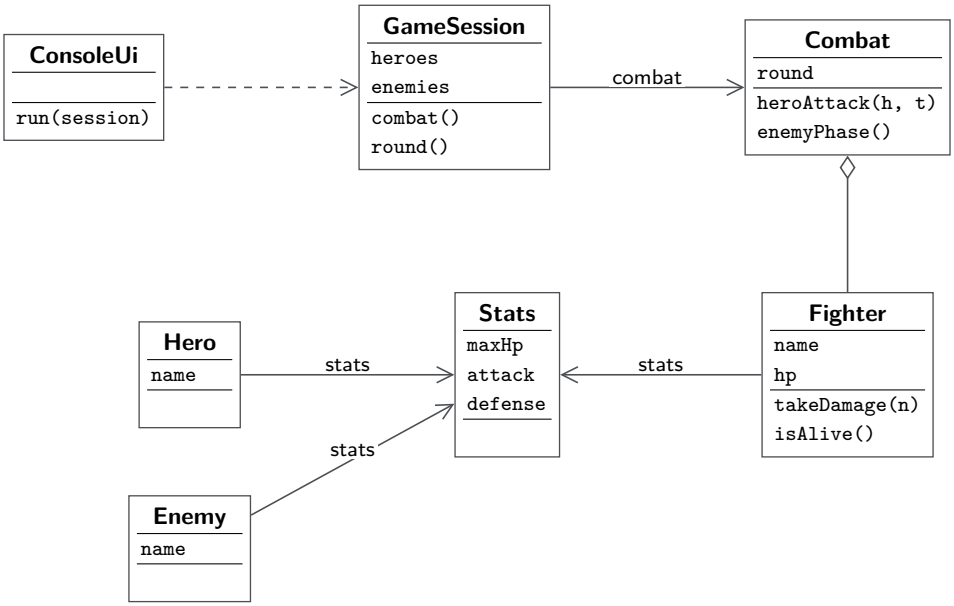


Diagrama UML del núcleo de Emberdeep tras el hito H1 (sin patrones).

Las clases que faltan las escribes tú en el ejercicio 1.5. Esta tabla dice qué hace cada una y con qué firma, para que encajen entre sí:

Clase	Paquete	Qué hace
`Combat`	`engine.combat`	Reglas del combate y contador de ronda `round()`. No imprime: devuelve `List<String>` con los mensajes.
`GameSession`	`engine.session`	Guarda los tres héroes, los enemigos de la planta 1 y su `Combat`; `round()` se delega en el combate.
`ConsoleUi`	`ui`	Pinta el estado, muestra el menú «[1] atacar», lee con `Scanner` y escribe los mensajes.
`Main`	`app`	Composition root: crea los datos, la sesión y la interfaz, y arranca.

La decisión de qué hace cada enemigo en su turno es la parte que más importa del hito, y la verás pronto como un problema. Cada enemigo actúa según su nombre:

`Combat.java (fragmento: el turno de los enemigos)`

```
1  public List<String> enemyPhase() {
2      List<String> messages = new ArrayList<>();
3      for (Fighter enemy : enemies) {
4          if (!enemy.isAlive()) {
5              continue;
6          }
7          if (heroesLost()) {
8              break;
9          }
10         Fighter target;
11         int damage;
12         if (enemy.name().equals("Goblin")) {
13             target = firstAlive(heroes);
14             damage = damageOf(enemy, target);
15         } else if (enemy.name().equals("Rata de ceniza")) {
```

```
            target = weakest(heroes);
            damage = damageOf(enemy, target);
        } else if (enemy.name().equals("Esqueleto")) {
            target = firstAlive(heroes);
            damage = 3;
        } else {
            target = firstAlive(heroes);
            damage = damageOf(enemy, target);
        }
        hit(enemy, target, damage, messages);
    }
    round++;
    return messages;
}
```

Cuatro auxiliares privados completan la clase: `damageOf(atacante, defensor)` calcula `max(1, ataque - defensa)`, `firstAlive(grupo)` devuelve el primer combatiente vivo, `weakest(grupo)` el vivo con menos vida y `hit(...)` aplica el daño y añade a la lista los mensajes del golpe. El goblin ataca al primer héroe vivo, la rata de ceniza al más débil y el esqueleto golpea con un daño fijo de 3 que ignora la defensa.

Hito 1: el núcleo sin patrones

Emberdeep nace aquí, desde cero. Al terminar el ejercicio 1.5 tienes un combate completo con `./gradlew run`: tres héroes contra Goblin, Rata de ceniza y Esqueleto, con un menú de una opción. Esta es la estructura:

Estructura de carpetas tras el capítulo 1

```
emberdeep/
|-- settings.gradle
|-- build.gradle
|-- gradlew
|-- gradlew.bat
|-- gradle/wrapper/
`-- src/main/java/emberdeep/
    |-- app/
    |   `-- Main.java              (composition root)
    |-- engine/
    |   |-- character/             Stats, Hero, Enemy, Fighter
    |   |-- combat/                Combat
    |   `-- session/               GameSession
    `-- ui/
        `-- ConsoleUi.java
```

1.6 Seis olores que ya se ven en el núcleo

El núcleo funciona, y ese es el punto: casi nada del código es «incorrecto». Duele cuando quieras cambiarlo. Estos seis síntomas ya están ahí, y cada uno tiene su capítulo. No hace falta entender los patrones todavía; basta con que reconozcas el síntoma.

Dónde se ve	Qué duele	Lo cura (capítulo)
La cadena de `if` por nombre en `enemyPhase`	Cada enemigo nuevo es otra rama en `Combat`, y el comportamiento depende de un texto.	Strategy (cap. 2)
`new Enemy(...)` escrito a mano en `Main`	Otra planta con otros enemigos obliga a repetir y cambiar `Main`.	Factory Method y Abstract Factory (cap. 3)
`new Hero("Aria", new Stats(30, 8, 3))`	Tres números sin nombre; con más opciones nadie recuerda el orden.	Builder (cap. 4)
`Stats` fijas durante el combate	El equipo y las mejoras temporales cambiarán el valor final del ataque o la defensa.	Decorator (cap. 6)
La acción «atacar» metida en `heroAttack` y en el menú	Deshacer o repetir una partida exige tratar cada acción como un objeto.	Command (cap. 7)
`Combat` devuelve `List<String>` ya redactada	Un HUD o un logro no pueden reaccionar a «hizo 7 de daño» sin leer texto.	Observer (cap. 9)

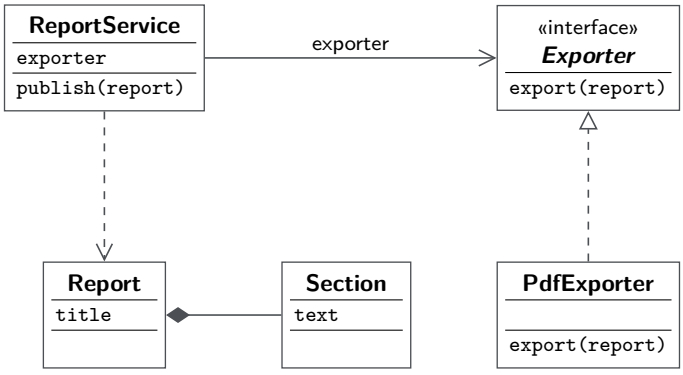
Qué no se cura

No todo en este núcleo es un olor. Que `Fighter` sea mutable y los *records* no lo sean, o que `Main` haga `new` de todo, es una decisión razonable por ahora. Aprender patrones también es aprender a dejar sin patrón lo que no duele.

1.7 Ejercicios

Ejercicio 1.1: Lee un diagrama de cinco clases ★☆☆☆☆

Este diagrama no es de Emberdeep. Contesta con una frase cada pregunta.



1. ¿Cuál de las cinco es una interfaz y cómo lo sabes?
2. ¿Qué relación hay entre `PdfExporter` y `Exporter`? ¿Qué palabra de Java usarías?
3. ¿Y entre `ReportService` y `Exporter`? ¿Qué tendría `ReportService` en su código?
4. ¿Y entre `ReportService` y `Report`? ¿En qué se diferencia de la anterior?
5. ¿Y entre `Report` y `Section`? Si descartas un `Report`, ¿qué pasa con sus `Section`?

6. ¿Quién depende de quién? Escribe las dependencias del diagrama y di si `ReportService` conoce a `PdfExporter`.

Está bien si cada respuesta nombra la relación y la regla de la leyenda en la que te apoyas.

Lectura de código ▪ solución en la pág. 15

Ejercicio 1.2: Un Gradle mínimo que lee del teclado ★☆☆☆☆

Crea un proyecto Gradle llamado `hola`, fuera de `emberdeep/`, que pregunte cómo te llamas y te salude.

1. Crea las carpetas con `mkdir -p hola/src/main/java/demo` y entra en `hola/`.
2. Crea `settings.gradle` con `rootProject.name = 'hola'`.
3. Crea `build.gradle` como el de la instalación de este capítulo, pero con `mainClass = 'demo.Main'` y con el bloque `run` que conecta `System.in`.
4. Crea `src/main/java/demo/Main.java` (paquete `demo`). Muestra «¿Cómo te llamas?», lee el nombre con `new Scanner(System.in).nextLine()` y muestra «Hola, » seguido del nombre.
5. Genera el wrapper y ejecuta el programa:

```
$ gradle wrapper --gradle-version 9.8.2
$ ./gradlew run
```

6. Borra el bloque `run` de `build.gradle`, ejecuta de nuevo y fíjate en el error.

Está bien si en el paso 5 escribes tu nombre y el programa te saluda, y si en el paso 6 el programa falla con una excepción en lugar de esperar tu texto.

Guiado ▪ solución en la pág. 15

Ejercicio 1.3: Tres olores en el turno de los enemigos ★★☆☆☆

Lee este fragmento de `Combat` (el turno de un enemigo, sección 1.5). Los auxiliares `firstAlive`, `weakest` y `damageOf` devuelven el primer héroe vivo, el vivo con menos vida y el daño máx$(1, \text{ataque} - \text{defensa})$.

```
Fighter target;
int damage;
if (enemy.name().equals("Goblin")) {
    target = firstAlive(heroes);
    damage = damageOf(enemy, target);
} else if (enemy.name().equals("Rata de ceniza")) {
    target = weakest(heroes);
    damage = damageOf(enemy, target);
} else if (enemy.name().equals("Esqueleto")) {
    target = firstAlive(heroes);
    damage = 3;
} else {
    target = firstAlive(heroes);
    damage = damageOf(enemy, target);
}
```

Señala tres olores concretos que harán sufrir al añadir enemigos nuevos. Para cada uno indica qué línea o qué expresión lo muestra y qué habría que cambiar si llega un «Murciélago» que ataca al héroe con menos defensa. Está bien si los tres olores son distintos entre sí y cada uno apunta a código del fragmento.

Lectura de código ▪ solución en la pág. 16

Ejercicio 1.4: Reparte las clases en paquetes ★★☆☆☆

Tienes diez clases de un juego como Emberdeep: `Main`, `Stats`, `Hero`, `Enemy`, `Fighter`, `Combat`, `GameSession`, `ConsoleUi`, `MenuReader` (lee y valida números del teclado) y `AnsiColors` (da color al texto en la consola).

1. Escribe una tabla con el paquete de cada clase: `app`, `engine.character`, `engine.combat`, `engine.session` o `ui`.
2. Esta versión de `Combat` rompe la regla «el motor no importa `ui`». Corrígela sin que el motor conozca la interfaz (supón que `ConsoleUi` tiene un método `print(String)`):

`Combat.java (versión con el defecto)`

```
package emberdeep.engine.combat;

import emberdeep.engine.character.Fighter;
import emberdeep.ui.ConsoleUi;

public class Combat {
    private final ConsoleUi ui;

    public Combat(ConsoleUi ui) {
        this.ui = ui;
    }

    public void attack(Fighter attacker, Fighter target) {
        int damage = Math.max(1, attacker.stats().attack() - target.stats().
    ↪ defense());
        target.takeDamage(damage);
        ui.print(attacker.name() + " ataca a " + target.name() + " (" +
    ↪ damage + ")");
    }
}
```

Está bien si ninguna clase de `engine` importa nada de `ui` y el mensaje sigue llegando a la pantalla.

Práctica ▪ solución en la pág. 17

Ejercicio 1.5: Hito 1: el núcleo jugable de Emberdeep ★★★☆☆

Implementa el hito H1 siguiendo la estructura de este capítulo.

- Parte de la carpeta `emberdeep/` con los archivos de Gradle de la instalación, el wrapper generado y los cuatro archivos de `engine/character/` de la sección 1.5.
- Escribe `Combat`, `GameSession`, `ConsoleUi` y `Main` con las firmas de la tabla de la sección. `Combat` tiene un constructor `Combat(List<Fighter> heroes, List<Fighter> enemies)`, `heroAttack(hero, target)` y el turno de los enemigos del fragmento, cuyos mensajes tienen el formato «Aria ataca a Goblin y le hace 7 de daño.» y, si el

defensor cae, «Goblin queda fuera de combate.».

- `ConsoleUi(Scanner in, PrintStream out)` tiene `run(GameSession)`: en cada turno de héroe muestra la vida de ambos bandos, el menú «[1] atacar» y la lista de enemigos vivos numerada, y vuelve a preguntar si la entrada no es un número válido.
- `Main` crea los héroes Aria (30 de vida, 8 de ataque, 3 de defensa), Borin (40, 6, 5) y Cael (24, 9, 2), y los enemigos Goblin (14, 6, 1), Rata de ceniza (8, 5, 0) y Esqueleto (18, 7, 2).

Está bien si:

1. `./gradlew run` compila y muestra el menú.
2. Tecleando siempre `1`, la primera ronda muestra estos mensajes y la partida acaba en la ronda 3 con «¡Victoria!»:

   Mensajes de la primera ronda

   ```
   Aria ataca a Goblin y le hace 7 de daño.
   Borin ataca a Goblin y le hace 5 de daño.
   Cael ataca a Goblin y le hace 8 de daño.
   Goblin queda fuera de combate.
   Rata de ceniza ataca a Cael y le hace 3 de daño.
   Esqueleto ataca a Aria y le hace 3 de daño.
   ```

3. Ninguna clase de `engine` importa `emberdeep.ui` ni usa `System.out`.

Pista: *el estado que cambia (la vida) está en `Fighter`; `Combat` y `ConsoleUi` no deben guardar nada que no necesiten.*

Proyecto ▪ solución en la pág. 18

1.8 Resumen

Resumen

- Un patrón es una solución con nombre a un problema recurrente de diseño, con sus consecuencias; no es una librería, ni un objetivo, ni un sustituto de entender el problema.
- Primero el olor y después el patrón: si no puedes señalar la línea que duele, aún no lo necesitas.
- En un diagrama UML la flecha apunta a lo que se conoce o se usa; los rombos están en el todo, y las interfaces con sus realizaciones marcan lo intercambiable.
- Emberdeep tiene tres paquetes: `engine` (reglas, sin imprimir ni importar `ui`), `ui` y `app`. `Main` es el composition root.
- El proyecto Gradle necesita `settings.gradle`, `build.gradle` con `application`, toolchain de Java 21 y `run { standardInput = System.in }`, y el wrapper (versión 9.8.2).
- H1 es un combate jugable sin patrones. Sus olores (el `if` por nombre y otros cinco) son el hilo de los capítulos siguientes.

Soluciones de los ejercicios

Solución del ejercicio 1.1

enunciado en la pág. 11

1. `Exporter`: su caja lleva «interface» y el nombre en cursiva, y solo tiene métodos.
2. Realización (línea discontinua con triángulo hueco hacia la interfaz): `class PdfExporter implements Exporter`.
3. Asociación (flecha abierta con la etiqueta `exporter`): `ReportService` guarda una referencia, un campo `private final Exporter exporter;`. Se asocia a la interfaz, no a una clase concreta.
4. Dependencia (flecha discontinua): `Report` solo aparece como parámetro de `publish(report)`, no se guarda. La anterior era un campo; esta, un uso pasajero.
5. Composición (rombo lleno en `Report`): las `Section` nacen y mueren con su `Report`; si descartas el informe, sus secciones dejan de existir con él.
6. Dependencias del diagrama: `ReportService` depende de `Exporter` y de `Report`; `PdfExporter` depende de `Exporter` (lo realiza); `Report` conoce a sus `Section`. `ReportService` *no* conoce a `PdfExporter`: no hay ninguna flecha entre ambos, y por eso se puede cambiar el exportador sin tocarlo.

Errores típicos: confundir asociación y dependencia (la primera guarda una referencia, la segunda solo usa el tipo) y leer la flecha de realización al revés (la punta apunta a la interfaz, a lo que se promete, no a quien lo cumple).

Solución del ejercicio 1.2

enunciado en la pág. 12

Estructura final:

Estructura de hola/

```
hola/
|-- settings.gradle
|-- build.gradle
|-- gradlew
|-- gradlew.bat
|-- gradle/wrapper/
`-- src/main/java/demo/
    `-- Main.java
```

settings.gradle

```
rootProject.name = 'hola'
```

build.gradle

```
plugins {
    id 'application'
}

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

application {
    mainClass = 'demo.Main'
}

run {
    standardInput = System.in
}
```

src/main/java/demo/Main.java

```
package demo;

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        System.out.println("¿Cómo te llamas?");
        String name = new Scanner(System.in).nextLine();
        System.out.println("Hola, " + name + ".");
    }
}
```

Comandos:

```
$ gradle wrapper --gradle-version 9.8.2
$ ./gradlew run
```

Al ejecutar, escribes tu nombre y ves «Hola, » seguido de él. Al quitar el bloque `run`, Gradle da al programa una entrada vacía y `nextLine()` lanza `NoSuchElementException: No line found`: el programa no espera tu texto.
Errores típicos: la clase está en el paquete `demo` pero `mainClass` dice otra cosa (Gradle no encuentra la clase principal); olvidar el `import java.util.Scanner`; ejecutar `./gradlew` desde fuera de la carpeta del proyecto.

Solución del ejercicio 1.3 enunciado en la pág. 12

1. **Cadena de `if` por nombre** (`enemy.name().equals("Goblin")...`). Cada enemigo nuevo es otra rama que obliga a editar `Combat`, una clase que no tendría por qué conocer a los enemigos. El «Murciélago» necesita una rama más. Además el comportamiento depende de un texto: renombrar «Goblin» a «Goblin chamán» cambia en silencio lo que hace, porque cae en el `else` final.

2. **Elección del objetivo repetida dentro de cada rama** (`target = firstAlive(heroes)` aparece tres veces). La política de objetivo está mezclada con la cadena de `if`, y para el «Murciélago» habría que escribir una función nueva (el héroe con menos defensa) y otra rama que la use.
3. **Daño fijo como número mágico** (`damage = 3` del esqueleto). Una regla del juego queda enterrada en un `if` de `Combat`, y la fórmula normal (`damageOf`) está duplicada en tres ramas. Si cambia la regla de daño, hay que recordar todas las líneas.

Otros olores válidos: el `else` final trata cualquier enemigo desconocido como un goblin sin avisar, y `enemyPhase` mezcla en un método elegir al enemigo, elegir el objetivo, calcular el daño y avanzar la ronda. Cualquier respuesta es correcta si apunta a una expresión concreta del fragmento y explica el coste de añadir un enemigo.
Error típico: señalar el `for` o los helpers; el dolor no está en recorrer la lista, sino en la decisión por nombre.

Solución del ejercicio 1.4 enunciado en la pág. 13

Clase	Paquete	Motivo
`Main`	`app`	Composition root: crea y conecta.
`Stats`, `Hero`, `Enemy`, `Fighter`	`engine.character`	Datos y estado de los combatientes.
`Combat`	`engine.combat`	Reglas del combate.
`GameSession`	`engine.session`	Estado de la partida.
`ConsoleUi`	`ui`	Pinta y lee de la consola.
`MenuReader`	`ui`	Lee y valida el teclado.
`AnsiColors`	`ui`	Presentación (colores).

La importación defectuosa se arregla haciendo que el motor *devuelva* los mensajes en lugar de imprimirlos; quien los pinta es la `ui`:

`Combat.java (corregida)`

```
package emberdeep.engine.combat;

import emberdeep.engine.character.Fighter;
import java.util.List;

public class Combat {
    public List<String> attack(Fighter attacker, Fighter target) {
        int damage = Math.max(1, attacker.stats().attack() - target.stats().
    ↪ defense());
        target.takeDamage(damage);
        return List.of(attacker.name() + " ataca a " + target.name() + " ("
    ↪  + damage + ")");
    }
}
```

Y en `ConsoleUi`, la llamada pasa a ser `combat.attack(a, b).forEach(this::print);` (con `print(String)` como método de la propia interfaz de usuario).
Errores típicos: mover `ConsoleUi` a `engine` para que el import sea legal (esconde el problema: el motor sigue atado a la consola); dejar el `import` sin usar; poner `Stats` en `ui`

por «ser datos». Una alternativa válida es una interfaz de salida definida en `engine` que `ui` implementa, pero para H1 es más de lo que el problema pide.

Solución del ejercicio 1.5 enunciado en la pág. 13

Parte de la carpeta con los archivos de Gradle, el wrapper y los cuatro archivos de `engine/character/` del capítulo (`Stats`, `Hero`, `Enemy` y `Fighter`), que se repiten aquí para que la solución esté completa.

engine/character/Stats.java, Hero.java y Enemy.java

Stats.java

```java
package emberdeep.engine.character;

public record Stats(int maxHp, int attack, int defense) {}
```

Hero.java

```java
package emberdeep.engine.character;

public record Hero(String name, Stats stats) {}
```

Enemy.java

```java
package emberdeep.engine.character;

public record Enemy(String name, Stats stats) {}
```

engine/character/Fighter.java

Fighter.java

```java
package emberdeep.engine.character;

public class Fighter {
    private final String name;
    private final Stats stats;
    private int hp;

    private Fighter(String name, Stats stats) {
        this.name = name;
        this.stats = stats;
        this.hp = stats.maxHp();
    }

    public static Fighter ofHero(Hero hero) {
        return new Fighter(hero.name(), hero.stats());
    }

    public static Fighter ofEnemy(Enemy enemy) {
        return new Fighter(enemy.name(), enemy.stats());
    }

    public String name() { return name; }
    public Stats stats() { return stats; }
    public int hp() { return hp; }
    public boolean isAlive() { return hp > 0; }

    public void takeDamage(int amount) {
        hp = Math.max(0, hp - amount);
    }
}
```

engine/combat/Combat.java (en tres tramos seguidos, dentro de la misma clase)

Combat.java (1/3: estado y turno de los héroes)

```java
package emberdeep.engine.combat;

import emberdeep.engine.character.Fighter;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

public class Combat {
    private final List<Fighter> heroes;
    private final List<Fighter> enemies;
    private int round = 1;

    public Combat(List<Fighter> heroes, List<Fighter> enemies) {
        this.heroes = heroes;
        this.enemies = enemies;
    }

    public int round() {
        return round;
    }

    public boolean heroesWon() {
        return enemies.stream().noneMatch(Fighter::isAlive);
    }

    public boolean heroesLost() {
        return heroes.stream().noneMatch(Fighter::isAlive);
    }

    public boolean isOver() {
        return heroesWon() || heroesLost();
    }

    public List<String> heroAttack(Fighter hero, Fighter target) {
        List<String> messages = new ArrayList<>();
        hit(hero, target, damageOf(hero, target), messages);
        return messages;
    }
```

Combat.java (2/3: turno de los enemigos)

```
    public List<String> enemyPhase() {
        List<String> messages = new ArrayList<>();
        for (Fighter enemy : enemies) {
            if (!enemy.isAlive()) {
                continue;
            }
            if (heroesLost()) {
                break;
            }
            Fighter target;
            int damage;
            if (enemy.name().equals("Goblin")) {
                target = firstAlive(heroes);
                damage = damageOf(enemy, target);
            } else if (enemy.name().equals("Rata de ceniza")) {
                target = weakest(heroes);
                damage = damageOf(enemy, target);
            } else if (enemy.name().equals("Esqueleto")) {
                target = firstAlive(heroes);
                damage = 3;
            } else {
                target = firstAlive(heroes);
                damage = damageOf(enemy, target);
            }
            hit(enemy, target, damage, messages);
        }
        round++;
        return messages;
    }
```

Combat.java (3/3: auxiliares privados y llave final)

```
    private void hit(Fighter attacker, Fighter target, int damage, List<String> messages) {
        target.takeDamage(damage);
        messages.add(attacker.name() + " ataca a " + target.name()
                + " y le hace " + damage + " de daño.");
        if (!target.isAlive()) {
            messages.add(target.name() + " queda fuera de combate.");
        }
    }

    private static int damageOf(Fighter attacker, Fighter target) {
        return Math.max(1, attacker.stats().attack() - target.stats().defense());
    }

    private static Fighter firstAlive(List<Fighter> group) {
        return group.stream().filter(Fighter::isAlive).findFirst().orElseThrow();
    }

    private static Fighter weakest(List<Fighter> group) {
        return group.stream().filter(Fighter::isAlive)
                .min(Comparator.comparingInt(Fighter::hp)).orElseThrow();
    }
}
```

engine/session/GameSession.java

GameSession.java

```java
package emberdeep.engine.session;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.combat.Combat;
import java.util.List;

public class GameSession {
    private final List<Fighter> heroes;
    private final List<Fighter> enemies;
    private final Combat combat;

    public GameSession(List<Hero> heroes, List<Enemy> floorOneEnemies) {
        this.heroes = heroes.stream().map(Fighter::ofHero).toList();
        this.enemies = floorOneEnemies.stream().map(Fighter::ofEnemy).
    ↪ toList();
        this.combat = new Combat(this.heroes, this.enemies);
    }

    public List<Fighter> heroes() { return heroes; }
    public List<Fighter> enemies() { return enemies; }
    public Combat combat() { return combat; }
    public int round() { return combat.round(); }
}
```

ui/ConsoleUi.java (en dos tramos seguidos, dentro de la misma clase)

ConsoleUi.java (1/2: partida)

```java
package emberdeep.ui;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.session.GameSession;
import java.io.PrintStream;
import java.util.List;
import java.util.Scanner;
import java.util.stream.Collectors;

public class ConsoleUi {
    private final Scanner in;
    private final PrintStream out;

    public ConsoleUi(Scanner in, PrintStream out) {
        this.in = in;
        this.out = out;
    }

    public void run(GameSession session) {
        Combat combat = session.combat();
        out.println("=== Emberdeep: planta 1 ===");
        while (!combat.isOver()) {
            out.println("--- Ronda " + session.round() + " ---");
            for (Fighter hero : session.heroes()) {
                if (!hero.isAlive() || combat.isOver()) {
                    continue;
                }
                out.println("Héroes:   " + describe(session.heroes()));
                out.println("Enemigos: " + describe(session.enemies()));
                out.println("Turno de " + hero.name() + ". [1] atacar");
                ask(1);
                Fighter target = askTarget(session.enemies());
                print(combat.heroAttack(hero, target));
            }
            if (!combat.isOver()) {
                print(combat.enemyPhase());
            }
        }
        out.println(combat.heroesWon() ? "¡Victoria!" : "Derrota...");
    }
```

ConsoleUi.java (2/2: auxiliares y llave final)

```
    private String describe(List<Fighter> group) {
        return group.stream()
                .map(f -> f.name() + " " + f.hp() + "/" + f.stats().maxHp())
                .collect(Collectors.joining(", "));
    }

    private void print(List<String> messages) {
        messages.forEach(out::println);
    }

    private int ask(int max) {
        while (true) {
            out.print("> ");
            out.flush();
            String line = in.nextLine().trim();
            try {
                int n = Integer.parseInt(line);
                if (n >= 1 && n <= max) {
                    return n;
                }
            } catch (NumberFormatException e) {
                // no es un número: se avisa más abajo
            }
            out.println("Escribe un número entre 1 y " + max + ".");
        }
    }

    private Fighter askTarget(List<Fighter> enemies) {
        List<Fighter> alive = enemies.stream().filter(Fighter::isAlive).
toList();
        StringBuilder menu = new StringBuilder("Objetivo:");
        for (int i = 0; i < alive.size(); i++) {
            menu.append("  [").append(i + 1).append("] ").append(alive.get(
i).name());
        }
        out.println(menu);
        return alive.get(ask(alive.size()) - 1);
    }
}
```

app/Main.java

Main.java

```java
package emberdeep.app;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.Stats;
import emberdeep.engine.session.GameSession;
import emberdeep.ui.ConsoleUi;
import java.util.List;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        List<Hero> heroes = List.of(
                new Hero("Aria", new Stats(30, 8, 3)),
                new Hero("Borin", new Stats(40, 6, 5)),
                new Hero("Cael", new Stats(24, 9, 2)));
        List<Enemy> floorOne = List.of(
                new Enemy("Goblin", new Stats(14, 6, 1)),
                new Enemy("Rata de ceniza", new Stats(8, 5, 0)),
                new Enemy("Esqueleto", new Stats(18, 7, 2)));
        GameSession session = new GameSession(heroes, floorOne);
        ConsoleUi ui = new ConsoleUi(new Scanner(System.in), System.out);
        ui.run(session);
    }
}
```

Por qué funciona. Los *records* describen; `Fighter` lleva la vida. `Combat` decide y devuelve texto; solo `ConsoleUi` usa `System.out`, y `Main` es el único sitio con `new` de los colaboradores. Con `1` en cada pregunta: ronda 1, los héroes dejan al goblin fuera ($7 + 5 + 8$ de daño sobre 14 de vida), la rata golpea a Cael (el de menos vida, 24) con $5 - 2 = 3$ y el esqueleto a Aria con su 3 fijo. En la ronda 2 Aria acaba con la rata (8 de daño, 8 de vida) y Borin y Cael dejan al esqueleto en 7; el esqueleto golpea a Aria otra vez. En la ronda 3 Aria (6) y Borin (4) lo derrotan y sale «¡Victoria!» sin que Cael llegue a actuar. En el turno de Borin verás `Héroes: Aria 24/30, Borin 40/40, Cael 21/24`.
Estructura final: la del árbol de la sección «En el proyecto». Este es el estado del que parte el capítulo siguiente.

Errores típicos:

- Olvidar el bloque `run` del `build.gradle`: el juego se cierra al primer `nextLine()` con `NoSuchElementException`.
- Que el esqueleto use `damageOf` en lugar del 3 fijo: el mensaje de la ronda 1 diría 4 en vez de 3 (7 de ataque menos 3 de defensa de Aria).
- Importar `emberdeep.ui` desde `engine`, o imprimir con `System.out` en `Combat`: rompe la regla de las capas.
- Dejar que `firstAlive` lance una excepción cuando ya no quedan héroes vivos: por eso `enemyPhase` comprueba `heroesLost()` antes de que actúe cada enemigo.

CAPÍTULO 2

Strategy: IA del enemigo y fórmula de daño

Objetivos del capítulo

- Reconocer el olor que pide Strategy: una cadena de `if` que elige un comportamiento.
- Describir las piezas de Strategy (estrategia, estrategias concretas y contexto) y dibujarlas en UML.
- Sustituir los `if` por nombre del combate por objetos intercambiables (`EnemyAi`).
- Expresar una estrategia como lambda cuando la interfaz tiene un solo método (`DamageFormula`).
- Inyectar un `java.util.Random` con semilla para que las partidas sean repetibles.

En el capítulo anterior terminaste con un combate que funciona y con una cadena de `if` que decide, según el nombre del enemigo, a quién ataca y con cuánta fuerza. Funciona, pero cada enemigo nuevo obliga a abrir `Combat`. En este capítulo curas ese primer dolor con tu primer patrón, Strategy, y aprendes a leer su ficha completa: el olor, las piezas, el ejemplo, el UML y cuándo no usarlo.

2.1 El olor que pide Strategy: los `if` por nombre

Este es el turno de los enemigos tal como lo dejaste en el capítulo 1:

Combat.java (H1, fragmento)

```
Fighter target;
int damage;
if (enemy.name().equals("Goblin")) {
    target = firstAlive(heroes);
    damage = damageOf(enemy, target);
} else if (enemy.name().equals("Rata de ceniza")) {
    target = weakest(heroes);
    damage = damageOf(enemy, target);
} else if (enemy.name().equals("Esqueleto")) {
    target = firstAlive(heroes);
    damage = 3;
} else {
    target = firstAlive(heroes);
    damage = damageOf(enemy, target);
}
```

Mira lo que mezcla esta cadena. Hay **dos decisiones** distintas: *a quién* ataca el enemigo (`firstAlive`, `weakest`) y *cuánto* daño hace (`damageOf`, o un 3 fijo). Las dos dependen de un

texto, el nombre. Y las dos viven dentro de `Combat`, una clase que no debería saber qué enemigos existen.

Si mañana llega un «Murciélago» que ataca al héroe con menos defensa, tienes que: escribir una función nueva, añadir otra rama al `if` y acordarte de que el `else` final tratará como un goblin a cualquier enemigo cuyo nombre escribas mal.

Ese es el olor: **una cadena de `if` o `switch` que elige entre variantes de un mismo comportamiento**. El patrón que lo cura se llama *Strategy*. El catálogo del GoF lo define como una familia de algoritmos, cada uno encapsulado y todos intercambiables, de forma que el algoritmo cambia sin que cambie quien lo usa [7]. Dicho con nuestro código: cada forma de elegir objetivo pasa a ser un objeto, y `Combat` deja de preguntar «¿quién eres?» para decir solo «elige».

Ese tipo de cambio, partir de un código que ya existe y llegar al patrón paso a paso, es justo el enfoque de la refactorización hacia patrones [10], y es el que sigue este libro: primero el dolor, luego la pieza.

2.2 Piezas del patrón Strategy

Strategy tiene tres piezas. La *estrategia* es la interfaz que fija qué se decide. Las *estrategias concretas* son las variantes. El *contexto* es quien guarda una estrategia y le delega la decisión. En Emberdeep hay además una segunda familia, la fórmula de daño, que sigue el mismo esquema.

Rol	Clase en el ejemplo	Propósito	Paquete
Estrategia (interfaz)	`EnemyAi`	Elige objetivo	`engine.combat.ai`
Estrategias concretas (clases)	`FirstAliveAi`; `LowestHpAi` y `RandomTargetAi` (ejercicio)	Un criterio cada una	`engine.combat.ai`
Contexto (clase)	`Fighter`	Consulta la IA de su `Enemy`	`engine.character`
Portador de la estrategia (record)	`Enemy(name, stats, ai)`	Solo guarda la IA	`engine.character`
Segunda estrategia (interfaz funcional)	`DamageFormula`	Calcula el daño	`engine.combat.damage`
Estrategias de la fórmula (clases o lambdas)	`FlatFormula`; `ScaledFormula` (ejercicio)	Una regla de daño cada una	`engine.combat.damage`
Cliente (clase)	`Main`	Elige y monta	`app`

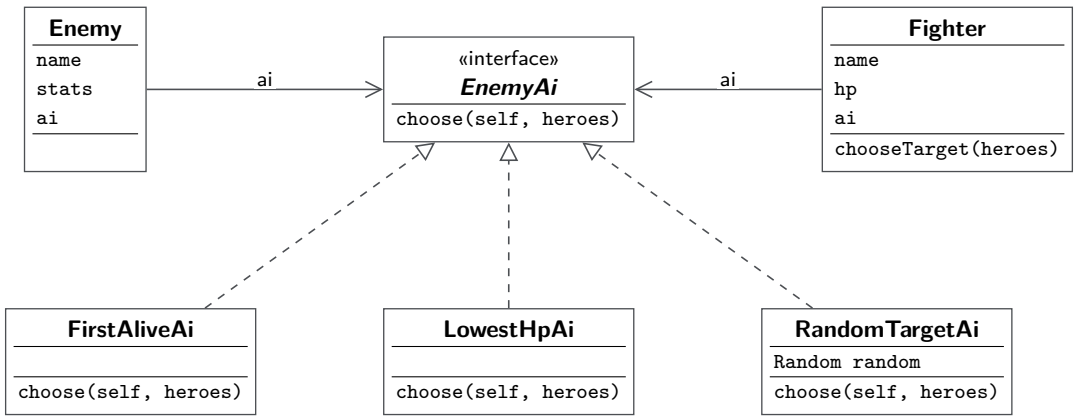


Diagrama UML del patrón Strategy: `Fighter` (contexto) conoce solo la interfaz `EnemyAi`; las tres IA son intercambiables.

Léelo con las reglas del capítulo anterior. La interfaz está arriba, con tres realizaciones debajo: esas tres clases son intercambiables para quien solo conoce `EnemyAi`. Nada del contexto señala a una clase concreta. Esa ausencia de flechas es la ganancia del patrón.

2.3 El ejemplo de Strategy, pieza a pieza: la IA del enemigo

2.3.1 La estrategia: `EnemyAi`

La interfaz dice qué se decide y con qué información. Recibe quien decide (`self`) y la lista de héroes, y devuelve el héroe elegido:

engine/combat/ai/EnemyAi.java

```
1  package emberdeep.engine.combat.ai;
2
3  import emberdeep.engine.character.Fighter;
4  import java.util.List;
5
6  public interface EnemyAi {
7      Fighter choose(Fighter self, List<Fighter> heroes);
8  }
```

Por qué recibe `self`: algunas IA necesitan saber quién las ejecuta (por ejemplo, para decidir según la vida del propio enemigo). Aunque las tres de este capítulo no lo usen, la firma no tendrá que cambiar cuando alguna lo necesite.

2.3.2 Una estrategia concreta: `FirstAliveAi`

Es la lógica del goblin del capítulo anterior, ahora en su propia clase:

engine/combat/ai/FirstAliveAi.java

```
1  package emberdeep.engine.combat.ai;
2
3  import emberdeep.engine.character.Fighter;
4  import java.util.List;
5
6  public class FirstAliveAi implements EnemyAi {
```

```
    @Override
    public Fighter choose(Fighter self, List<Fighter> heroes) {
        return heroes.stream().filter(Fighter::isAlive).findFirst().orElseThrow
    ↪ ();
    }
}
```

No tiene campos: no guarda nada entre llamadas. Una estrategia sin estado se puede compartir entre muchos enemigos con una sola instancia. `orElseThrow()` no es un descuido: `Combat` comprueba `heroesLost()` antes de dar el turno a cada enemigo, así que siempre queda algún héroe vivo.

2.3.3 El portador y el contexto: `Enemy` y `Fighter`

`Enemy` sigue siendo un record de datos y gana un campo, la IA. Cada enemigo *lleva* su estrategia:

engine/character/Enemy.java

```
package emberdeep.engine.character;

import emberdeep.engine.combat.ai.EnemyAi;

public record Enemy(String name, Stats stats, EnemyAi ai) {}
```

El contexto es `Fighter`. Guarda la IA que recibe de su `Enemy` y ofrece un método que se la consulta. Aquí solo van las partes que cambian respecto del capítulo anterior (la solución del ejercicio 2.4 trae la clase entera):

Fighter.java (fragmento: lo que cambia)

```
private final EnemyAi ai;   // null en los héroes: los elige el jugador

private Fighter(String name, Stats stats, EnemyAi ai) {
    this.name = name;
    this.stats = stats;
    this.ai = ai;
    this.hp = stats.maxHp();
}

public static Fighter ofHero(Hero hero) {
    return new Fighter(hero.name(), hero.stats(), null);
}

public static Fighter ofEnemy(Enemy enemy) {
    return new Fighter(enemy.name(), enemy.stats(), enemy.ai());
}

public Fighter chooseTarget(List<Fighter> heroes) {
    if (ai == null) {
        throw new IllegalStateException(name + " no tiene IA: es un héroe");
    }
    return ai.choose(this, heroes);
}
```

Es la delegación que define el patrón: `Fighter` no sabe *cómo* se elige, solo a quién preguntar. El `null` de los héroes es provisional y se nota: un método que solo vale para enemigos. Es un olor menor que dejamos pasar, porque los héroes no tienen IA y de momento no hay dónde ponerla mejor.

2.3.4 El resultado en Combat

Con la estrategia en su sitio, el turno de los enemigos pierde toda la cadena:

Combat.java (fragmento: el turno de los enemigos)

```
public List<String> enemyPhase() {
    List<String> messages = new ArrayList<>();
    for (Fighter enemy : enemies) {
        if (!enemy.isAlive()) {
            continue;
        }
        if (heroesLost()) {
            break;
        }
        Fighter target = enemy.chooseTarget(heroes);
        hit(enemy, target, damageOf(enemy, target), messages);
    }
    round++;
    return messages;
}
```

Añadir el Murciélago es ahora escribir *una* clase (`LowestDefenseAi`) y una línea en `Main`. `Combat` no se toca.

2.3.5 Aleatoriedad repetible: Random con semilla inyectada

Una IA que elige un héroe al azar necesita números aleatorios, y el azar complica dos cosas: probar el juego y repetir una partida. La solución es tratar la fuente de azar como una dependencia más.

`java.util.Random` es un generador de números pseudoaleatorios. Dos cosas importan:

- `new Random(42)` crea un generador con *semilla* 42. La semilla fija toda la secuencia: dos generadores con la misma semilla producen los mismos números en el mismo orden. Sin semilla (`new Random()`), cada ejecución sale distinta.
- `nextInt(n)` devuelve un entero de 0 a $n-1$, ambos incluidos. Para elegir entre tres héroes, `random.nextInt(3)` da el índice 0, 1 o 2.

Prueba de la semilla (no es parte de Emberdeep)

```
import java.util.Random;

public class SeedDemo {
    public static void main(String[] args) {
        Random a = new Random(42);
        Random b = new Random(42);
        for (int i = 0; i < 6; i++) {
            System.out.print(a.nextInt(3) + " ");
```

```
        }
        System.out.println();
        for (int i = 0; i < 6; i++) {
            System.out.print(b.nextInt(3) + " ");
        }
        System.out.println();
    }
}
```

Las dos líneas imprimen lo mismo: `2 0 0 2 0 1`. La regla de este libro es **inyectar** el `Random` por constructor y no hacer `new Random()` dentro de la clase que lo usa. Así `Main`, el composition root, decide la semilla: con una fija las partidas son repetibles (útil para depurar y para comparar dos ejecuciones); sin semilla serían distintas cada vez. Es la misma idea que ya conoces: una dependencia se pasa de fuera.

Un solo Random, no uno por clase

Si cada IA creara su propio `new Random(42)`, todas repetirían la misma secuencia. Se crea *un* `Random` en `Main` y se comparte con quien lo necesite.

2.4 Strategy con lambdas: la fórmula de daño

La segunda decisión del combate es *cuánto* daño se hace. En el capítulo anterior era `max(1, ataque - defensa)` escrito dentro de `Combat`, y el esqueleto lo ignoraba con un 3 fijo. Ahora será una estrategia más. Pero esta tiene un rasgo que la distingue de `EnemyAi`: su interfaz tiene un único método y las variantes no guardan estado.

2.4.1 Una interfaz funcional

Una *interfaz funcional* es una interfaz con exactamente un método abstracto. Java permite crear sus instancias con una *lambda*, una función anónima con la forma `(parámetros) -> expresión`. La anotación `@FunctionalInterface` es opcional, pero hace que el compilador proteja esa condición.

engine/combat/damage/DamageFormula.java

```
package emberdeep.engine.combat.damage;

@FunctionalInterface
public interface DamageFormula {
    int damage(int attack, int defense);
}
```

Recibe números y no `Fighter`: la fórmula solo necesita ataque y defensa, y así sirve también cuando esas cifras salgan de otro sitio.

2.4.2 Una versión de clase y otra de lambda

La regla de H1 como clase es la forma que ya conoces:

engine/combat/damage/FlatFormula.java

```java
package emberdeep.engine.combat.damage;

public class FlatFormula implements DamageFormula {
    @Override
    public int damage(int attack, int defense) {
        return Math.max(1, attack - defense);
    }
}
```

La misma regla como lambda ocupa una línea. La segunda fórmula del hito, la escalada, reduce el daño en proporción a la defensa en lugar de restarla: $\max(1, \lfloor \text{ataque} \times 100/(100 + \text{defensa}) \rfloor)$. Aquí la tienes como lambda:

Dos formas de escribir la misma estrategia

```java
DamageFormula flat = new FlatFormula();                          // clase
DamageFormula flatLambda = (attack, defense) -> Math.max(1, attack - defense);
DamageFormula scaled = (attack, defense) ->
        Math.max(1, attack * 100 / (100 + defense));

Combat combat = new Combat(heroes, enemies, scaled);         // la que quieras
```

Con ataque 7 y defensa 2, la plana da 5 y la escalada $700/102 = 6$ (la división entera descarta decimales). Con defensa 0 las dos dan el ataque completo.

2.4.3 Clase o lambda

Las dos formas son Strategy; cambia el coste de escribirlas.

Mejor una clase cuando...	**Mejor una lambda cuando...**
La estrategia guarda estado o dependencias (`RandomTargetAi` guarda un `Random`).	Es una expresión corta y sin estado.
Tiene un nombre del dominio que merece aparecer en el código y en las pruebas.	Se usa en un solo sitio y el nombre aportaría poco.
Se reutiliza en varios puntos o tendrá más de un método.	La interfaz tiene un único método abstracto.

El JDK trae interfaces funcionales genéricas en `java.util.function` [15]: `Function<T,R>` (recibe un `T` y devuelve un `R`), `Predicate<T>` (devuelve `boolean`) o `IntBinaryOperator` (dos `int` y devuelve otro). Esta última tiene exactamente la forma de nuestra fórmula, con el método `applyAsInt`. Se podría usar, pero `DamageFormula` gana en dos cosas: su nombre cuenta qué es, y puedes añadirle métodos con valor por defecto más adelante sin tocar a quien la usa. Las lambdas y las referencias a método (`Fighter::isAlive`) son la herramienta del lenguaje que lo hace posible [18].

Funciones de primera clase y patrones

Que Strategy se reduzca a pasar una función no es casualidad. Norvig defendió en 1996 que en los lenguajes con funciones como valores algunos patrones se vuelven mucho más simples

o desaparecen [11]. Java 21 está a medio camino: tiene lambdas, pero sigue pidiendo una interfaz que les dé tipo. Por eso aquí el patrón sigue teniendo su interfaz, y la lambda es solo una forma corta de escribir una implementación.

Si añades un segundo método, las lambdas dejan de compilar

Una lambda solo puede implementar una interfaz con un único método abstracto. Si algún día `DamageFormula` necesita otro método, o lo haces `default` (con cuerpo) o las lambdas del proyecto se rompen. Con `@FunctionalInterface` el error aparece en la propia interfaz y no en cada lambda.

Por qué `EnemyAi` no lleva la anotación aunque hoy también tenga un único método: porque es una pieza del juego con clases que guardan estado, y esperamos que crezca. Una interfaz marcada como funcional es una promesa de que seguirá teniendo un solo método.

2.5 Strategy: cuándo sí y cuándo no

Sí compensa cuando:

- Tienes varias formas de hacer lo mismo y el código que las elige crece con cada variante nueva (el olor de la primera sección).
- Quieres cambiar la variante sin tocar a quien la usa: por configuración, por elección del jugador o en mitad de la ejecución.
- Quieres probar cada variante por separado, o reutilizarla en otro contexto.

No compensa cuando:

- Hay dos variantes y no vas a tener más: un `boolean` o un `if` se lee mejor que una interfaz y dos clases.
- La variante es un *dato*, no un comportamiento. Si lo único que cambia entre dos enemigos es un número (el 3 fijo del esqueleto), basta un campo.
- Las variantes se parecen tanto que se resuelven con un parámetro.

Qué pagas. Más clases y un paso de indirección al leer el código. Y la elección no desaparece: se mueve a quien monta los objetos (`Main`), que debe conocer todas las variantes. Strategy no elimina el «cuál usar»; lo lleva a un único sitio razonable.

Una pista del contraste con Bridge

Dos jerarquías que varían de forma independiente (por ejemplo, qué se muestra y dónde se escribe) se parecen a Strategy, pero se resuelven con otro patrón, Bridge. Lo contrastamos en el capítulo 10, cuando el proyecto lo pida. Por ahora, `EnemyAi` y `DamageFormula` son dos estrategias independientes que no se conocen entre sí.

2.6 En el proyecto: H2 con Strategy

El hito H2 parte del estado del capítulo anterior y aplica Strategy a las dos decisiones del combate. Estos son los cambios:

- **Nuevo** `engine/combat/ai/`: `EnemyAi`, `FirstAliveAi`, `LowestHpAi` (el `weakest` del capítulo anterior) y `RandomTargetAi(Random)`.
- **Nuevo** `engine/combat/damage/`: `DamageFormula`, `FlatFormula` (ataque − defensa, mínimo 1) y `ScaledFormula` (ataque × 100 / (100 + defensa), mínimo 1).
- `Enemy` pasa a `Enemy(name, stats, ai)`; `Fighter` guarda la IA y gana `chooseTarget(heroes)`.
- `Combat` recibe la fórmula por constructor, `Combat(heroes, enemies, formula)`. Desaparecen los `if` por nombre y los auxiliares `firstAlive` y `weakest`.
- `GameSession` recibe la fórmula y se la pasa al combate.
- `ConsoleUi` gana `askFormula()`, que pregunta qué fórmula usar antes de empezar. `Main` fija la semilla del `Random` (42) y monta cada enemigo con su IA.

Cambia una regla del juego, y es deliberado. El esqueleto ya no hace un daño fijo de 3: la fórmula se aplica a todos. Con la plana, el esqueleto (7 de ataque) hace $7 - 2 = 5$ a Cael. Recuerda que el 3 fijo era uno de los olores del capítulo anterior.

Hito 2: IA y fórmula intercambiables

Tras este hito, cada enemigo lleva su propia IA (Goblin: `FirstAliveAi`; Rata de ceniza: `LowestHpAi`; Esqueleto: `RandomTargetAi`) y el jugador elige la fórmula de daño al empezar. Con la misma semilla, dos ejecuciones dan el mismo combate. Esta es la estructura:

Estructura de carpetas tras el capítulo 2

```
emberdeep/
|-- settings.gradle
|-- build.gradle
|-- gradlew
|-- gradlew.bat
|-- gradle/wrapper/
`-- src/main/java/emberdeep/
    |-- app/
    |   `-- Main.java              (composition root)
    |-- engine/
    |   |-- character/             Stats, Hero, Enemy, Fighter
    |   |-- combat/
    |   |   |-- Combat.java
    |   |   |-- ai/                EnemyAi, FirstAliveAi, LowestHpAi,
    ↪ RandomTargetAi
    |   |   `-- damage/            DamageFormula, FlatFormula,
    ↪ ScaledFormula
    |   `-- session/               GameSession
    `-- ui/
        `-- ConsoleUi.java
```

Lo practicas en el ejercicio 2.4. Antes tienes tres ejercicios más pequeños, en otros dominios, para dominar el patrón.

2.7 Ejercicios

Ejercicio 2.1: De un switch a una interfaz de puntuación ★☆☆☆☆

Este código calcula la puntuación de una partida según el modo. No es de Emberdeep: son dos archivos sueltos, `Mode.java` y `ScoreBoard.java`, sin paquete.

`ScoreBoard.java` y `Mode.java`

```
public enum Mode { EASY, HARD }

public class ScoreBoard {
    public int score(Mode mode, int kills, int secondsLeft) {
        return switch (mode) {
            case EASY -> kills * 10;
            case HARD -> kills * 25 + secondsLeft;
        };
    }
}
```

Conviértelo a Strategy con estos pasos (cada clase en su archivo):

1. Crea la interfaz `ScoreRule` con un método `int score(int kills, int secondsLeft)`.
2. Crea `EasyScore` y `HardScore`, que implementan `ScoreRule` con las dos fórmulas del `switch`.
3. Cambia `ScoreBoard`: recibe una `ScoreRule` en el constructor, la guarda y su método `score(int kills, int secondsLeft)` delega en ella. No debe quedar ningún `if` ni `switch`.
4. Escribe una clase `Main` que elija la regla según el primer argumento (`"hard"` elige `HardScore`; cualquier otra cosa, `EasyScore`) e imprima la puntuación de 4 muertes con 30 segundos restantes.
5. Borra `Mode`.

Está bien si, tras compilar con `javac *.java`, `java Main` imprime 40, `java Main hard` imprime 130, y ni `ScoreBoard` ni ningún archivo mencionan `Mode`.

Guiado ▪ solución en la pág. 40

Ejercicio 2.2: Lee un Strategy: envíos de una tienda ★★☆☆☆

Lee este código (no es de Emberdeep). Los precios son enteros en euros.

Shipping.java

```java
interface ShippingPolicy { int cost(int weightKg); }

class StandardShipping implements ShippingPolicy {
    public int cost(int weightKg) { return 3 + 2 * weightKg; }
}

class ExpressShipping implements ShippingPolicy {
    public int cost(int weightKg) { return 8 + 4 * weightKg; }
}

class FreeShipping implements ShippingPolicy {
    public int cost(int weightKg) { return 0; }
}

class Order {
    private final int weightKg;
    private ShippingPolicy policy;

    Order(int weightKg, ShippingPolicy policy) {
        this.weightKg = weightKg;
        this.policy = policy;
    }

    void changePolicy(ShippingPolicy policy) { this.policy = policy; }

    int total(int itemsPrice) { return itemsPrice + policy.cost(weightKg); }
}

public class Shipping {
    public static void main(String[] args) {
        Order order = new Order(2, new StandardShipping());
        System.out.println(order.total(20));
        order.changePolicy(new ExpressShipping());
        System.out.println(order.total(20));
    }
}
```

1. Haz una tabla con el rol de Strategy que cumple cada clase o interfaz (estrategia, estrategia concreta, contexto, cliente).
2. ¿Qué imprime el programa?
3. Quieres añadir `PickupShipping` (recogida en tienda, coste 1 euro). ¿Qué archivo o clase hay que tocar? ¿Cuál no?
4. ¿Dónde está ahora la decisión de qué política usar?

Está bien si cada rol tiene su clase, la salida es correcta y la respuesta a la pregunta 3 distingue lo que se añade de lo que se deja intacto.

Lectura de código ▪ solución en la pág. 41

Ejercicio 2.3: Un guardián que se pone a la defensiva ★★★☆☆

Partes de esta interfaz y de un guardián con vida (no es de Emberdeep; dos archivos, `Behavior.java` y `Guardian.java`):

`Behavior.java` y `Guardian.java`

```
@FunctionalInterface
public interface Behavior {
    String act(String actor);
}

public class Guardian {
    private final String name;
    private final int maxHp;
    private int hp;

    public Guardian(String name, int maxHp) {
        this.name = name;
        this.maxHp = maxHp;
        this.hp = maxHp;
    }

    public void takeDamage(int amount) {
        hp = Math.max(0, hp - amount);
    }
}
```

Haz que el guardián cambie de comportamiento en mitad del combate:

1. `Guardian` recibe en el constructor dos comportamientos, `Guardian(String name, int maxHp, Behavior opening, Behavior defensive)`. Empieza con `opening`. Añade `setBehavior(Behavior)` y `String act()`, que delega en el comportamiento actual pasándole su nombre.
2. En `takeDamage`, cuando tras el daño le queda *menos del 30 %* de su vida máxima, el guardián pasa a `defensive`. Calcula el umbral solo con enteros. La decisión de cambiar vive en `Guardian`, no en los comportamientos.
3. En un `Main`, define los dos comportamientos como **lambdas**: el primero devuelve `actor + " ataca con furia."` y el segundo `actor + " se cubre con el escudo."`. Crea un `Guardian("Golem", 20, ...)`, y llama en este orden a `act()`, `takeDamage(5)`, `act()`, `takeDamage(10)`, `act()`, imprimiendo lo que devuelve cada `act()`.

Está bien si la salida es exactamente:

```
Golem ataca con furia.
Golem ataca con furia.
Golem se cubre con el escudo.
```

y si, con 20 de vida máxima, un guardián al que le quedan justo 6 (el 30 %) *no* cambia.

Pista: *Menos del 30 % de `maxHp` equivale a `hp * 100 < maxHp * 30`, sin decimales.*

Práctica ▪ solución en la pág. 42

Ejercicio 2.4: Hito 2: IA y fórmula de daño en Emberdeep ★★★☆☆

Implementa el hito H2 partiendo del estado del capítulo anterior (solución del ejercicio 1.5).

- Crea `engine/combat/ai/` con `EnemyAi` (`Fighter choose(Fighter self, List<Fighter> heroes)`), `FirstAliveAi`, `LowestHpAi` (el héroe vivo con menos vida; `Comparator.comparingInt(Fighter::hp)` y `min`) y `RandomTargetAi`, que recibe un `Random` por constructor y elige con `nextInt` entre los héroes vivos.
- Crea `engine/combat/damage/` con `DamageFormula` (`int damage(int attack, int defense)`, funcional), `FlatFormula` y `ScaledFormula`, ambas con mínimo 1.
- `Enemy` pasa a `Enemy(name, stats, ai)`. `Fighter` guarda la IA y tiene `chooseTarget(heroes)`. `Combat` recibe la fórmula, no tiene ningún `if` por nombre y ya no tiene `firstAlive` ni `weakest`. `GameSession` recibe la fórmula. `ConsoleUi` gana `int askFormula()`, que muestra las dos opciones y devuelve 1 o 2.
- `Main` crea `ConsoleUi` primero, elige la fórmula con `askFormula()`, crea *un* `Random(42)` y monta a Goblin con `FirstAliveAi`, a la Rata de ceniza con `LowestHpAi` y al Esqueleto con `RandomTargetAi`.

Está bien si:

1. `./gradlew run` compila y pregunta la fórmula antes de empezar.
2. Eligiendo `1` y tecleando siempre `1`, la primera ronda muestra estos mensajes y la partida acaba en la ronda 3 con «¡Victoria!» (en el turno de Borin de la ronda 3 se ve `Héroes: Aria 26/30, Borin 40/40, Cael 16/24`):

   Mensajes de la primera ronda con la fórmula plana

   ```
   Aria ataca a Goblin y le hace 7 de daño.
   Borin ataca a Goblin y le hace 5 de daño.
   Cael ataca a Goblin y le hace 8 de daño.
   Goblin queda fuera de combate.
   Rata de ceniza ataca a Cael y le hace 3 de daño.
   Esqueleto ataca a Cael y le hace 5 de daño.
   ```

3. Eligiendo `2`, esos dos últimos mensajes dicen 4 y 6 de daño.
4. Repetir la ejecución con la misma elección da exactamente los mismos mensajes.

Pista: *`Combat` calcula el daño con `formula.damage(atacante.stats().attack(), defensor.stats().defense())`; la IA solo decide el objetivo.*

Proyecto ▪ solución en la pág. 44

2.8 Resumen

Resumen

- El olor de Strategy es una cadena de `if` o `switch` que elige entre variantes de un comportamiento. El patrón saca cada variante a un objeto con la misma interfaz.
- Piezas: la estrategia (interfaz), las estrategias concretas y el contexto, que guarda una y le delega. En Emberdeep: `EnemyAi`, `FirstAliveAi`… y `Fighter`.
- Añadir una variante es añadir una clase; el contexto no se toca. La elección de cuál usar

se mueve al composition root.

- Si la interfaz tiene un único método y la variante no guarda estado, una lambda basta (`DamageFormula`). Una clase es mejor con estado o con un nombre del dominio.
- `Random` se inyecta con semilla fija desde `Main`: las partidas son repetibles y todas las clases comparten el mismo generador.
- No uses Strategy para dos variantes que no crecerán ni para simples datos: un `boolean` o un campo se lee mejor.
- H2: los `if` por nombre desaparecen; cada enemigo lleva su IA y el jugador elige la fórmula de daño al empezar. Cambia una regla (el esqueleto ya no hace daño fijo).

Soluciones de los ejercicios

Solución del ejercicio 2.1 enunciado en la pág. 36

Los cinco archivos finales (sin `Mode.java`):

`ScoreRule.java`

```
public interface ScoreRule {
    int score(int kills, int secondsLeft);
}
```

`EasyScore.java`

```
public class EasyScore implements ScoreRule {
    @Override
    public int score(int kills, int secondsLeft) {
        return kills * 10;
    }
}
```

`HardScore.java`

```
public class HardScore implements ScoreRule {
    @Override
    public int score(int kills, int secondsLeft) {
        return kills * 25 + secondsLeft;
    }
}
```

ScoreBoard.java

```java
public class ScoreBoard {
    private final ScoreRule rule;

    public ScoreBoard(ScoreRule rule) {
        this.rule = rule;
    }

    public int score(int kills, int secondsLeft) {
        return rule.score(kills, secondsLeft);
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        ScoreRule rule = args.length > 0 && args[0].equals("hard")
                ? new HardScore()
                : new EasyScore();
        ScoreBoard board = new ScoreBoard(rule);
        System.out.println(board.score(4, 30));
    }
}
```

Ejecución:

```
$ javac *.java
$ java Main
40
$ java Main hard
130
```

Por qué funciona. `ScoreBoard` (el contexto) solo conoce `ScoreRule`; cada modo es una clase y el `switch` desapareció. El `?:` de `Main` es la elección, que se ha movido al único sitio que debe conocer las variantes. Easy: $4 \cdot 10 = 40$. Hard: $4 \cdot 25 + 30 = 130$.
Errores típicos: dejar el `switch` dentro de `ScoreBoard` y llamar a las clases desde ahí (se mueve el problema, no se cura); no borrar `Mode` y mantener dos formas de elegir; olvidar `@Override`, que sin él un error en la firma crea un método nuevo en vez de implementar el de la interfaz.

Solución del ejercicio 2.2

enunciado en la pág. 36

1. Roles:

Clase	Rol
`ShippingPolicy`	Estrategia (interfaz)
`StandardShipping`, `ExpressShipping`, `FreeShipping`	Estrategias concretas
`Order`	Contexto: guarda una política y le delega `co`
`Shipping` (su `main`)	Cliente: crea las políticas y elige

2. Imprime `27` y `36`. Estándar: $20 + (3 + 2 \cdot 2) = 27$. Exprés, tras `changePolicy`: $20 + (8 + 4 \cdot 2) = 36$.
3. Solo se añade una clase nueva, `PickupShipping implements ShippingPolicy`, con `cost` devolviendo 1. `Order`, `ShippingPolicy` y las tres clases existentes no se tocan. Solo el cliente (`Shipping.main`) cambia, y únicamente si quiere usar la política nueva: una línea, `new PickupShipping()`.
4. En el cliente: es quien escribe `new StandardShipping()` y `new ExpressShipping()`. `Order` no sabe cuál tiene.

Errores típicos: tomar a `Order` por una estrategia (es la que *usa* la estrategia, no la que varía) y decir que hay que modificar `Order` para soportar la nueva política.

Solución del ejercicio 2.3 enunciado en la pág. 38

`Behavior.java`

```java
@FunctionalInterface
public interface Behavior {
    String act(String actor);
}
```

Guardian.java

```java
public class Guardian {
    private final String name;
    private final int maxHp;
    private final Behavior defensive;
    private Behavior behavior;
    private int hp;

    public Guardian(String name, int maxHp, Behavior opening, Behavior
    ↪ defensive) {
        this.name = name;
        this.maxHp = maxHp;
        this.behavior = opening;
        this.defensive = defensive;
        this.hp = maxHp;
    }

    public void setBehavior(Behavior behavior) {
        this.behavior = behavior;
    }

    public String act() {
        return behavior.act(name);
    }

    public void takeDamage(int amount) {
        hp = Math.max(0, hp - amount);
        if (hp * 100 < maxHp * 30) {
            setBehavior(defensive);
        }
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Behavior aggressive = actor -> actor + " ataca con furia.";
        Behavior defensive = actor -> actor + " se cubre con el escudo.";

        Guardian golem = new Guardian("Golem", 20, aggressive, defensive);
        System.out.println(golem.act());
        golem.takeDamage(5);
        System.out.println(golem.act());
        golem.takeDamage(10);
        System.out.println(golem.act());
    }
}
```

Por qué funciona. Tras 5 de daño le quedan 15: $15 \cdot 100 = 1500$ no es menor que $20 \cdot 30 = 600$, sigue agresivo. Tras 10 más le quedan 5: $500 < 600$, pasa a defensivo. Con 6 de vida, $600 < 600$ es falso: el umbral es «menos de» el 30 %, no «como mucho». Las lambdas implementan `Behavior` porque tiene un solo método; `Guardian` (el contexto)

toma la decisión de cambiar, y las estrategias ni se enteran. Una variante válida tipa los comportamientos como `Function<String, String>` del JDK y llama a `apply` en lugar de `act`, con el mismo resultado.
Errores típicos: poner el `if` del 30 % dentro de la lambda (la estrategia no conoce la vida); calcular con `double` y comparar con `0.3` (funciona, pero arrastra decimales que aquí no hacen falta); olvidar que `setBehavior` existe para que quien lo necesite cambie la estrategia desde fuera.

Solución del ejercicio 2.4 enunciado en la pág. 39

Parte del estado del capítulo 1. `Stats.java` y `Hero.java` no cambian. Cambian `Enemy`, `Fighter`, `Combat`, `GameSession`, `ConsoleUi` y `Main`, y hay siete archivos nuevos.
engine/combat/ai/

EnemyAi.java

```java
package emberdeep.engine.combat.ai;

import emberdeep.engine.character.Fighter;
import java.util.List;

public interface EnemyAi {
    Fighter choose(Fighter self, List<Fighter> heroes);
}
```

FirstAliveAi.java

```java
package emberdeep.engine.combat.ai;

import emberdeep.engine.character.Fighter;
import java.util.List;

public class FirstAliveAi implements EnemyAi {
    @Override
    public Fighter choose(Fighter self, List<Fighter> heroes) {
        return heroes.stream().filter(Fighter::isAlive).findFirst().
    ↪ orElseThrow();
    }
}
```

LowestHpAi.java

```java
package emberdeep.engine.combat.ai;

import emberdeep.engine.character.Fighter;
import java.util.Comparator;
import java.util.List;

public class LowestHpAi implements EnemyAi {
    @Override
    public Fighter choose(Fighter self, List<Fighter> heroes) {
        return heroes.stream()
                .filter(Fighter::isAlive)
                .min(Comparator.comparingInt(Fighter::hp))
                .orElseThrow();
    }
}
```

RandomTargetAi.java

```java
package emberdeep.engine.combat.ai;

import emberdeep.engine.character.Fighter;
import java.util.List;
import java.util.Random;

public class RandomTargetAi implements EnemyAi {
    private final Random random;

    public RandomTargetAi(Random random) {
        this.random = random;
    }

    @Override
    public Fighter choose(Fighter self, List<Fighter> heroes) {
        List<Fighter> alive = heroes.stream().filter(Fighter::isAlive).
    ↪ toList();
        return alive.get(random.nextInt(alive.size()));
    }
}
```

engine/combat/damage/

DamageFormula.java

```java
package emberdeep.engine.combat.damage;

@FunctionalInterface
public interface DamageFormula {
    int damage(int attack, int defense);
}
```

FlatFormula.java

```java
package emberdeep.engine.combat.damage;

public class FlatFormula implements DamageFormula {
    @Override
    public int damage(int attack, int defense) {
        return Math.max(1, attack - defense);
    }
}
```

ScaledFormula.java

```java
package emberdeep.engine.combat.damage;

public class ScaledFormula implements DamageFormula {
    @Override
    public int damage(int attack, int defense) {
        return Math.max(1, attack * 100 / (100 + defense));
    }
}
```

engine/character/Enemy.java y Fighter.java

Enemy.java

```java
package emberdeep.engine.character;

import emberdeep.engine.combat.ai.EnemyAi;

public record Enemy(String name, Stats stats, EnemyAi ai) {}
```

Fighter.java

```java
package emberdeep.engine.character;

import emberdeep.engine.combat.ai.EnemyAi;
import java.util.List;

public class Fighter {
    private final String name;
    private final Stats stats;
    private final EnemyAi ai;   // null en los héroes: los elige el jugador
    private int hp;

    private Fighter(String name, Stats stats, EnemyAi ai) {
        this.name = name;
        this.stats = stats;
        this.ai = ai;
        this.hp = stats.maxHp();
    }

    public static Fighter ofHero(Hero hero) {
        return new Fighter(hero.name(), hero.stats(), null);
    }

    public static Fighter ofEnemy(Enemy enemy) {
        return new Fighter(enemy.name(), enemy.stats(), enemy.ai());
    }

    public String name() { return name; }
    public Stats stats() { return stats; }
    public int hp() { return hp; }
    public boolean isAlive() { return hp > 0; }

    public void takeDamage(int amount) {
        hp = Math.max(0, hp - amount);
    }

    public Fighter chooseTarget(List<Fighter> heroes) {
        if (ai == null) {
            throw new IllegalStateException(name + " no tiene IA: es un hé
    ↪ roe");
        }
        return ai.choose(this, heroes);
    }
}
```

engine/combat/Combat.java (en dos tramos seguidos, dentro de la misma clase)

Combat.java (1/2: estado y turno de los héroes)

```java
package emberdeep.engine.combat;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.damage.DamageFormula;
import java.util.ArrayList;
import java.util.List;

public class Combat {
    private final List<Fighter> heroes;
    private final List<Fighter> enemies;
    private final DamageFormula formula;
    private int round = 1;

    public Combat(List<Fighter> heroes, List<Fighter> enemies,
    ↪ DamageFormula formula) {
        this.heroes = heroes;
        this.enemies = enemies;
        this.formula = formula;
    }

    public int round() {
        return round;
    }

    public boolean heroesWon() {
        return enemies.stream().noneMatch(Fighter::isAlive);
    }

    public boolean heroesLost() {
        return heroes.stream().noneMatch(Fighter::isAlive);
    }

    public boolean isOver() {
        return heroesWon() || heroesLost();
    }

    public List<String> heroAttack(Fighter hero, Fighter target) {
        List<String> messages = new ArrayList<>();
        hit(hero, target, damageOf(hero, target), messages);
        return messages;
    }
```

Combat.java (2/2: turno de los enemigos y auxiliares)

```

    public List<String> enemyPhase() {
        List<String> messages = new ArrayList<>();
        for (Fighter enemy : enemies) {
            if (!enemy.isAlive()) {
                continue;
            }
            if (heroesLost()) {
                break;
            }
            Fighter target = enemy.chooseTarget(heroes);
            hit(enemy, target, damageOf(enemy, target), messages);
        }
        round++;
        return messages;
    }

    private void hit(Fighter attacker, Fighter target, int damage, List<
    ↪ String> messages) {
        target.takeDamage(damage);
        messages.add(attacker.name() + " ataca a " + target.name()
                + " y le hace " + damage + " de daño.");
        if (!target.isAlive()) {
            messages.add(target.name() + " queda fuera de combate.");
        }
    }

    private int damageOf(Fighter attacker, Fighter target) {
        return formula.damage(attacker.stats().attack(), target.stats().
    ↪ defense());
    }
}
```

engine/session/GameSession.java

GameSession.java

```java
package emberdeep.engine.session;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.damage.DamageFormula;
import java.util.List;

public class GameSession {
    private final List<Fighter> heroes;
    private final List<Fighter> enemies;
    private final Combat combat;

    public GameSession(List<Hero> heroes, List<Enemy> floorOneEnemies,
    ↪ DamageFormula formula) {
        this.heroes = heroes.stream().map(Fighter::ofHero).toList();
        this.enemies = floorOneEnemies.stream().map(Fighter::ofEnemy).
    ↪ toList();
        this.combat = new Combat(this.heroes, this.enemies, formula);
    }

    public List<Fighter> heroes() { return heroes; }
    public List<Fighter> enemies() { return enemies; }
    public Combat combat() { return combat; }
    public int round() { return combat.round(); }
}
```

ui/ConsoleUi.java (en dos tramos seguidos, dentro de la misma clase)

ConsoleUi.java (1/2: fórmula y partida)

```java
package emberdeep.ui;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.session.GameSession;
import java.io.PrintStream;
import java.util.List;
import java.util.Scanner;
import java.util.stream.Collectors;

public class ConsoleUi {
    private final Scanner in;
    private final PrintStream out;

    public ConsoleUi(Scanner in, PrintStream out) {
        this.in = in;
        this.out = out;
    }

    public int askFormula() {
        out.println("Fórmula de daño:");
        out.println("  [1] plana: ataque - defensa");
        out.println("  [2] escalada: ataque x 100 / (100 + defensa)");
        return ask(2);
    }

    public void run(GameSession session) {
        Combat combat = session.combat();
        out.println("=== Emberdeep: planta 1 ===");
        while (!combat.isOver()) {
            out.println("--- Ronda " + session.round() + " ---");
            for (Fighter hero : session.heroes()) {
                if (!hero.isAlive() || combat.isOver()) {
                    continue;
                }
                out.println("Héroes:   " + describe(session.heroes()));
                out.println("Enemigos: " + describe(session.enemies()));
                out.println("Turno de " + hero.name() + ". [1] atacar");
                ask(1);
                Fighter target = askTarget(session.enemies());
                print(combat.heroAttack(hero, target));
            }
            if (!combat.isOver()) {
                print(combat.enemyPhase());
            }
        }
        out.println(combat.heroesWon() ? "¡Victoria!" : "Derrota...");
    }
```

ConsoleUi.java (2/2: auxiliares y llave final)

```
    private String describe(List<Fighter> group) {
        return group.stream()
                .map(f -> f.name() + " " + f.hp() + "/" + f.stats().maxHp())
                .collect(Collectors.joining(", "));
    }

    private void print(List<String> messages) {
        messages.forEach(out::println);
    }

    private int ask(int max) {
        while (true) {
            out.print("> ");
            out.flush();
            String line = in.nextLine().trim();
            try {
                int n = Integer.parseInt(line);
                if (n >= 1 && n <= max) {
                    return n;
                }
            } catch (NumberFormatException e) {
                // no es un número: se avisa más abajo
            }
            out.println("Escribe un número entre 1 y " + max + ".");
        }
    }

    private Fighter askTarget(List<Fighter> enemies) {
        List<Fighter> alive = enemies.stream().filter(Fighter::isAlive).
toList();
        StringBuilder menu = new StringBuilder("Objetivo:");
        for (int i = 0; i < alive.size(); i++) {
            menu.append("  [").append(i + 1).append("] ").append(alive.get(
i).name());
        }
        out.println(menu);
        return alive.get(ask(alive.size()) - 1);
    }
}
```

app/Main.java

Main.java

```java
package emberdeep.app;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.ai.FirstAliveAi;
import emberdeep.engine.combat.ai.LowestHpAi;
import emberdeep.engine.combat.ai.RandomTargetAi;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.damage.ScaledFormula;
import emberdeep.engine.session.GameSession;
import emberdeep.ui.ConsoleUi;
import java.util.List;
import java.util.Random;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        ConsoleUi ui = new ConsoleUi(new Scanner(System.in), System.out);
        DamageFormula formula = ui.askFormula() == 1
                ? new FlatFormula()
                : new ScaledFormula();
        Random random = new Random(42);

        List<Hero> heroes = List.of(
                new Hero("Aria", new Stats(30, 8, 3)),
                new Hero("Borin", new Stats(40, 6, 5)),
                new Hero("Cael", new Stats(24, 9, 2)));
        List<Enemy> floorOne = List.of(
                new Enemy("Goblin", new Stats(14, 6, 1), new FirstAliveAi())
    ↪ ,
                new Enemy("Rata de ceniza", new Stats(8, 5, 0), new
    ↪ LowestHpAi()),
                new Enemy("Esqueleto", new Stats(18, 7, 2), new
    ↪ RandomTargetAi(random)));
        GameSession session = new GameSession(heroes, floorOne, formula);
        ui.run(session);
    }
}
```

Por qué funciona. Cada decisión del combate es una estrategia que `Main` monta: `Fighter` consulta la IA de su `Enemy`, y `Combat` pide el daño a la fórmula. En `Combat` ya no hay nombres de enemigos. `Main` crea un único `Random(42)` y lo comparte con la IA aleatoria. Con la fórmula plana y siempre `1`: en la ronda 1 el goblin cae (7+5+8 sobre 14), la rata ataca a Cael (el de menos vida: 24) con $5 - 2 = 3$, y el esqueleto, con el primer `nextInt(3)` de la semilla 42, que es 2, elige al tercer héroe y le hace $7 - 2 = 5$. En la ronda 2 la rata cae a manos de Aria y el esqueleto golpea a Aria (4); en la ronda 3 Aria y Borin lo derrotan. Con la escalada, el daño a Cael es 4 y 6 (500/102 y 700/102).

Errores típicos:

- Crear un `new Random(42)` por enemigo o dentro de `RandomTargetAi`: la IA deja de ser repetible o repite la misma secuencia.

- Olvidar quitar `firstAlive` y `weakest` de `Combat`, o dejar un resto de `if` por nombre.
- Llamar a `askFormula()` con la sesión ya creada: `GameSession` necesita la fórmula en el constructor, por eso `Main` crea primero la `ConsoleUi`.
- Usar `chooseTarget` con un héroe: lanza `IllegalStateException`.

Estructura final: la del árbol de la sección «En el proyecto». Este es el estado del que parte el capítulo siguiente.

Parte II

Crear y compartir objetos

CAPÍTULO 3

Factory Method y Abstract Factory: enemigos y objetos por bioma

Objetivos del capítulo

- Reconocer el olor que pide una fábrica: `new` de clases concretas repartido por el código.
- Aplicar Factory Method: un creador abstracto fija el esqueleto y cada subclase decide qué producto crea.
- Aplicar Abstract Factory: una interfaz que crea una familia de productos que van juntos (enemigos y objetos de un bioma).
- Distinguir la fábrica simple con `enum` y `switch` exhaustivo de los dos patrones del GoF, y saber qué es nuevo si ya los conocías.
- Dar el hito H3: Emberdeep con dos biomas, un `Inventory` inmutable y los spawners retirados.

Hasta ahora el grupo se enfrentaba siempre a los mismos tres enemigos, montados a mano en `Main`. Emberdeep va a tener plantas distintas: una cripta con esqueletos y una caverna con goblins, cada una con sus propios objetos. Si cada sitio que necesita un enemigo escribe su `new Enemy(...)`, cualquier cambio en cómo se construye un enemigo toca medio proyecto, y nada impide que una cripta reparta una poción de caverna. En este capítulo repartes la creación entre fábricas, con dos patrones que se parecen pero resuelven problemas distintos.

3.1 El olor que pide Factory Method: `new` por todas partes

Así monta `Main` los enemigos de una planta con lo que tienes tras el capítulo anterior, y así se vería si añadieras una segunda planta:

Main.java (fragmento): dos plantas a mano

```
List<Enemy> floorOne = List.of(
        new Enemy("Esqueleto", new Stats(18, 7, 2), new RandomTargetAi(random))
    ↪ ,
        new Enemy("Rata de ceniza", new Stats(8, 5, 0), new LowestHpAi()),
        new Enemy("Zombi", new Stats(22, 5, 1), new FirstAliveAi()));
List<Enemy> floorTwo = List.of(
        new Enemy("Goblin", new Stats(14, 6, 1), new FirstAliveAi()),
        new Enemy("Murciélago", new Stats(10, 6, 0), new LowestHpAi()),
        new Enemy("Araña", new Stats(16, 7, 2), new RandomTargetAi(random)));
```

Tres cosas duelen aquí:

- **Quien crea conoce las clases concretas.** `Main` importa `FirstAliveAi`, `LowestHpAi` y `RandomTargetAi`, y sabe qué IA y qué estadísticas lleva cada enemigo. Esos datos de

diseño del juego no son cosa del composition root.

- **El `new` se repite.** En este capítulo `Enemy` va a ganar un campo (`kind`) y su constructor cambiará. Cada `new Enemy(...)` del proyecto dejará de compilar a la vez. Con diez sitios que creen enemigos, son diez ediciones.
- **Nada centraliza las reglas de creación.** Si un día todo enemigo debe nacer con IA (un `null` es un error), tendrás que acordarte de comprobarlo en cada `new`.

Ese es el olor: **el cliente decide la clase concreta que crea, y lo hace en muchos sitios**. Ya conoces una primera cura, la fábrica simple (un método estático con un `switch`); la comparamos con los patrones del GoF en la sección 3.7. Aquí empezamos por Factory Method, cuya intención en el catálogo es definir una interfaz para crear un objeto y dejar que las subclases decidan qué clase instanciar [7]. Dicho con nuestro código: una clase abstracta sabe *cuándo* se crea un enemigo y qué reglas cumple siempre; cada subclase sabe *cuál*.

3.2 Piezas del patrón Factory Method

Factory Method tiene cinco piezas. El *producto* es lo que se crea. El *creador* es una clase abstracta que usa el producto y declara el *método fábrica*, un método abstracto que devuelve el producto. Los *creadores concretos* implementan ese método y eligen la clase concreta. El *cliente* trabaja solo con el creador.

Rol	Clase en el ejemplo	Propósito	Paquete
Producto (record)	`Enemy`	El enemigo que se crea	`engine.character`
Creador (clase abstracta)	`EnemySpawner`	Fija reglas; `spawn` usa `create`	`engine.world.biome`
Creadores concretos (clases)	`CryptSpawner`, `CavernSpawner`	Crean los enemigos de su bioma	`engine.world.biome`
Método fábrica (método abstracto)	`create(EnemyKind)`	Elige la clase concreta	`engine.world.biome`
Cliente (clase)	`GameSession`	Pide enemigos al creador	`engine.session`

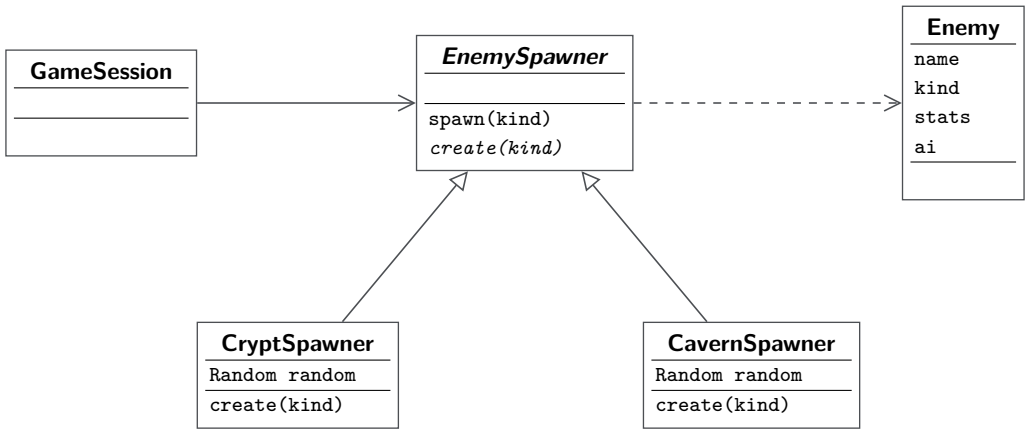


Diagrama UML del patrón Factory Method: `GameSession` conoce solo a `EnemySpawner`; cada subclase decide qué `Enemy` crea.

Léelo con la leyenda del capítulo 1. Los nombres en cursiva son abstractos. La flecha discontinua de `EnemySpawner` a `Enemy` dice que el creador depende del producto; las flechas de herencia, que las dos subclases rellenan el hueco. `GameSession` no apunta a ninguna clase concreta.

3.3 El ejemplo de Factory Method, pieza a pieza: los spawners

3.3.1 El producto: `Enemy` gana un `kind`

Para que un método fábrica sepa qué crear necesita una petición. Usamos un `enum`, el tipo de Java para un conjunto cerrado de valores con nombre. `EnemyKind` tiene seis tipos, tres por bioma:

engine/character/EnemyKind.java

```
package emberdeep.engine.character;

public enum EnemyKind {
    SKELETON, ASH_RAT, ZOMBIE,    // cripta
    GOBLIN, BAT, SPIDER           // caverna
}
```

El record `Enemy` guarda el tipo como un campo más. Es el cambio de forma que anunciaba el olor: desde ahora, `Enemy(name, kind, stats, ai)`.

engine/character/Enemy.java

```
package emberdeep.engine.character;

import emberdeep.engine.combat.ai.EnemyAi;

public record Enemy(String name, EnemyKind kind, Stats stats, EnemyAi ai) {}
```

`Fighter` no cambia: sigue leyendo nombre, estadísticas e IA del `Enemy`. Guardar el tipo no es un capricho. Más adelante hará falta saber de qué clase de enemigo se trata (sus resistencias, su plantilla) sin mirar su nombre, el mismo `if` por texto que curaste en el capítulo anterior.

3.3.2 El creador: `EnemySpawner`

`EnemySpawner` es una clase abstracta con dos métodos. `create` es el método fábrica: abstracto, lo implementan las subclases. `spawn` es el que usa el cliente y añade lo que todos los creadores comparten, una regla que ninguna subclase tiene que repetir:

engine/world/biome/EnemySpawner.java

```
package emberdeep.engine.world.biome;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.EnemyKind;

public abstract class EnemySpawner {

    public final Enemy spawn(EnemyKind kind) {
```

```
        Enemy enemy = create(kind);
        if (enemy.ai() == null) {
            throw new IllegalStateException(enemy.name() + " nace sin IA");
        }
        return enemy;
    }

    protected abstract Enemy create(EnemyKind kind);
}
```

`spawn` es `final`: el esqueleto no se puede saltar. `create` es `protected`: solo la jerarquía lo llama, el cliente pasa por `spawn`. Esa estructura, un método que fija los pasos y deja uno abierto a las subclases, es una idea que reencontrarás en otros patrones; aquí lo que importa es que el *cuándo* y las reglas viven en un único sitio.

3.3.3 Los creadores concretos

Cada bioma sabe qué enemigos tiene, con qué estadísticas y con qué IA. La cripta:

engine/world/biome/CryptSpawner.java

```
package emberdeep.engine.world.biome;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.ai.FirstAliveAi;
import emberdeep.engine.combat.ai.LowestHpAi;
import emberdeep.engine.combat.ai.RandomTargetAi;
import java.util.Random;

public class CryptSpawner extends EnemySpawner {
    private final Random random;

    public CryptSpawner(Random random) {
        this.random = random;
    }

    @Override
    protected Enemy create(EnemyKind kind) {
        return switch (kind) {
            case SKELETON -> new Enemy("Esqueleto", kind,
                    new Stats(18, 7, 2), new RandomTargetAi(random));
            case ASH_RAT -> new Enemy("Rata de ceniza", kind,
                    new Stats(8, 5, 0), new LowestHpAi());
            case ZOMBIE -> new Enemy("Zombi", kind,
                    new Stats(22, 5, 1), new FirstAliveAi());
            case GOBLIN, BAT, SPIDER -> throw new IllegalArgumentException(
                    kind + " no vive en la cripta");
        };
    }
}
```

El `Random` llega por constructor, como en el capítulo anterior: `Main` crea uno solo y lo reparte. Fíjate en el `switch`. Es una *expresión* (devuelve un valor) sobre un `enum` y no tiene `default`:

nombramos los seis valores. Si mañana añades un séptimo a `EnemyKind`, el compilador marcará este `switch` como incompleto y te obligará a decidir qué hace la cripta con él. La caverna es la misma estructura con otros datos:

engine/world/biome/CavernSpawner.java

```
package emberdeep.engine.world.biome;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.ai.FirstAliveAi;
import emberdeep.engine.combat.ai.LowestHpAi;
import emberdeep.engine.combat.ai.RandomTargetAi;
import java.util.Random;

public class CavernSpawner extends EnemySpawner {
    private final Random random;

    public CavernSpawner(Random random) {
        this.random = random;
    }

    @Override
    protected Enemy create(EnemyKind kind) {
        return switch (kind) {
            case GOBLIN -> new Enemy("Goblin", kind,
                    new Stats(14, 6, 1), new FirstAliveAi());
            case BAT -> new Enemy("Murciélago", kind,
                    new Stats(10, 6, 0), new LowestHpAi());
            case SPIDER -> new Enemy("Araña", kind,
                    new Stats(16, 7, 2), new RandomTargetAi(random));
            case SKELETON, ASH_RAT, ZOMBIE -> throw new
    ↪ IllegalArgumentException(
                    kind + " no vive en la caverna");
        };
    }
}
```

3.3.4 El cliente: `GameSession`

El cliente recibe un `EnemySpawner` y la lista de tipos que quiere. No sabe si detrás hay una cripta o una caverna:

GameSession.java (fragmento: el constructor con spawner)

```
public GameSession(List<Hero> heroes, EnemySpawner spawner,
                   List<EnemyKind> kinds, DamageFormula formula) {
    this.heroes = heroes.stream().map(Fighter::ofHero).toList();
    this.enemies = kinds.stream()
            .map(spawner::spawn)
            .map(Fighter::ofEnemy)
            .toList();
    this.combat = new Combat(this.heroes, this.enemies, formula);
}
```

`spawner::spawn` es una referencia a método, como las del capítulo anterior. Añadir un tercer bioma es escribir una subclase de `EnemySpawner` y cambiar una línea de `Main`. Nadie más se entera.

Una sola creación por llamada

Cada `spawn` construye un `Enemy` y una IA nuevos. `RandomTargetAi` guarda un `Random`, y lo que se comparte entre todas las IA es ese único `Random` de `Main`, la fuente de azar de la partida, nunca uno por IA.

3.4 El olor que pide Abstract Factory: un esqueleto con una poción de caverna

Los enemigos ya salen de la fábrica correcta. Ahora Emberdeep necesita *objetos*. Un `Item` tiene nombre, tipo (`ItemKind`: `HEALING` o `BOMB`) y potencia; lo defines en la sección siguiente. Una poción cura y una bomba hace daño. Cada bioma tiene los suyos (la cripta, agua bendita y bombas de ceniza; la caverna, hongos y bombas de roca). La tentación es repetir el truco:

`Main.java (fragmento): dos decisiones por separado`

```
EnemySpawner spawner = floor == 1
        ? new CryptSpawner(random)
        : new CavernSpawner(random);
Item bomb = floor == 2
        ? new Item("Bomba de ceniza", ItemKind.BOMB, 10)
        : new Item("Bomba de roca", ItemKind.BOMB, 14);
```

Lee la segunda condición con atención: dice `floor == 2`, la contraria de la primera. Un descuido y la cripta reparte una bomba de roca. El compilador no avisa, porque ambas cosas son un `Item` y un `EnemySpawner` válidos. Este es el olor: **objetos que deben ir juntos (los de un bioma) se eligen con decisiones independientes**. Cada vez que añadas un tipo de objeto, hay otra condición que mantener sincronizada con las demás.

Abstract Factory lo cura dando una sola decisión: se elige *una* fábrica de bioma y de ella salen enemigos y objetos. Su intención en el catálogo es proporcionar una interfaz para crear familias de objetos relacionados sin especificar sus clases concretas [7]. La mezcla pasa de ser un descuido posible a un caso en el que ni siquiera hay dónde equivocarse: el cliente sostiene una sola fábrica.

3.5 Piezas del patrón Abstract Factory

Abstract Factory tiene cinco piezas. La *fábrica abstracta* es una interfaz con un método por cada producto de la familia. Las *fábricas concretas* la implementan, una por familia. Los *productos abstractos* son los tipos que devuelve la interfaz. Los *productos concretos* son lo que cada fábrica crea de verdad. El *cliente* usa solo las interfaces.

Rol	Clase en el ejemplo	Propósito	Paquete
Fábrica abstracta (interfaz)	`BiomeFactory`	Crea enemigos y objetos	`engine.world.biome`
Fábricas concretas (clases)	`CryptBiome`, `CavernBiome`	Una familia coherente cada una	`engine.world.biome`
Productos abstractos (records)	`Enemy`; `Item` (con `ItemKind`, un `enum` aparte)	Lo que devuelve la fábrica	`engine.character`; `engine.item`
Productos concretos	Los enemigos y objetos de cada bioma	Datos propios del bioma	`engine.world.biome`
Cliente (clase)	`GameSession`	Usa una sola fábrica	`engine.session`

Los productos «abstractos» de Emberdeep son records: no hay una jerarquía de clases `Goblin extends Enemy`. Lo que distingue a un goblin de un esqueleto son datos (nombre, tipo, estadísticas, IA), y en Java moderno los datos se modelan con un record. El patrón no exige herencia en los productos: exige que el cliente reciba productos de una misma familia sin nombrar su clase.

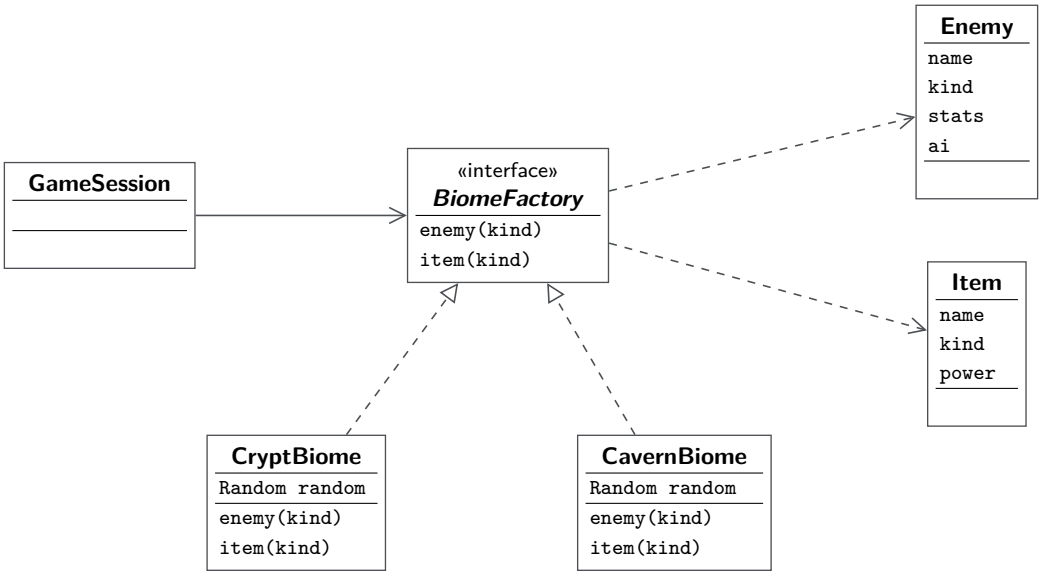


Diagrama UML del patrón Abstract Factory: `GameSession` conoce solo la interfaz `BiomeFactory`; cada bioma entrega su familia de `Enemy` e `Item`.

3.6 El ejemplo de Abstract Factory, pieza a pieza: los biomas

3.6.1 Los productos: `Item` e `ItemKind`

El segundo producto es el objeto. Un `Item` tiene nombre, un tipo y una potencia (los puntos de vida que cura una poción o el daño de una bomba). Como con los enemigos, el tipo es un `enum` aparte:

engine/item/ItemKind.java

```
1 package emberdeep.engine.item;
2
```

```
3 public enum ItemKind { HEALING, BOMB }
```

engine/item/Item.java

```
1 package emberdeep.engine.item;
2
3 public record Item(String name, ItemKind kind, int power) {}
```

Se crean en el paquete nuevo `engine/item/`, que también acogerá el inventario.

3.6.2 La fábrica abstracta y las dos fábricas concretas

La interfaz tiene un método por producto. Por convención de este libro, nombres cortos y en minúscula, como en el diagrama:

engine/world/biome/BiomeFactory.java

```
1  package emberdeep.engine.world.biome;
2
3  import emberdeep.engine.character.Enemy;
4  import emberdeep.engine.character.EnemyKind;
5  import emberdeep.engine.item.Item;
6  import emberdeep.engine.item.ItemKind;
7
8  public interface BiomeFactory {
9      Enemy enemy(EnemyKind kind);
10
11     Item item(ItemKind kind);
12 }
```

`CryptBiome` reúne lo que hacía `CryptSpawner` y añade los objetos de la cripta. Es la misma estructura de `switch` sin `default`:

engine/world/biome/CryptBiome.java

```
1  package emberdeep.engine.world.biome;
2
3  import emberdeep.engine.character.Enemy;
4  import emberdeep.engine.character.EnemyKind;
5  import emberdeep.engine.character.Stats;
6  import emberdeep.engine.combat.ai.FirstAliveAi;
7  import emberdeep.engine.combat.ai.LowestHpAi;
8  import emberdeep.engine.combat.ai.RandomTargetAi;
9  import emberdeep.engine.item.Item;
10 import emberdeep.engine.item.ItemKind;
11 import java.util.Random;
12
13 public class CryptBiome implements BiomeFactory {
14     private final Random random;
15
16     public CryptBiome(Random random) {
17         this.random = random;
18     }
19
```

```
    @Override
    public Enemy enemy(EnemyKind kind) {
        return switch (kind) {
            case SKELETON -> new Enemy("Esqueleto", kind,
                    new Stats(18, 7, 2), new RandomTargetAi(random));
            case ASH_RAT -> new Enemy("Rata de ceniza", kind,
                    new Stats(8, 5, 0), new LowestHpAi());
            case ZOMBIE -> new Enemy("Zombi", kind,
                    new Stats(22, 5, 1), new FirstAliveAi());
            case GOBLIN, BAT, SPIDER -> throw new IllegalArgumentException(
                    kind + " no vive en la cripta");
        };
    }

    @Override
    public Item item(ItemKind kind) {
        return switch (kind) {
            case HEALING -> new Item("Poción de cripta", kind, 12);
            case BOMB -> new Item("Bomba de ceniza", kind, 10);
        };
    }
}
```

engine/world/biome/CavernBiome.java

```
package emberdeep.engine.world.biome;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.ai.FirstAliveAi;
import emberdeep.engine.combat.ai.LowestHpAi;
import emberdeep.engine.combat.ai.RandomTargetAi;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;
import java.util.Random;

public class CavernBiome implements BiomeFactory {
    private final Random random;

    public CavernBiome(Random random) {
        this.random = random;
    }

    @Override
    public Enemy enemy(EnemyKind kind) {
        return switch (kind) {
            case GOBLIN -> new Enemy("Goblin", kind,
                    new Stats(14, 6, 1), new FirstAliveAi());
            case BAT -> new Enemy("Murciélago", kind,
                    new Stats(10, 6, 0), new LowestHpAi());
            case SPIDER -> new Enemy("Araña", kind,
                    new Stats(16, 7, 2), new RandomTargetAi(random));
            case SKELETON, ASH_RAT, ZOMBIE -> throw new
    ↪ IllegalArgumentException(
                    kind + " no vive en la caverna");
```

```
        };
    }

    @Override
    public Item item(ItemKind kind) {
        return switch (kind) {
            case HEALING -> new Item("Poción de caverna", kind, 10);
            case BOMB -> new Item("Bomba de roca", kind, 14);
        };
    }
}
```

3.6.3 El cliente con una sola fábrica

`GameSession` cambia el `EnemySpawner` por un `BiomeFactory`. De la misma fábrica salen los enemigos de la planta y los objetos con los que empieza el grupo, así que no hay una segunda decisión que mantener sincronizada:

GameSession.java (fragmento: el constructor con la fábrica de bioma)

```
public GameSession(List<Hero> heroes, BiomeFactory biome,
                   List<EnemyKind> kinds, DamageFormula formula) {
    this.heroes = heroes.stream().map(Fighter::ofHero).toList();
    this.enemies = kinds.stream()
            .map(biome::enemy)
            .map(Fighter::ofEnemy)
            .toList();
    this.combat = new Combat(this.heroes, this.enemies, formula);
    this.inventory = new Inventory(List.of())
            .with(biome.item(ItemKind.HEALING))
            .with(biome.item(ItemKind.HEALING))
            .with(biome.item(ItemKind.BOMB));
}
```

`Inventory` se presenta en la sección 3.9. De momento fíjate en el patrón: una planta de cripta solo puede dar objetos de cripta porque `biome` es la única fuente.

La garantía tiene un límite

Abstract Factory garantiza la familia *si el cliente la usa como única fuente*. No impide que alguien escriba `new Item("Bomba de roca", ...)` en otro rincón del código, ni que se llame a `CryptBiome.enemy(EnemyKind.GOBLIN)`: por eso lanza una excepción. Con un `enum` compartido por los dos biomas el tipo no puede evitarlo. Se podría separar `EnemyKind` en un `enum` por bioma, con el coste de otra interfaz. Aquí decidimos que la excepción basta y que el `switch` sin `default` nos avisa al crecer.

3.6.4 Por qué los spawners se retiran

Mira lo que hace ahora `CryptBiome`: crea los mismos enemigos que `CryptSpawner` y además los objetos. `CryptSpawner` se ha quedado sin nada que aportar, y mantener las dos clases significa mantener dos copias de las estadísticas del esqueleto. Los tres spawners cumplieron su función en este capítulo (mostrar Factory Method con un solo producto) y se **borran al final del hito**: `EnemySpawner`, `CryptSpawner` y `CavernSpawner` no aparecen en el árbol final. Esto no significa

que Factory Method sobrara. Significa que, cuando hay una familia de productos, la fábrica abstracta es la forma natural de crecer: aquí empezamos con un creador y evolucionamos a una familia cuando aparece el segundo producto.

Abstract Factory se implementa con métodos fábrica

Cada método de `BiomeFactory` (`enemy`, `item`) es un método fábrica, y cada bioma decide qué clase concreta crea. En el catálogo, de hecho, las fábricas abstractas se suelen implementar así [7]. La diferencia está en la unidad: Factory Method se ocupa de *un producto* y de heredar el esqueleto de su creación; Abstract Factory, de *una familia* y de que se elija junta. Fíjate también en que Factory Method usa herencia (la subclase decide) y Abstract Factory usa composición: `GameSession` recibe una fábrica como objeto.

3.7 Si ya lo conoces: lo nuevo aquí es...

Si ya usabas «Factory», casi seguro era la *fábrica simple*: una clase o un método estático que devuelve objetos según un parámetro. Es un idioma útil de Java más que un patrón del GoF; los libros de introducción la presentan como paso previo a los dos patrones [5], y [3] recomienda considerar fábricas estáticas en lugar de constructores. En Emberdeep sería esto:

`EnemyFactory.java (fábrica simple, no forma parte del proyecto)`

```
public final class EnemyFactory {
    private EnemyFactory() {}

    public static Enemy create(EnemyKind kind, Random random) {
        return switch (kind) {
            case SKELETON -> new Enemy("Esqueleto", kind,
                    new Stats(18, 7, 2), new RandomTargetAi(random));
            case ASH_RAT -> new Enemy("Rata de ceniza", kind,
                    new Stats(8, 5, 0), new LowestHpAi());
            case ZOMBIE -> new Enemy("Zombi", kind,
                    new Stats(22, 5, 1), new FirstAliveAi());
            case GOBLIN -> new Enemy("Goblin", kind,
                    new Stats(14, 6, 1), new FirstAliveAi());
            case BAT -> new Enemy("Murciélago", kind,
                    new Stats(10, 6, 0), new LowestHpAi());
            case SPIDER -> new Enemy("Araña", kind,
                    new Stats(16, 7, 2), new RandomTargetAi(random));
        };
    }
}
```

Un `switch` sobre un `enum` que devuelve un valor es exhaustivo cuando cubre todos los valores: el compilador lo exige [14]. Si añades un valor y olvidas su caso, el programa no compila, que es mejor que descubrirlo en ejecución. Lo mismo vale para una interfaz `sealed` con *pattern matching*, que cubre el ejercicio 3.4 [2]. Esta fábrica funciona y es suficiente en muchos proyectos. Comparada con los dos patrones:

	Fábrica simple	**Factory Method**	**Abstract Factory**
Qué es	Un método (estático) con un `switch`	Una clase abstracta con un método fábrica	Una interfaz con un método por producto
Quién decide la clase concreta	El `switch`, dentro de la fábrica	La subclase elegida	La fábrica concreta elegida
Productos	Uno (o varios sin relación)	Uno por creador	Una familia que va junta
Añadir una variante	Editar el `switch`	Escribir una subclase	Escribir una fábrica
Añadir un producto nuevo	Otro método o `switch`	Otro método fábrica en el creador	Cambiar la interfaz y todas las fábricas
Se apoya en	Un parámetro	Herencia	Composición
En Emberdeep	`EnemyFactory` (solo ejemplo)	`EnemySpawner` (se retira)	`BiomeFactory`

Lo nuevo aquí, si ya conocías la fábrica simple, es:

- **El creador con esqueleto.** En Factory Method el cliente no llama al método fábrica, llama a un método del creador que lo usa (`spawn`), y ahí viven las reglas comunes. La fábrica simple no tiene dónde ponerlas salvo repetirlas.
- **Variar sin editar.** Añadir un bioma es una clase nueva. En la fábrica simple es una rama más de un `switch` que ya mezcla seis enemigos de dos biomas.
- **La familia.** Abstract Factory no es «una fábrica con más métodos»: lo que aporta es que una sola elección determina todos los productos. Eso no lo da ninguna fábrica simple.
- **El criterio de elección.** Antes de añadir clases, pregúntate si la fábrica simple te basta: a menudo sí.

3.8 Factory Method y Abstract Factory: cuándo sí y cuándo no

Factory Method compensa cuando:

- El cliente no debe conocer la clase concreta que crea, y varias formas de crear comparten un esqueleto o unas reglas (en el ejemplo, «nunca un enemigo sin IA»).
- Hay una jerarquía que ya existe por otro motivo y cada subclase aporta su producto.
- Esperas más variantes con el tiempo y quieres añadirlas sin tocar al cliente.

No compensa cuando hay una sola clase concreta (un `new` es más claro), cuando lo único que cambia es un dato (basta pasar el dato), o cuando la variante cabe en una lambda: un `Supplier<Enemy>`, del paquete `java.util.function` [15], es una función sin parámetros que devuelve un objeto, y un mapa de `Supplier` sustituye muchas jerarquías de creadores. Lo pagas en una clase creadora más una subclase por variante.

Abstract Factory compensa cuando:

- Varios productos deben ser coherentes entre sí y el error de mezclarlos es real (la cripta y la caverna).
- El resto del sistema debe funcionar con cualquier familia sin saber cuál es.

No compensa con un solo producto (Factory Method o la fábrica simple bastan) ni con una única familia que nunca se va a repetir. Y tiene un coste que conviene decir claro: añadir una *familia* nueva es fácil, pero añadir un *tipo de producto* nuevo (por ejemplo, trampas) obliga a cambiar la interfaz y todas las fábricas concretas.

Qué pagas. Más clases e interfaces, una indirección al leer y, como siempre, la elección no desaparece: se mueve al composition root (`Main`), que sigue decidiendo qué fábrica monta. El catálogo web de Refactoring.Guru recoge esta misma distinción con diagramas [16].

3.9 En el proyecto: H3 con Factory Method y Abstract Factory

El hito H3 hace tres cosas: crea enemigos y objetos con las fábricas del bioma de cada planta, añade un inventario del grupo y retira los spawners. Estos son los cambios:

- **Nuevo** `EnemyKind` (en `engine/character/`); `Enemy` pasa a `Enemy(name, kind, stats, ai)`.
- **Nuevo** `engine/item/`: `Item(name, kind, power)`, `ItemKind` (`HEALING`, `BOMB`) e `Inventory`.
- **Nuevo** `engine/world/biome/`: `BiomeFactory`, `CryptBiome` y `CavernBiome`. Durante el hito pasan por aquí `EnemySpawner`, `CryptSpawner` y `CavernSpawner`, que se borran al final.
- `GameSession` recibe la fábrica del bioma y la lista de tipos de enemigo; guarda un `Inventory` y ofrece `inventory()` y `replaceInventory(Inventory)`.
- `ConsoleUi` muestra una línea «Objetos» en cada turno y ya no pone «planta 1» en la cabecera, porque ahora hay dos plantas. `Main` elige la fábrica según un argumento: sin argumentos, planta 1 (cripta); con `2`, planta 2 (caverna).

3.9.1 El inventario: un record inmutable

El grupo empieza con dos pociones y una bomba. Un `Inventory` es un record con la lista de objetos. Será **inmutable**: `with(item)` y `without(item)` no cambian el inventario, devuelven uno *nuevo*. Quien lo usa lo sustituye por el que recibe:

engine/item/Inventory.java

```
package emberdeep.engine.item;

import java.util.ArrayList;
import java.util.List;

public record Inventory(List<Item> items) {

    public Inventory {
        items = List.copyOf(items);
    }

    public Inventory with(Item item) {
        List<Item> next = new ArrayList<>(items);
        next.add(item);
        return new Inventory(next);
    }

```

```
18      public Inventory without(Item item) {
19          List<Item> next = new ArrayList<>(items);
20          next.remove(item);
21          return new Inventory(next);
22      }
23  }
```

El constructor compacto (sin paréntesis de parámetros) se ejecuta antes de asignar los campos. `List.copyOf` hace una copia que no se puede modificar, así que ni el código de fuera ni una lista que se pasara por referencia pueden alterar el inventario. `without` quita la primera aparición del objeto (los records comparan por valor, así que dos pociones iguales son «el mismo» objeto para `remove`). `GameSession` guarda el inventario actual y lo *sustituye* al cambiar: en este hito aún nada gasta objetos, pero el método `replaceInventory` queda preparado para quien lo haga.

Hito 3: dos biomas, objetos y un inventario inmutable

Tras este hito, la planta 1 solo da enemigos y objetos de cripta; la planta 2, de caverna. El grupo empieza con dos pociones y una bomba de su bioma. Ningún archivo del motor, salvo las fábricas, importa `CryptBiome` ni `CavernBiome`. Los spawners ya no existen. Esta es la estructura:

Estructura de carpetas tras el capítulo 3

```
emberdeep/
|-- settings.gradle
|-- build.gradle
|-- gradlew
|-- gradlew.bat
|-- gradle/wrapper/
`-- src/main/java/emberdeep/
    |-- app/
    |   `-- Main.java              (composition root)
    |-- engine/
    |   |-- character/             Stats, Hero, Enemy, EnemyKind, Fighter
    |   |-- combat/
    |   |   |-- Combat.java
    |   |   |-- ai/                EnemyAi, FirstAliveAi, LowestHpAi,
    ↪ RandomTargetAi
    |   |   `-- damage/            DamageFormula, FlatFormula,
    ↪ ScaledFormula
    |   |-- item/                  Item, ItemKind, Inventory
    |   |-- session/               GameSession
    |   `-- world/
    |       `-- biome/             BiomeFactory, CryptBiome, CavernBiome
    `-- ui/
        `-- ConsoleUi.java
```

Lo practicas en el ejercicio 3.5. Antes tienes cuatro ejercicios más pequeños, en otros dominios.

3.10 Ejercicios

Ejercicio 3.1: De tres new a un Factory Method ★☆☆☆☆

Este código no es de Emberdeep: son cuatro archivos sueltos sin paquete, y tres clases repiten el mismo patrón, un `new` y el mismo mensaje.

`Potion.java, Chest.java, Vendor.java y Altar.java`

```java
public record Potion(String name, int heal) {}

public class Chest {
    public String open() {
        Potion potion = new Potion("Poción menor", 10);
        return "Obtienes " + potion.name() + " (+" + potion.heal() + ")";
    }
}

public class Vendor {
    public String open() {
        Potion potion = new Potion("Poción mayor", 25);
        return "Obtienes " + potion.name() + " (+" + potion.heal() + ")";
    }
}

public class Altar {
    public String open() {
        Potion potion = new Potion("Poción sagrada", 40);
        return "Obtienes " + potion.name() + " (+" + potion.heal() + ")";
    }
}
```

Aplica Factory Method con estos pasos (cada clase en su archivo):

1. Crea la clase abstracta `PotionSource` con un método público y `final String open()`, que llama a `create()` y construye el mensaje «Obtienes … (+N)» una sola vez, y con el método fábrica `protected abstract Potion create()`.
2. Haz que `Chest`, `Vendor` y `Altar` extiendan `PotionSource` y conserven solo `create()`, con su `new Potion(...)` de antes.
3. Escribe una clase `Main` con una `List<PotionSource>` de los tres e imprime el resultado de `open()` de cada uno.

Está bien si, tras compilar con `javac *.java`, `java Main` imprime

```
Obtienes Poción menor (+10)
Obtienes Poción mayor (+25)
Obtienes Poción sagrada (+40)
```

y si el texto «Obtienes» aparece en un único archivo.

Guiado ▪ solución en la pág. 76

Ejercicio 3.2: Lee un código donde la cripta suelta un hongo de caverna ★★☆☆☆

Lee este código (no es de Emberdeep; son ocho archivos sin paquete, listados juntos).

`Encounter.java y sus fuentes`

```
public interface EnemySource { String enemy(); }
public interface LootSource { String drop(); }

public class CryptEnemies implements EnemySource {
    public String enemy() { return "Esqueleto"; }
}
public class CavernEnemies implements EnemySource {
    public String enemy() { return "Goblin"; }
}
public class CryptLoot implements LootSource {
    public String drop() { return "Agua bendita"; }
}
public class CavernLoot implements LootSource {
    public String drop() { return "Hongo curativo"; }
}

public class Encounter {
    private final EnemySource enemies;
    private final LootSource loot;

    public Encounter(EnemySource enemies, LootSource loot) {
        this.enemies = enemies;
        this.loot = loot;
    }

    public String describe() {
        return enemies.enemy() + " suelta " + loot.drop();
    }
}

public class Main {
    public static void main(String[] args) {
        Encounter ok = new Encounter(new CryptEnemies(), new CryptLoot());
        Encounter bad = new Encounter(new CryptEnemies(), new CavernLoot());
        System.out.println(ok.describe());
        System.out.println(bad.describe());
    }
}
```

1. ¿Qué imprime el programa?
2. ¿Cuál de las dos líneas es incoherente con el juego y por qué compila sin queja?
3. ¿Qué patrón evita el fallo? Haz una tabla con el rol de cada clase o interfaz (fábrica abstracta, fábricas concretas, productos, cliente) tras aplicarlo.
4. Tras el cambio, ¿qué recibe el constructor de `Encounter`? ¿Puede construirse ahora un encuentro con el enemigo de la cripta y el botín de la caverna?

Está bien si identificas la causa (dos decisiones independientes que el tipo no relaciona), nombras Abstract Factory con sus roles y señalas que la garantía depende de que el cliente

reciba una sola fábrica.

Lectura de código ▪ solución en la pág. 77

Ejercicio 3.3: Añade el bioma Volcano sin tocar al cliente ★★☆☆☆

Partes de este programa (no es de Emberdeep; siete archivos sin paquete, listados juntos).

Un BiomeFactory sencillo

```
public record Monster(String name, int hp) {}
public record Loot(String name, int value) {}

public interface BiomeFactory {
    Monster monster();
    Loot loot();
}

public class CryptBiome implements BiomeFactory {
    public Monster monster() { return new Monster("Esqueleto", 18); }
    public Loot loot() { return new Loot("Agua bendita", 12); }
}

public class CavernBiome implements BiomeFactory {
    public Monster monster() { return new Monster("Goblin", 14); }
    public Loot loot() { return new Loot("Hongo curativo", 10); }
}

public class Encounter {
    private final BiomeFactory biome;

    public Encounter(BiomeFactory biome) {
        this.biome = biome;
    }

    public String describe() {
        Monster monster = biome.monster();
        Loot loot = biome.loot();
        return monster.name() + " (" + monster.hp() + " PV) guarda "
                + loot.name() + " (valor " + loot.value() + ")";
    }
}

public class Main {
    public static void main(String[] args) {
        String name = args.length > 0 ? args[0] : "crypt";
        BiomeFactory biome = switch (name) {
            case "cavern" -> new CavernBiome();
            default -> new CryptBiome();
        };
        System.out.println(new Encounter(biome).describe());
    }
}
```

Añade el bioma `Volcano`: un `VolcanoBiome` cuyo monstruo es un «Elemental de lava» con 30 PV y cuyo botín es «Ceniza ardiente» de valor 15, y haz que `java Main volcano` lo use.

No cambies `Encounter` ni `BiomeFactory`.
Está bien si `java Main volcano` imprime

```
Elemental de lava (30 PV) guarda Ceniza ardiente (valor 15)
```

y `java Main` y `java Main cavern` siguen imprimiendo lo de antes (esqueleto con agua bendita; goblin con hongo curativo), y solo has creado un archivo y editado otro.

Pista: *El único sitio que nombra clases concretas es el composition root.*

Práctica ▪ solución en la pág. 78

Ejercicio 3.4: Una fábrica simple que el compilador vigila ★★★☆☆

Partes de estos archivos (no son de Emberdeep; sin paquete). Un `Drop` describe qué suelta un enemigo, y `ItemFactory` lo convierte en un `Item`:

`Drop.java, Item.java e ItemFactory.java`

```java
public sealed interface Drop permits GoldDrop, PotionDrop, KeyDrop {}
public record GoldDrop(int amount) implements Drop {}
public record PotionDrop(String size) implements Drop {}
public record KeyDrop(int door) implements Drop {}

public record Item(String name, int value) {}

public class ItemFactory {
    public static Item create(Drop drop) {
        if (drop instanceof GoldDrop g) {
            return new Item("Oro", g.amount());
        } else if (drop instanceof PotionDrop p) {
            return new Item("Poción " + p.size(), 10);
        } else {
            return new Item("Llave", 1);
        }
    }
}
```

Una interfaz `sealed` limita qué clases pueden implementarla (las de su `permits`). Un `switch` con *patrones de tipo* (`case GoldDrop g ->`) sobre una interfaz sellada es exhaustivo si nombra todos los tipos permitidos [2].

1. Reescribe `create` con un `switch` que devuelva el `Item`, sin `default` y sin `instanceof`. La llave debe llamarse «Llave» seguida de la puerta (`"Llave 3"`) y valer 1.
2. Añade un cuarto tipo, `ScrollDrop(String spell)`, a `Drop` y compila *sin* tocar `create`: copia el primer mensaje de error que da el compilador.
3. Cubre el nuevo caso: un pergamino se llama «Pergamino de» seguido del hechizo y vale 5.
4. Escribe un `Main` que cree un `GoldDrop(25)`, un `PotionDrop("grande")`, un `KeyDrop(3)` y un `ScrollDrop("fuego")` e imprima el `Item` de cada uno.

Está bien si el paso 2 no compila, si el resultado final imprime

```
Item[name=Oro, value=25]
Item[name=Poción grande, value=10]
Item[name=Llave 3, value=1]
Item[name=Pergamino de fuego, value=5]
```

y si en `ItemFactory` no queda ningún `default` ni `instanceof`.

Pista: *Con un `switch` como expresión, cada rama es `case Tipo variable -> expresión;`.*

Práctica ▪ solución en la pág. 78

Ejercicio 3.5: Hito 3: biomas y objetos en Emberdeep ★★★☆☆

Implementa el hito H3 partiendo del estado del capítulo anterior (solución del ejercicio 2.4), en dos pasos.

1. **Factory Method.** Crea `EnemyKind` y pasa `Enemy` a `Enemy(name, kind, stats, ai)`. Crea `EnemySpawner`, `CryptSpawner` y `CavernSpawner` en `engine/world/biome/` con los seis enemigos del capítulo (cripta: Esqueleto con `RandomTargetAi`, Rata de ceniza con `LowestHpAi`, Zombi con `FirstAliveAi`; caverna: Goblin con `FirstAliveAi`, Murciélago con `LowestHpAi`, Araña con `RandomTargetAi`). Haz que `GameSession` reciba un `EnemySpawner` y una `List<EnemyKind>`. Comprueba que el juego arranca.
2. **Abstract Factory.** Crea `engine/item/` con `ItemKind`, `Item` e `Inventory` (inmutable, con `with`, `without` e `items`). Crea `BiomeFactory`, `CryptBiome` y `CavernBiome`. `GameSession` recibe ahora un `BiomeFactory`, crea los enemigos con él y empieza con dos pociones y una bomba del bioma en su `Inventory`; añade `inventory()` y `replaceInventory(Inventory)`. `ConsoleUi` muestra la línea «Objetos: . . . » antes del turno de cada héroe. `Main` usa `CavernBiome` si el primer argumento es `2` y `CryptBiome` en cualquier otro caso. **Borra** los tres spawners.

Las estadísticas son: Esqueleto (18, 7, 2), Rata de ceniza (8, 5, 0), Zombi (22, 5, 1), Goblin (14, 6, 1), Murciélago (10, 6, 0) y Araña (16, 7, 2). Objetos: cripta, «Poción de cripta» (`HEALING`, 12) y «Bomba de ceniza» (`BOMB`, 10); caverna, «Poción de caverna» (10) y «Bomba de roca» (14).
Está bien si:

1. `./gradlew run` muestra en cada turno de héroe una línea de objetos como esta:

   Línea de objetos en la cripta

   ```
   Objetos:  Poción de cripta, Poción de cripta, Bomba de ceniza
   ```

   Eligiendo `1` (plana) y tecleando siempre `1`, la primera ronda termina con estos mensajes y la partida acaba en la ronda 4 con «¡Victoria!»:

   Ronda 1 en la cripta, fórmula plana

   ```
   Aria ataca a Esqueleto y le hace 6 de daño.
   Borin ataca a Esqueleto y le hace 4 de daño.
   Cael ataca a Esqueleto y le hace 7 de daño.
   Esqueleto ataca a Cael y le hace 5 de daño.
   Rata de ceniza ataca a Cael y le hace 3 de daño.
   Zombi ataca a Aria y le hace 2 de daño.
   ```

2. Para la caverna se arranca con el argumento 2:

   Planta 2

   ```
   ./gradlew run --args="2"
   ```

   La línea de objetos es `Objetos:` seguida de «Poción de caverna, Poción de caverna, Bomba de roca». Con la misma elección, la ronda 1 acaba con «Murciélago ataca a Cael y le hace 4 de daño.» y «Araña ataca a Cael y le hace 5 de daño.», y la partida termina en la ronda 3 con «¡Victoria!».

3. Ningún archivo de `src` contiene la palabra «Spawner», y ningún archivo fuera de `engine/world/biome/` y `app/Main.java` menciona `CryptBiome` ni `CavernBiome`.

4. El inventario es inmutable: si añades temporalmente a `Main` estas líneas, la salida es «1 0». Bórralas después.

   Comprobación temporal en Main.java

   ```
   Inventory a = new Inventory(List.of(new Item("X", ItemKind.BOMB, 1)));
   Inventory b = a.without(a.items().get(0));
   System.out.println(a.items().size() + " " + b.items().size());
   ```

Pista: *El orden de los enemigos en la lista de* `kinds` *es el orden en que atacan: en la cripta,* `SKELETON`*,* `ASH_RAT`*,* `ZOMBIE`*.*

Proyecto ▪ solución en la pág. 79

3.11 Resumen

Resumen

- El olor de las fábricas es un `new` de clases concretas repartido por el código, y objetos que deben ir juntos elegidos con decisiones independientes.
- Factory Method: un creador abstracto fija el esqueleto (`spawn`) y cada subclase implementa el método fábrica (`create`). Añadir una variante es añadir una subclase.
- Abstract Factory: una interfaz con un método por producto (`BiomeFactory`) y una fábrica concreta por familia. Una sola elección determina enemigos y objetos.
- La fábrica simple con `enum` y `switch` exhaustivo ya funciona y el compilador avisa si falta un caso. Los patrones aportan reglas comunes y familias, no un `switch` mejor.
- Añadir una familia es fácil; añadir un producto a la familia toca la interfaz y todas las fábricas. La garantía vale mientras el cliente use solo la fábrica.
- H3: `Enemy(name, kind, stats, ai)`, `Item`, `ItemKind`, `Inventory` inmutable y dos biomas. Los spawners sirvieron de ejemplo y se borran.

Soluciones de los ejercicios

Solución del ejercicio 3.1

enunciado en la pág. 70

Los cinco archivos finales:

PotionSource.java

```java
public abstract class PotionSource {

    public final String open() {
        Potion potion = create();
        return "Obtienes " + potion.name() + " (+" + potion.heal() + ")";
    }

    protected abstract Potion create();
}
```

Chest.java, Vendor.java y Altar.java (tres archivos)

```java
public class Chest extends PotionSource {
    @Override
    protected Potion create() {
        return new Potion("Poción menor", 10);
    }
}

public class Vendor extends PotionSource {
    @Override
    protected Potion create() {
        return new Potion("Poción mayor", 25);
    }
}

public class Altar extends PotionSource {
    @Override
    protected Potion create() {
        return new Potion("Poción sagrada", 40);
    }
}
```

Main.java (Potion.java no cambia)

```java
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<PotionSource> sources = List.of(new Chest(), new Vendor(), new
    ↪ Altar());
        for (PotionSource source : sources) {
            System.out.println(source.open());
        }
    }
}
```

Por qué funciona. `open()` es el método del creador que el cliente llama; dentro usa `create()`, el método fábrica, que cada subclase implementa. El mensaje se construye una vez en `PotionSource`, y el `new Potion` vive solo en el método fábrica de cada subclase. `Main` trabaja con `PotionSource` y no conoce `Potion` en concreto.
Errores típicos:

- Dejar `open()` abstracto o sin `final` y reescribirlo en cada subclase: el mensaje vuelve a repetirse.
- Hacer `create()` `private`: las subclases no pueden sobrescribirlo (con `protected` o paquete sí).
- Olvidar `import java.util.List;` en `Main`.

Solución del ejercicio 3.2 enunciado en la pág. 71

1. Imprime

```
Esqueleto suelta Agua bendita
Esqueleto suelta Hongo curativo
```

2. La segunda línea es incoherente: un esqueleto de cripta no suelta un hongo de caverna. Compila porque `Encounter` pide un `EnemySource` y un `LootSource`, y `CryptEnemies` y `CavernLoot` lo son. Son dos decisiones independientes y nada en los tipos las relaciona.
3. Es Abstract Factory.

Rol	Clase	Nota
Fábrica abstracta (interfaz)	Nueva, por ejemplo `BiomeFactory` con `enemy()` y `drop()`	Sustituye a `EnemySource` y `LootSource`
Fábricas concretas	`CryptBiome`, `CavernBiome`	Cada una devuelve su enemigo y su botín
Productos	El enemigo y el botín (aquí, textos)	Siempre del mismo bioma
Cliente	`Encounter`	Recibe una sola fábrica

4. `Encounter(BiomeFactory biome)` y `describe()` devuelve `biome.enemy() + " suelta " + biome.drop()`. Ya no se puede construir ese encuentro mezclado: hay un único argumento y todo sale de él.

Matiz. La garantía depende de que el cliente use la fábrica como única fuente. Si `Encounter` siguiera pidiendo un `LootSource` aparte, el problema volvería.
Errores típicos:

- Responder «Factory Method»: crea un solo producto y no relaciona dos productos.
- Mantener las dos interfaces antiguas y solo añadir una tercera: sigue habiendo dos puertas para mezclar.

Solución del ejercicio 3.3

enunciado en la pág. 72

Un archivo nuevo y una línea en `Main`:

`VolcanoBiome.java`

```
public class VolcanoBiome implements BiomeFactory {
    public Monster monster() { return new Monster("Elemental de lava", 30);
    ↪ }
    public Loot loot() { return new Loot("Ceniza ardiente", 15); }
}
```

`Main.java (única edición: el caso volcano)`

```
public class Main {
    public static void main(String[] args) {
        String name = args.length > 0 ? args[0] : "crypt";
        BiomeFactory biome = switch (name) {
            case "cavern" -> new CavernBiome();
            case "volcano" -> new VolcanoBiome();
            default -> new CryptBiome();
        };
        System.out.println(new Encounter(biome).describe());
    }
}
```

Por qué funciona. `Encounter` solo conoce la interfaz, así que no se entera del bioma nuevo. Quien nombra clases concretas es el composition root, y es el único que cambia. Con `java Main volcano` se imprime `Elemental de lava (30 PV) guarda Ceniza ardiente (valor 15)`.

Errores típicos:

- Meter un `if (biome instanceof VolcanoBiome)` o un `switch` en `Encounter`: el cliente vuelve a conocer las clases concretas.
- Olvidar `implements BiomeFactory`: el `case` no compilaría, porque el `switch` devuelve un `BiomeFactory`.
- Tocar la interfaz para añadir un método: obligaría a cambiar las otras dos fábricas.

Solución del ejercicio 3.4

enunciado en la pág. 73

Estado final (el paso 2 se hace con `ScrollDrop` añadido y `create` sin tocar):

`Drop.java (cambia el permits; GoldDrop, PotionDrop y KeyDrop no cambian)`

```
public sealed interface Drop permits GoldDrop, PotionDrop, KeyDrop,
    ↪ ScrollDrop {}
```

`ScrollDrop.java (nuevo)`

```
public record ScrollDrop(String spell) implements Drop {}
```

ItemFactory.java

```java
public class ItemFactory {
    public static Item create(Drop drop) {
        return switch (drop) {
            case GoldDrop g -> new Item("Oro", g.amount());
            case PotionDrop p -> new Item("Poción " + p.size(), 10);
            case KeyDrop k -> new Item("Llave " + k.door(), 1);
            case ScrollDrop s -> new Item("Pergamino de " + s.spell(), 5);
        };
    }
}
```

Main.java

```java
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<Drop> drops = List.of(
                new GoldDrop(25),
                new PotionDrop("grande"),
                new KeyDrop(3),
                new ScrollDrop("fuego"));
        for (Drop drop : drops) {
            System.out.println(ItemFactory.create(drop));
        }
    }
}
```

Por qué funciona. El `switch` es una expresión sobre una interfaz sellada y cubre sus cuatro tipos permitidos, así que es exhaustivo y no necesita `default`. En el paso 2, con `ScrollDrop` ya permitido pero sin su `case`, `javac` se queja en el `switch` con el mensaje «the switch expression does not cover all possible input values». El compilador te obliga a decidir qué hace la fábrica con cada tipo nuevo. La versión con `if/else` habría tratado cualquier cosa nueva como una llave.

Errores típicos:

- Añadir un `default -> new Item("Llave", 1)` «para que compile»: se pierde la protección, que era el objetivo.
- Olvidar la palabra `ScrollDrop` en el `permits`: sin eso, la clase no puede implementar `Drop`.

Solución del ejercicio 3.5

enunciado en la pág. 74

Parte del estado del capítulo 2. `Stats`, `Hero`, `Fighter`, `Combat`, el paquete `ai` y el paquete `damage` no cambian. Esta solución muestra el estado *final*; el paso 1 son los tres archivos del texto del capítulo (`EnemySpawner`, `CryptSpawner`, `CavernSpawner`) y el constructor de `GameSession` con `EnemySpawner`, que se sustituyen en el paso 2.

engine/character/

EnemyKind.java

```
package emberdeep.engine.character;

public enum EnemyKind {
    SKELETON, ASH_RAT, ZOMBIE,    // cripta
    GOBLIN, BAT, SPIDER           // caverna
}
```

Enemy.java

```
package emberdeep.engine.character;

import emberdeep.engine.combat.ai.EnemyAi;

public record Enemy(String name, EnemyKind kind, Stats stats, EnemyAi ai) {}
```

engine/item/

ItemKind.java

```
package emberdeep.engine.item;

public enum ItemKind { HEALING, BOMB }
```

Item.java

```
package emberdeep.engine.item;

public record Item(String name, ItemKind kind, int power) {}
```

Inventory.java

```java
package emberdeep.engine.item;

import java.util.ArrayList;
import java.util.List;

public record Inventory(List<Item> items) {

    public Inventory {
        items = List.copyOf(items);
    }

    public Inventory with(Item item) {
        List<Item> next = new ArrayList<>(items);
        next.add(item);
        return new Inventory(next);
    }

    public Inventory without(Item item) {
        List<Item> next = new ArrayList<>(items);
        next.remove(item);
        return new Inventory(next);
    }
}
```

engine/world/biome/

BiomeFactory.java

```java
package emberdeep.engine.world.biome;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;

public interface BiomeFactory {
    Enemy enemy(EnemyKind kind);

    Item item(ItemKind kind);
}
```

CryptBiome.java

```java
package emberdeep.engine.world.biome;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.ai.FirstAliveAi;
import emberdeep.engine.combat.ai.LowestHpAi;
import emberdeep.engine.combat.ai.RandomTargetAi;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;
import java.util.Random;

public class CryptBiome implements BiomeFactory {
    private final Random random;

    public CryptBiome(Random random) {
        this.random = random;
    }

    @Override
    public Enemy enemy(EnemyKind kind) {
        return switch (kind) {
            case SKELETON -> new Enemy("Esqueleto", kind,
                    new Stats(18, 7, 2), new RandomTargetAi(random));
            case ASH_RAT -> new Enemy("Rata de ceniza", kind,
                    new Stats(8, 5, 0), new LowestHpAi());
            case ZOMBIE -> new Enemy("Zombi", kind,
                    new Stats(22, 5, 1), new FirstAliveAi());
            case GOBLIN, BAT, SPIDER -> throw new IllegalArgumentException(
                    kind + " no vive en la cripta");
        };
    }

    @Override
    public Item item(ItemKind kind) {
        return switch (kind) {
            case HEALING -> new Item("Poción de cripta", kind, 12);
            case BOMB -> new Item("Bomba de ceniza", kind, 10);
        };
    }
}
```

CavernBiome.java

```java
package emberdeep.engine.world.biome;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.ai.FirstAliveAi;
import emberdeep.engine.combat.ai.LowestHpAi;
import emberdeep.engine.combat.ai.RandomTargetAi;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;
import java.util.Random;

public class CavernBiome implements BiomeFactory {
    private final Random random;

    public CavernBiome(Random random) {
        this.random = random;
    }

    @Override
    public Enemy enemy(EnemyKind kind) {
        return switch (kind) {
            case GOBLIN -> new Enemy("Goblin", kind,
                    new Stats(14, 6, 1), new FirstAliveAi());
            case BAT -> new Enemy("Murciélago", kind,
                    new Stats(10, 6, 0), new LowestHpAi());
            case SPIDER -> new Enemy("Araña", kind,
                    new Stats(16, 7, 2), new RandomTargetAi(random));
            case SKELETON, ASH_RAT, ZOMBIE -> throw new
    ↪ IllegalArgumentException(
                    kind + " no vive en la caverna");
        };
    }

    @Override
    public Item item(ItemKind kind) {
        return switch (kind) {
            case HEALING -> new Item("Poción de caverna", kind, 10);
            case BOMB -> new Item("Bomba de roca", kind, 14);
        };
    }
}
```

engine/session/GameSession.java

GameSession.java

```java
package emberdeep.engine.session;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Fighter;
import emberdeep.engine.character.Hero;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.item.Inventory;
import emberdeep.engine.item.ItemKind;
import emberdeep.engine.world.biome.BiomeFactory;
import java.util.List;

public class GameSession {
    private final List<Fighter> heroes;
    private final List<Fighter> enemies;
    private final Combat combat;
    private Inventory inventory;

    public GameSession(List<Hero> heroes, BiomeFactory biome,
                       List<EnemyKind> kinds, DamageFormula formula) {
        this.heroes = heroes.stream().map(Fighter::ofHero).toList();
        this.enemies = kinds.stream()
                .map(biome::enemy)
                .map(Fighter::ofEnemy)
                .toList();
        this.combat = new Combat(this.heroes, this.enemies, formula);
        this.inventory = new Inventory(List.of())
                .with(biome.item(ItemKind.HEALING))
                .with(biome.item(ItemKind.HEALING))
                .with(biome.item(ItemKind.BOMB));
    }

    public List<Fighter> heroes() { return heroes; }
    public List<Fighter> enemies() { return enemies; }
    public Combat combat() { return combat; }
    public int round() { return combat.round(); }
    public Inventory inventory() { return inventory; }

    public void replaceInventory(Inventory next) {
        this.inventory = next;
    }
}
```

ui/ConsoleUi.java (en tres tramos seguidos, dentro de la misma clase)

ConsoleUi.java (1/3: imports y fórmula)

```java
package emberdeep.ui;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.item.Inventory;
import emberdeep.engine.item.Item;
import emberdeep.engine.session.GameSession;
import java.io.PrintStream;
import java.util.List;
import java.util.Scanner;
import java.util.stream.Collectors;

public class ConsoleUi {
    private final Scanner in;
    private final PrintStream out;

    public ConsoleUi(Scanner in, PrintStream out) {
        this.in = in;
        this.out = out;
    }

    public int askFormula() {
        out.println("Fórmula de daño:");
        out.println("  [1] plana: ataque - defensa");
        out.println("  [2] escalada: ataque x 100 / (100 + defensa)");
        return ask(2);
    }
```

ConsoleUi.java (2/3: la partida)

```
    public void run(GameSession session) {
        Combat combat = session.combat();
        out.println("=== Emberdeep ===");
        while (!combat.isOver()) {
            out.println("--- Ronda " + session.round() + " ---");
            for (Fighter hero : session.heroes()) {
                if (!hero.isAlive() || combat.isOver()) {
                    continue;
                }
                out.println("Héroes:   " + describe(session.heroes()));
                out.println("Enemigos: " + describe(session.enemies()));
                out.println("Objetos:  " + describeItems(session.inventory()
    ↪ ));
                out.println("Turno de " + hero.name() + ". [1] atacar");
                ask(1);
                Fighter target = askTarget(session.enemies());
                print(combat.heroAttack(hero, target));
            }
            if (!combat.isOver()) {
                print(combat.enemyPhase());
            }
        }
        out.println(combat.heroesWon() ? "¡Victoria!" : "Derrota...");
    }
```

ConsoleUi.java (3/3: auxiliares y llave final)

```
    private String describe(List<Fighter> group) {
        return group.stream()
                .map(f -> f.name() + " " + f.hp() + "/" + f.stats().maxHp())
                .collect(Collectors.joining(", "));
    }

    private String describeItems(Inventory inventory) {
        if (inventory.items().isEmpty()) {
            return "(ninguno)";
        }
        return inventory.items().stream()
                .map(Item::name)
                .collect(Collectors.joining(", "));
    }

    private void print(List<String> messages) {
        messages.forEach(out::println);
    }

    private int ask(int max) {
        while (true) {
            out.print("> ");
            out.flush();
            String line = in.nextLine().trim();
            try {
                int n = Integer.parseInt(line);
                if (n >= 1 && n <= max) {
                    return n;
                }
            } catch (NumberFormatException e) {
                // no es un número: se avisa más abajo
            }
            out.println("Escribe un número entre 1 y " + max + ".");
        }
    }

    private Fighter askTarget(List<Fighter> enemies) {
        List<Fighter> alive = enemies.stream().filter(Fighter::isAlive).toList();
        StringBuilder menu = new StringBuilder("Objetivo:");
        for (int i = 0; i < alive.size(); i++) {
            menu.append("  [").append(i + 1).append("] ").append(alive.get(i).name());
        }
        out.println(menu);
        return alive.get(ask(alive.size()) - 1);
    }
}
```

app/Main.java

Main.java

```java
package emberdeep.app;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.damage.ScaledFormula;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.biome.BiomeFactory;
import emberdeep.engine.world.biome.CavernBiome;
import emberdeep.engine.world.biome.CryptBiome;
import emberdeep.ui.ConsoleUi;
import java.util.List;
import java.util.Random;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        ConsoleUi ui = new ConsoleUi(new Scanner(System.in), System.out);
        DamageFormula formula = ui.askFormula() == 1
                ? new FlatFormula()
                : new ScaledFormula();
        Random random = new Random(42);

        boolean cavern = args.length > 0 && args[0].equals("2");
        BiomeFactory biome = cavern
                ? new CavernBiome(random)
                : new CryptBiome(random);
        List<EnemyKind> kinds = cavern
                ? List.of(EnemyKind.GOBLIN, EnemyKind.BAT, EnemyKind.SPIDER)
                : List.of(EnemyKind.SKELETON, EnemyKind.ASH_RAT, EnemyKind.ZOMBIE);

        List<Hero> heroes = List.of(
                new Hero("Aria", new Stats(30, 8, 3)),
                new Hero("Borin", new Stats(40, 6, 5)),
                new Hero("Cael", new Stats(24, 9, 2)));
        GameSession session = new GameSession(heroes, biome, kinds, formula);
        ui.run(session);
    }
}
```

Por qué funciona. Cada bioma es una fábrica completa: `GameSession` pide enemigos y objetos a la misma `BiomeFactory`, así que la cripta no puede repartir objetos de caverna. Solo `Main` y el paquete `biome` nombran las clases concretas. Los spawners fueron el escalón intermedio y desaparecen; `grep -r Spawner src/` no encuentra nada. `Inventory` copia la lista en el constructor compacto, y `with` y `without` crean otro, así que el original no cambia. En la cripta con la fórmula plana y siempre `1`: en la ronda 1 el esqueleto, con el primer `nextInt(3)` de la semilla 42 (que es 2), elige a Cael; la rata, el de menos vida, también ataca a Cael; el zombi, al primero vivo, Aria. El esqueleto cae en la ronda 2, la rata también,

y el zombi en la ronda 4. En la caverna, el goblin cae en la ronda 1 y la partida termina en la ronda 3.

Errores típicos:

- Dejar los spawners «por si acaso»: duplican estadísticas y el criterio de «hecho» pide su ausencia.
- Crear un `new Random(42)` por bioma: cambia las trazas. Solo se crea uno, en `Main`.
- Mutar la lista de `Inventory` con `items().add(...)`: lanza `UnsupportedOperationException` porque la copia es inmutable. Para añadir, usa `with` y sustituye el inventario.
- Cambiar el orden de la lista de tipos: cambia quién ataca primero y la traza.
- Ejecutar la caverna sin el argumento: sin `2`, `Main` monta la cripta.

Estructura final: la del árbol de la sección «En el proyecto». Este es el estado del que parte el capítulo siguiente.

CAPÍTULO 4

Builder y Singleton: héroes y mazmorra paso a paso

Objetivos del capítulo

- Reconocer el olor que pide un Builder: un constructor con muchos parámetros o un objeto que solo es válido si se monta entero.
- Aplicar Builder: pasos que devuelven `this` y una validación única en `build()`, incluido un builder anidado.
- Decidir cuándo basta un `record` con constructor compacto y cuándo hace falta un Builder.
- Aplicar Singleton (holder y `enum`), medir su coste y conocer la alternativa: crear la instancia una vez en `Main`.
- Dar el hito H4: `HeroBuilder`, mazmorra de tres plantas con `DungeonBuilder` y `GameConfig`.

Conoces Builder y Singleton, así que este capítulo es de repaso con otro enfoque: aquí medimos cuánto cuesta cada uno y en qué momento dejan de compensar. Emberdeep ya tiene héroes y dos plantas, pero los héroes se montan con tres números sueltos y las plantas con listas escritas a mano en `Main`. Además, pronto habrá datos (la semilla del azar, el número de plantas) que muchas clases querrán leer, y la tentación de hacerlos globales es fuerte. Primero, Builder.

4.1 El olor que pide Builder: un constructor con ocho parámetros

Hoy un héroe es `new Hero("Aria", new Stats(30, 8, 3))`: dos parámetros y tres números dentro de `Stats`. Ya cuesta recordar el orden de los números. Imagina el héroe que verás en capítulos posteriores, con equipo y habilidades, si siguiera el mismo camino:

Hipotético, no forma parte del proyecto

```
Hero hero = new Hero("Aria", 30, 8, 3, 5, "espada", "cota", true);
```

¿Qué es el `5`? ¿Y el `true`? El compilador no protege nada: dos `int` intercambiados compilan igual. Este es el olor: **un constructor con muchos parámetros, algunos opcionales, que se llama con valores sin nombre**. Hay un segundo síntoma, más importante: el objeto solo tiene sentido si cumple reglas (nombre no vacío, vida positiva) y esas reglas se comprueban, o no, en cada sitio que lo crea.

Builder separa el *montaje* del objeto del objeto final. Su intención en el catálogo es separar la construcción de un objeto complejo de su representación, de modo que el mismo proceso pueda crear representaciones distintas [7]. En Java, la forma habitual es un objeto auxiliar con un método por cada dato, que se encadenan, y un `build()` que valida todo de golpe [3].

4.2 Piezas del patrón Builder

Rol	Clase en el ejemplo	Propósito	Paquete
Producto (records)	`Hero`; `Dungeon` y `Floor`	Lo que se monta	`engine.character`; `engine.world`
Builder (clase)	`HeroBuilder`	Guarda los datos y valida	`engine.character`
Builder anidado (clase interna)	`DungeonBuilder.FloorBuilder`	Monta una planta dentro del builder de la mazmorra	`engine.world`
Pasos (métodos)	`name`, `stats`, `floor`, `biome`, `enemies`	Dan un dato y devuelven el builder	`engine.character`; `engine.world`
Validación (método)	`build()`	Comprueba y crea el producto	`engine.character`; `engine.world`
Cliente (clase)	`Main`	Encadena los pasos	`app`

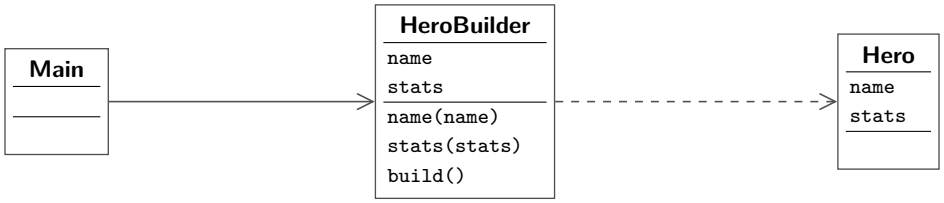


Diagrama UML del patrón Builder: `Main` da los pasos a `HeroBuilder`; `build()` valida y crea el `Hero`.

El «director» del GoF no se usa

El catálogo original incluye un *director*, una clase que sabe en qué orden dar los pasos. En Java casi nunca existe: el propio cliente encadena los pasos, como hará `Main`. Por eso la tabla no lo lista.

4.3 El ejemplo de Builder, pieza a pieza: `HeroBuilder`

El producto no cambia: sigue siendo `Hero(name, stats)` del capítulo 1. El builder guarda los datos en campos y cada paso devuelve `this`, que es lo que permite encadenar las llamadas:

engine/character/HeroBuilder.java

```
1  package emberdeep.engine.character;
2
3  public class HeroBuilder {
4      private String name;
5      private Stats stats;
6
7      public HeroBuilder name(String name) {
8          this.name = name;
9          return this;
10     }
11
12     public HeroBuilder stats(Stats stats) {
13         this.stats = stats;
```

```
        return this;
    }

    public Hero build() {
        if (name == null || name.isBlank()) {
            throw new IllegalStateException("El héroe necesita un nombre");
        }
        if (stats == null) {
            throw new IllegalStateException("El héroe necesita estadísticas");
        }
        if (stats.maxHp() <= 0 || stats.attack() <= 0 || stats.defense() < 0) {
            throw new IllegalStateException("Estadísticas no válidas: " + stats
    ↪ );
        }
        return new Hero(name, stats);
    }
}
```

Tres decisiones a notar. Los pasos **no validan**: aceptan cualquier cosa, porque algunas reglas dependen de varios datos y solo se pueden comprobar cuando están todos. Toda la validación vive en `build()`, un único sitio. La excepción es `IllegalStateException`: el builder está en un estado en el que no puede construir. Y la defensa admite 0 (hay héroes sin armadura); la vida y el ataque deben ser positivos. Se usa así:

Uso (fragmento)

```
Hero aria = new HeroBuilder()
        .name("Aria")
        .stats(new Stats(30, 8, 3))
        .build();
```

Un builder se usa una vez

El builder guarda estado mutable. Si lo reutilizas para otro héroe sin volver a dar todos los pasos, el segundo héroe hereda datos del primero. Crea uno nuevo (`new HeroBuilder()`) por cada objeto.

4.4 Si ya lo conoces: Builder y `record`

Un `record` ya trae mucho de lo que se pedía a un Builder: es inmutable y su *constructor compacto* valida antes de asignar los campos [14]:

Hero con constructor compacto (alternativa, no es el proyecto)

```
public record Hero(String name, Stats stats) {
    public Hero {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("El héroe necesita un nombre");
        }
    }
}
```

Para dos campos es suficiente, y es lo que deberías usar si ese fuera todo. Aquí el Builder se

justifica por lo que viene: el héroe ganará campos opcionales (equipo, habilidades) y entonces la llamada del principio del capítulo dejará de ser legible. Qué elegir:

	Constructor compacto de un `record`	**Builder**
Parámetros	Pocos, casi todos obligatorios	Muchos, varios opcionales con valor por defecto
Validación	Reglas sobre los campos del propio objeto	Reglas entre datos que llegan por pasos, o que dependen de ellos
Legibilidad de la llamada	Por posición	Cada dato lleva el nombre de su paso
Montaje en pasos o anidado	No	Sí (el builder de la mazmorra)
Coste	Ninguno	Una clase más y estado mutable

Hay una tercera vía para cambiar *un* dato de un objeto ya creado: un método `with` que devuelve una copia modificada, como el `with` de `Inventory` del capítulo anterior. Lo nuevo aquí, si ya conocías Builder, es:

- **Cuándo no usarlo.** Con pocos parámetros un `record` basta (ejercicio 4.3).
- **Validar en un solo sitio**, en `build()`, y no en cada paso.
- **Builders anidados**, que montan un árbol de objetos (siguiente sección).
- **La refactorización hacia el Builder**: llegar a él desde código que ya repite el montaje, como recoge [10] con *Encapsulate Composite with Builder*.

4.5 Builder anidado: `DungeonBuilder` y `FloorBuilder`

Emberdeep tendrá una mazmorra de tres plantas: cripta, caverna y de nuevo cripta (un bioma puede repetirse). Cada planta es un bioma y una lista de tipos de enemigo; más adelante tendrá también un mapa. Dos records modelan el resultado:

`engine/world/Floor.java y Dungeon.java (dos archivos)`

```
package emberdeep.engine.world;

public record Floor(BiomeFactory biome, List<EnemyKind> enemies) {
    public Floor {
        enemies = List.copyOf(enemies);
    }
}

public record Dungeon(List<Floor> floors) {
    public Dungeon {
        floors = List.copyOf(floors);
    }
}
```

(Sin imports para abreviar; la solución del hito trae los archivos completos.) Montar una mazmorra a mano exige construir cada `Floor` y luego la lista, y nada impide una mazmorra

vacía o una planta sin enemigos. Un builder *anidado* lo resuelve: el builder de la mazmorra ofrece `floor()`, que devuelve un builder de planta; al terminar la planta, `done()` te devuelve al builder de la mazmorra:

Uso (fragmento de Main)

```
Dungeon dungeon = new DungeonBuilder()
        .floor().biome(crypt).enemies(cryptKinds).done()
        .floor().biome(cavern).enemies(cavernKinds).done()
        .floor().biome(crypt).enemies(cryptKinds).done()
        .build();
```

`FloorBuilder` es una clase *interna* (no `static`) de `DungeonBuilder`: puede llamar a la lista `floors` de su builder padre. Su `done()` añade la planta y devuelve el padre con `DungeonBuilder.this`. La parte clave, en `build()`:

DungeonBuilder.java (fragmento: la validación)

```
public Dungeon build() {
    if (floors.isEmpty()) {
        throw new IllegalStateException("La mazmorra necesita al menos una
    ↪ planta");
    }
    for (int i = 0; i < floors.size(); i++) {
        if (floors.get(i).enemies().isEmpty()) {
            throw new IllegalStateException("La planta " + (i + 1) + " no tiene
    ↪  enemigos");
        }
    }
    return new Dungeon(floors);
}
```

El mapa de cada planta llega en el capítulo 5 como un paso más de `FloorBuilder`; no hace falta tocar nada de lo anterior. Los archivos completos están en la solución del ejercicio de proyecto.

4.6 El olor que pide Singleton: la configuración que todos leen

`Main` ya reparte una semilla de azar y una fórmula de daño. Pronto querrás más datos comunes: el número de plantas, la semilla. ¿Cómo llegan a quien los necesita? La salida fácil es un acceso global. Así lo harían tres clases del juego:

Hipotético: tres clases leen la misma configuración

```
// en ConsoleUi:    if (floor > GlobalSettings.getInstance().floors()) { ... }
// en GameSession:  new Random(GlobalSettings.getInstance().seed())
// en Combat:       GlobalSettings.getInstance().seed()
```

Funciona, y el código es corto. Pero mira lo que ha pasado: `ConsoleUi`, `GameSession` y `Combat` dependen de `GlobalSettings` y **nada en sus constructores lo dice**. Para saber qué necesita una clase tienes que leer su cuerpo entero. Ese es el olor de fondo: estado global accesible desde cualquier sitio. Nystrom, en el capítulo que dedica a Singleton en su libro sobre patrones para juegos, lo resume con un título: por qué nos arrepentimos de usarlo, empezando por que es una variable global con otra cara [12].

El patrón, sin embargo, responde a una necesidad real: garantizar que de una clase hay una sola instancia y dar acceso a ella [7]. Lo veremos primero y luego juzgaremos si compensa.

4.7 Piezas del patrón Singleton

Rol	Clase en el ejemplo	Propósito	Paquete
Singleton (clase final)	`GlobalSettings`	Tiene una sola instancia, constructor privado	`app` (ejemplo)
Variante (enum)	`SettingsEnum` con `INSTANCE`	La misma garantía sin código extra	`app` (ejemplo)
Acceso (método estático)	`getInstance()`	Punto único de acceso	`app` (ejemplo)
Alternativa en Emberdeep (record)	`GameConfig`	Creada en `Main` y pasada por constructor	`app`
Clientes (clases)	Quienes leen la configuración	Piden el dato	varios

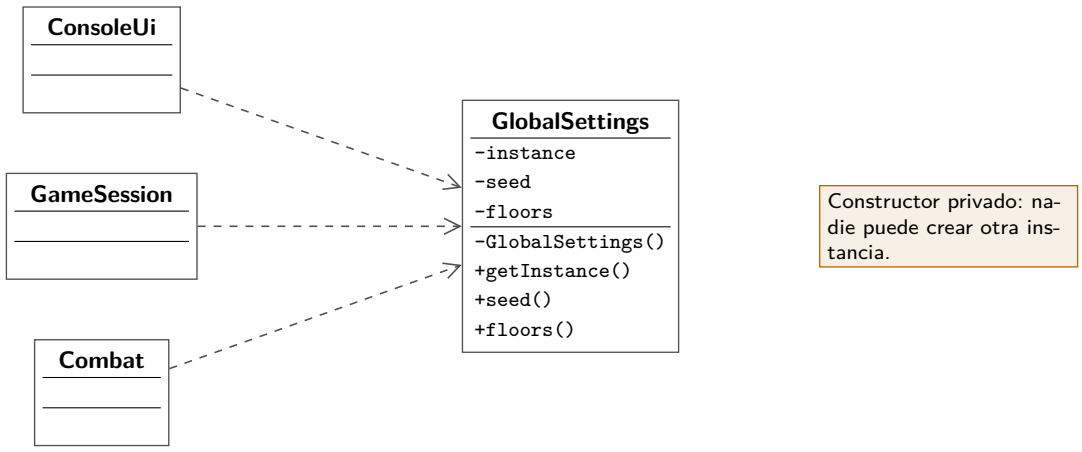


Diagrama UML del patrón Singleton: los clientes dependen de una instancia global que ninguno recibe por constructor.

4.8 El ejemplo de Singleton, pieza a pieza: holder y `enum`

4.8.1 Primer intento: perezoso, y con un fallo

La versión que primero se le ocurre a casi todo el mundo crea la instancia la primera vez que se pide («inicialización perezosa»):

GlobalSettings.java (versión ingenua, con fallo)

```
public final class GlobalSettings {
    private static GlobalSettings instance;

    private GlobalSettings() {}

```

```
    public static GlobalSettings getInstance() {
        if (instance == null) {
            instance = new GlobalSettings();
        }
        return instance;
    }
}
```

Con un solo hilo funciona. Con dos tiene un fallo, que explica el siguiente recuadro.

Dos hilos y una carrera de datos

Un *hilo* es una secuencia de instrucciones que Java ejecuta a la vez que otras. Un programa normal tiene uno solo, el de `main`. Para crear otro se pasa lo que debe hacer a un `Thread`; `start()` lo arranca y `join()` espera a que termine (puede lanzar `InterruptedException`, por eso `main` declara `throws`):

Dos hilos (fragmento)

```
Thread t = new Thread(() -> System.out.println("desde otro hilo"));
t.start();   // el hilo empieza a ejecutarse en paralelo
t.join();    // esperamos a que termine antes de seguir
```

Una *carrera de datos* ocurre cuando dos hilos leen y escriben lo mismo sin coordinarse, y el resultado depende de quién llega antes. En `getInstance` el hilo A comprueba `instance == null` y es cierto. Antes de que asigne, el hilo B hace la misma comprobación, también cierta. Ambos ejecutan `new` y hay **dos instancias**. No ocurre siempre: depende de la sincronización de los hilos, y por eso es difícil de reproducir. Se arregla haciendo que la creación no sea una comprobación en dos pasos.

4.8.2 Solución 1: el holder

Java inicializa una clase una sola vez y de forma segura entre hilos, la primera vez que se usa [3]. El truco, llamado *holder*, es poner la instancia en una clase interna que no se carga hasta que alguien la necesita:

app/GlobalSettings.java (ejemplo, no está en el proyecto)

```
package emberdeep.app;

public final class GlobalSettings {
    private final long seed = 42L;
    private final int floors = 3;

    private GlobalSettings() {}

    private static final class Holder {
        private static final GlobalSettings INSTANCE = new GlobalSettings();
    }

    public static GlobalSettings getInstance() {
        return Holder.INSTANCE;
    }
```

```
16
17      public long seed() { return seed; }
18      public int floors() { return floors; }
19  }
```

`Holder` no se inicializa hasta la primera llamada a `getInstance()`, así que es perezoso, y no hay ninguna comprobación que pueda cruzarse. No usa `synchronized`.

4.8.3 Solución 2: el `enum`

Un `enum` con un solo valor da la misma garantía con menos código, y además resiste la reflexión y la serialización, que pueden romper un constructor privado. Es la opción que Bloch prefiere [3]:

app/SettingsEnum.java (ejemplo, no está en el proyecto)

```
1  package emberdeep.app;
2
3  public enum SettingsEnum {
4      INSTANCE;
5
6      public long seed() { return 42L; }
7      public int floors() { return 3; }
8  }
```

Se usa como `SettingsEnum.INSTANCE.seed()`. Su límite: un `enum` no puede extender otra clase (sí implementar interfaces).

4.9 Si ya lo conoces: lo nuevo en Singleton es medir su coste

Escribir un Singleton ya lo sabías. Lo nuevo es ponerle precio:

- **Estado global.** Cualquier clase lo toca sin que su constructor lo diga. No ves las dependencias, y un cambio en la instancia afecta a clases que no esperabas.
- **Pruebas.** No puedes darle a una clase una configuración distinta para probarla, porque ella misma llama a `getInstance()`. Con una instancia por constructor sí. Lo practicarás en el ejercicio 4.4.
- **Hilos.** El Singleton ingenuo tiene una carrera de datos; el holder y el `enum` la evitan. Si el Singleton además guarda estado mutable, sus datos siguen siendo compartidos entre hilos.
- **Salida.** Quitar un Singleton de un proyecto es una refactorización conocida, *Inline Singleton* [10].

4.10 Singleton: cuándo sí y cuándo no; la alternativa de `Main`

La pregunta no es «¿hay una sola instancia?», sino «¿necesito además que se pueda llegar a ella desde cualquier sitio?». Casi siempre lo primero sí y lo segundo no. La alternativa: crear la instancia *una vez* en `Main`, el composition root del capítulo 1, y pasarla por constructor a quien la use. Sigue habiendo una sola, pero cada clase declara que la necesita.

Singleton compensa cuando el recurso es único de verdad y no se puede construir varias veces (un registro del sistema), el acceso desde muchos sitios es inevitable y no vas a necesitar sustituirlo en pruebas. **No compensa** para configuración, listas de datos o cualquier cosa que quieras probar con otro valor: ahí, una instancia pasada por constructor. Lo pagas en que algunas clases reciban un parámetro más, un coste pequeño y visible.

Emberdeep aplica la alternativa con un `record`:

app/GameConfig.java

```
1  package emberdeep.app;
2
3  import emberdeep.engine.combat.damage.DamageFormula;
4  import java.util.Objects;
5
6  public record GameConfig(long seed, int floors, DamageFormula defaultFormula) {
7      public GameConfig {
8          if (floors < 1) {
9              throw new IllegalArgumentException("floors debe ser al menos 1");
10         }
11         Objects.requireNonNull(defaultFormula, "defaultFormula");
12     }
13 }
```

`Main` crea un `GameConfig` una sola vez y lo recibe por su constructor; nadie más puede alcanzarlo sin que se lo pasen. En una prueba puedes crear otra configuración distinta sin que ninguna global se entere. Para una primera lectura del patrón y sus variantes, tienes [5] y [16]; lo importante es que usar un Singleton sea una decisión y no un hábito.

4.11 En el proyecto: H4 con Builder y Singleton

El hito H4 hace tres cosas: los héroes salen de un `HeroBuilder`, la mazmorra de tres plantas sale de un `DungeonBuilder` con `FloorBuilder` anidado, y los datos comunes viven en un `GameConfig` creado una sola vez en `Main`. Cambios:

- **Nuevo** `HeroBuilder` (en `engine/character/`); `Hero` no cambia.
- **Nuevo** `Floor`, `Dungeon` y `DungeonBuilder` en `engine/world/`.
- **Nuevo** `GameConfig` en `app/`: semilla 42, tres plantas y la fórmula plana por defecto.
- `Main` deja de ser solo un método estático: guarda el `GameConfig` que le pasa su propio `main`. El primer argumento es ahora el número de planta (1 a 3; la 3 es otra cripta). Con un segundo argumento `default` no se pregunta la fórmula: se usa la de la configuración.
- `GameSession`, `Combat` y el resto del motor no cambian.

Hito 4: héroes y mazmorra construidos paso a paso

Tras este hito, el grupo sale de tres `HeroBuilder`; la mazmorra tiene tres plantas (cripta, caverna, cripta) construidas con `DungeonBuilder`; y un `build()` inválido lanza `IllegalStateException`. La estructura:

Estructura de carpetas tras el capítulo 4

```
emberdeep/
|-- settings.gradle
|-- build.gradle
|-- gradlew
|-- gradlew.bat
|-- gradle/wrapper/
`-- src/main/java/emberdeep/
    |-- app/
    |   |-- Main.java              (composition root)
    |   `-- GameConfig.java
    |-- engine/
    |   |-- character/             Stats, Hero, HeroBuilder, Enemy,
    ↪ EnemyKind, Fighter
    |   |-- combat/
    |   |   |-- Combat.java
    |   |   |-- ai/                EnemyAi, FirstAliveAi, LowestHpAi,
    ↪ RandomTargetAi
    |   |   `-- damage/            DamageFormula, FlatFormula,
    ↪ ScaledFormula
    |   |-- item/                  Item, ItemKind, Inventory
    |   |-- session/               GameSession
    |   `-- world/
    |       |-- Dungeon.java, Floor.java, DungeonBuilder.java
    |       `-- biome/             BiomeFactory, CryptBiome, CavernBiome
    `-- ui/
        `-- ConsoleUi.java
```

Lo practicas en el ejercicio 4.5. Antes, cuatro ejercicios más pequeños, en otros dominios.

4.12 Ejercicios

Ejercicio 4.1: Un QuestBuilder que valida en build ★★☆☆☆

Vas a crear una misión (`Quest`) con dos datos obligatorios y dos opcionales. Son archivos nuevos, sin paquete, no de Emberdeep. Pasos:

1. Crea el `record Quest(String title, int reward, int minLevel, String giver)`.
2. Dentro del record, declara la clase `public static class Builder`, con los campos `title` y `reward` (obligatorios) y `minLevel` (valor por defecto 1) y `giver` (por defecto `"Anónimo"`).
3. Añade un método por campo, con el mismo nombre del campo, que guarde el valor y devuelva `this`.
4. Escribe `Quest build()`: lanza `IllegalStateException` con el mensaje «Falta el título» si `title` es `null` o está en blanco, y con «La recompensa debe ser positiva» si `reward` no es mayor que 0. Si todo está bien, devuelve el `Quest`.
5. En una clase `Main`, crea una misión «Limpiar la cripta» con recompensa 50 y solo esos datos, otra «Matar al rey goblin» con recompensa 200, nivel 3 y dador «Mara»,

y un intento con título «Sin premio» y sin recompensa, capturando la excepción e imprimiendo «Error: » y su mensaje.

Está bien si `java Main` imprime

```
Quest[title=Limpiar la cripta, reward=50, minLevel=1, giver=Anónimo]
Quest[title=Matar al rey goblin, reward=200, minLevel=3, giver=Mara]
Error: La recompensa debe ser positiva
```

y si los dos opcionales pueden omitirse sin pasar `null`.

Guiado ▪ solución en la pág. 106

Ejercicio 4.2: Dos hilos, dos instancias ★★☆☆☆

Este programa (no es de Emberdeep; dos archivos sin paquete) debería usar un único `MapRegistry`. El constructor tarda un poco en construirse: `Thread.sleep(100)` pausa el hilo 100 ms, y su `try/catch` es obligatorio porque `sleep` puede lanzar `InterruptedException`. Se simula así una creación lenta.

MapRegistry.java y Main.java

```java
public class MapRegistry {
    private static MapRegistry instance;
    private static int created = 0;

    private MapRegistry() {
        created++;
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
            throw new IllegalStateException(e);
        }
    }

    public static MapRegistry getInstance() {
        if (instance == null) {
            instance = new MapRegistry();
        }
        return instance;
    }

    public static int created() { return created; }
}

public class Main {
    public static void main(String[] args) throws InterruptedException {
        MapRegistry[] seen = new MapRegistry[2];
        Thread a = new Thread(() -> seen[0] = MapRegistry.getInstance());
        Thread b = new Thread(() -> seen[1] = MapRegistry.getInstance());
        a.start();
        b.start();
        a.join();
        b.join();
        System.out.println((seen[0] == seen[1]) + " " + MapRegistry.created
    ↪ ());
    }
}
```

1. Ejecútalo varias veces. ¿Qué imprime en casi todas y qué debería imprimir?
2. Explica qué ocurre, paso a paso, entre los dos hilos.
3. Corrígelo con el holder, sin tocar `Main`.

Está bien si, tras corregirlo, imprime siempre `true 1`, y si explicas la carrera con las palabras «comprobación» y «asignación».

Pista: *Ahora el hilo que llega segundo ve* `instance == null` *mientras el primero aún está dentro del constructor.*

Depuración ▪ solución en la pág. 107

Ejercicio 4.3: Record, Builder o with ★★★☆☆

Para cada clase de la tabla (ninguna es de Emberdeep), decide si basta un `record` con constructor compacto, hace falta un Builder, o conviene un método `with`.

Clase	Descripción
`Coordinate`	Dos enteros `x` e `y`, ambos no negativos.
`Spell`	Obligatorios: `name` y `manaCost` (mayor que 0). Opcionales: `damage`, `range`, `cooldown`, `element` y `area`. Regla: si `area` es mayor que 0, `cooldown` debe ser al menos 1.
`Stats`	`maxHp`, `attack`, `defense`, todos obligatorios. Al subir de nivel un héroe, solo cambia el ataque.

Entrega: (1) una tabla con la decisión de cada clase y dos criterios que la justifican (usa los de la sección 4.4); (2) el código que decidiste para cada una: el constructor compacto de `Coordinate`, el `Spell` con su Builder completo, y el método `withAttack` de `Stats`.
Está bien si cada decisión se apoya en al menos dos criterios (número de opcionales, reglas entre campos, legibilidad, un solo cambio), si el Builder valida la regla entre `area` y `cooldown` en `build()` y no en los pasos, y si el código de las tres clases es completo.

Diseño ▪ solución en la pág. 108

Ejercicio 4.4: De getInstance a una instancia pasada por constructor ★★★☆☆

Partes de estos archivos (no son de Emberdeep; sin paquete).

Settings, Welcome, Reward, Report y Main

```java
public final class Settings {
    private static final Settings INSTANCE = new Settings();
    private final String playerName = "Aria";
    private final int difficulty = 2;

    private Settings() {}

    public static Settings getInstance() { return INSTANCE; }
    public String playerName() { return playerName; }
    public int difficulty() { return difficulty; }
}

public class Welcome {
    public String text() {
        return "Hola, " + Settings.getInstance().playerName();
    }
}

public class Reward {
    public int gold() {
        return 10 * Settings.getInstance().difficulty();
    }
}

public class Report {
    public String text() {
        Settings s = Settings.getInstance();
        return s.playerName() + " juega en dificultad " + s.difficulty();
    }
}

public class Main {
    public static void main(String[] args) {
        System.out.println(new Welcome().text());
        System.out.println("Recompensa: " + new Reward().gold());
        System.out.println(new Report().text());
    }
}
```

Refactoriza para que no exista `getInstance()`: `Settings` pasa a ser un `record Settings(String playerName, int difficulty)`, y `Welcome`, `Reward` y `Report` reciben el `Settings` por constructor. Después, en `main`, usa cada clase con el `Settings("Aria", 2)` y con otro, `Settings("Test", 5)`.
Está bien si `java Main` imprime

```
Hola, Aria
Recompensa: 20
Aria juega en dificultad 2
Hola, Test
Recompensa: 50
Test juega en dificultad 5
```

y si ninguna clase del programa contiene la palabra `getInstance`.

Pista: *Cada una de las tres clases guarda el* `Settings` *en un campo* `private final`*.*

Práctica ▪ solución en la pág. 111

Ejercicio 4.5: Hito 4: héroes y mazmorra en Emberdeep ★★★☆☆

Implementa el hito H4 partiendo del estado del capítulo anterior (solución del ejercicio 3.5), en tres pasos.

1. **`HeroBuilder`.** En `engine/character/`, con los pasos `name` y `stats` y un `build()` que lanza `IllegalStateException` si el nombre es `null` o está en blanco, si faltan las estadísticas, o si la vida o el ataque no son positivos o la defensa es negativa. `Main` crea con él a Aria (30, 8, 3), Borin (40, 6, 5) y Cael (24, 9, 2).
2. **Mazmorra.** En `engine/world/`, los records `Floor(biome, enemies)` y `Dungeon(floors)` (con copia inmutable de las listas) y `DungeonBuilder` con `FloorBuilder` interno (`biome`, `enemies(List<EnemyKind>)`, `done()`). `build()` falla si no hay plantas o si alguna no tiene enemigos. `Main` monta cripta, caverna y cripta, reutilizando el mismo `CryptBiome`.
3. **`GameConfig`.** En `app/`, un `record` con semilla, número de plantas y fórmula por defecto, que `Main` crea una sola vez (semilla 42, 3 plantas, `FlatFormula`) y recibe por constructor. El primer argumento es el número de planta (por defecto 1; fuera de rango, `IllegalArgumentException`); con un segundo argumento `default` no se pregunta la fórmula.

Está bien si:

1. Sin argumentos y con el argumento `2`, el juego da exactamente las mismas trazas que al final del capítulo anterior (cripta y caverna), y con el argumento `3` las de la cripta, con los mismos enemigos. Por ejemplo, para la planta 3:

   Planta 3

   ```
   ./gradlew run --args="3"
   ```

2. Con dos argumentos, el segundo `default`, no aparece la pregunta de la fórmula y se juega con la fórmula plana:

   Sin pregunta de fórmula

   ```
   ./gradlew run --args="1 default"
   ```

3. Con el argumento `4` el programa termina con `IllegalArgumentException`.
4. Si añades temporalmente estas líneas al principio de `play`, la salida son tres mensajes; bórralas después:

Comprobación temporal en Main.java

```
List<Runnable> checks = List.of(
        () -> new DungeonBuilder().build(),
        () -> new DungeonBuilder().floor()
                .biome(new CryptBiome(new Random(1))).done().build(),
        () -> new HeroBuilder().name(" ").build());
for (Runnable check : checks) {
    try {
        check.run();
    } catch (IllegalStateException e) {
        System.out.println(e.getMessage());
    }
}
```

```
La mazmorra necesita al menos una planta
La planta 1 no tiene enemigos
El héroe necesita un nombre
```

Pista: *Un `Runnable` es una función sin parámetros ni resultado; `check.run()` la ejecuta.*

Proyecto ▪ solución en la pág. 112

4.13 Resumen

Resumen

- El olor de Builder es un constructor con muchos parámetros sin nombre, o un objeto que solo vale si se monta entero. Builder da un paso por dato y valida todo en `build()`.
- Un `record` con constructor compacto basta con pocos parámetros y reglas sobre sus propios campos; el Builder compensa con muchos opcionales, reglas entre datos o montaje anidado. Para cambiar un dato, un `with`.
- Un builder anidado (`FloorBuilder` dentro de `DungeonBuilder`) monta un árbol; `done()` vuelve al padre. No se usa el «director» del GoF.
- El Singleton ingenuo tiene una carrera de datos con dos hilos; el holder y el `enum` la evitan.
- El coste de Singleton es el estado global: dependencias ocultas y pruebas difíciles. La alternativa es crear la instancia una vez en `Main` y pasarla por constructor.
- H4: `HeroBuilder`, `Dungeon`, `Floor`, `DungeonBuilder` con `FloorBuilder` y `GameConfig`; mazmorra de tres plantas.

Soluciones de los ejercicios

Solución del ejercicio 4.1

enunciado en la pág. 99

Quest.java

```
public record Quest(String title, int reward, int minLevel, String giver) {

    public static class Builder {
        private String title;
        private int reward;
        private int minLevel = 1;
        private String giver = "Anónimo";

        public Builder title(String title) {
            this.title = title;
            return this;
        }

        public Builder reward(int reward) {
            this.reward = reward;
            return this;
        }

        public Builder minLevel(int minLevel) {
            this.minLevel = minLevel;
            return this;
        }

        public Builder giver(String giver) {
            this.giver = giver;
            return this;
        }

        public Quest build() {
            if (title == null || title.isBlank()) {
                throw new IllegalStateException("Falta el título");
            }
            if (reward <= 0) {
                throw new IllegalStateException("La recompensa debe ser
    ↪ positiva");
            }
            return new Quest(title, reward, minLevel, giver);
        }
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Quest simple = new Quest.Builder()
                .title("Limpiar la cripta")
                .reward(50)
                .build();
        Quest full = new Quest.Builder()
                .title("Matar al rey goblin")
                .reward(200)
                .minLevel(3)
                .giver("Mara")
                .build();
        System.out.println(simple);
        System.out.println(full);
        try {
            new Quest.Builder().title("Sin premio").build();
        } catch (IllegalStateException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Por qué funciona. Los obligatorios no tienen valor por defecto útil (`null` y 0) y `build()` los comprueba; los opcionales nacen con su valor por defecto, así que se pueden omitir. Cada paso devuelve `this`, y por eso se encadenan. Desde fuera, la clase se nombra `Quest.Builder` porque es una clase anidada `static`.

Errores típicos:

- Olvidar `static`: sin él, `new Quest.Builder()` necesitaría un `Quest` previo y no compila.
- Validar en cada paso en lugar de en `build()`: el orden de los pasos pasa a importar.
- Devolver `void` en un paso: la cadena se rompe.

Solución del ejercicio 4.2

enunciado en la pág. 100

1. En casi todas las ejecuciones imprime `false 2`; debería imprimir `true 1` (la misma instancia y una sola creación).
2. El hilo A llama a `getInstance()`, comprueba `instance == null` (cierto) y entra en el constructor, que tarda 100 ms. `instance` aún no está asignada, porque la asignación ocurre cuando el constructor termina. Mientras tanto el hilo B hace la misma comprobación, que sigue siendo cierta, y también construye. Cada hilo asigna la suya: dos instancias. La comprobación y la asignación son dos pasos separados, y nada impide que otro hilo se cuele entre ellos.
3. Con el holder (`Main` no cambia):

MapRegistry.java (corregido)

```
public class MapRegistry {
    private static int created = 0;

    private MapRegistry() {
        created++;
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
            throw new IllegalStateException(e);
        }
    }

    private static class Holder {
        private static final MapRegistry INSTANCE = new MapRegistry();
    }

    public static MapRegistry getInstance() {
        return Holder.INSTANCE;
    }

    public static int created() { return created; }
}
```

Por qué funciona. Java inicializa `Holder` una sola vez; si dos hilos lo piden a la vez, uno inicializa y el otro espera. No hay comprobación en dos pasos. Ahora imprime siempre `true` 1.

Alternativa. Marcar `getInstance` como `synchronized` también lo arregla, pero bloquea cada llamada aunque la instancia ya exista.

Errores típicos:

- Quitar el `sleep` y concluir que «ya funciona»: la carrera sigue ahí, solo se vuelve rara. No funcionar a veces es peor que no funcionar nunca.
- Arreglar `Main` (por ejemplo, serializando los hilos con `join` antes de arrancar el segundo): el fallo está en la clase, no en quien la usa.

Solución del ejercicio 4.3 enunciado en la pág. 101

Clase	Decisión	Criterios
`Coordinate`	`record` con constructor compacto	Dos parámetros obligatorios; la regla (no negativos) es sobre sus propios campos. Un Builder sería más código que el propio record.
`Spell`	Builder	Cinco opcionales (la llamada por posición sería ilegible) y una regla entre campos (`area` y `cooldown`) que solo se puede comprobar con todos los datos.
`Stats`	`record` más `withAttack`	Tres campos obligatorios, pero el uso real es copiar y cambiar uno solo; un método `with` lo expresa sin builder.

Coordinate.java y Stats.java

```java
public record Coordinate(int x, int y) {
    public Coordinate {
        if (x < 0 || y < 0) {
            throw new IllegalArgumentException("Coordenada negativa");
        }
    }
}

public record Stats(int maxHp, int attack, int defense) {
    public Stats withAttack(int newAttack) {
        return new Stats(maxHp, newAttack, defense);
    }
}
```

Spell.java

```java
public record Spell(String name, int manaCost, int damage, int range,
                    int cooldown, String element, int area) {

    public static class Builder {
        private String name;
        private int manaCost;
        private int damage;
        private int range = 1;
        private int cooldown;
        private String element = "neutro";
        private int area;

        public Builder name(String name) { this.name = name; return this; }
        public Builder manaCost(int v) { this.manaCost = v; return this; }
        public Builder damage(int v) { this.damage = v; return this; }
        public Builder range(int v) { this.range = v; return this; }
        public Builder cooldown(int v) { this.cooldown = v; return this; }
        public Builder element(String v) { this.element = v; return this; }
        public Builder area(int v) { this.area = v; return this; }

        public Spell build() {
            if (name == null || name.isBlank()) {
                throw new IllegalStateException("Falta el nombre");
            }
            if (manaCost <= 0) {
                throw new IllegalStateException("El coste debe ser positivo
    ↪ ");
            }
            if (area > 0 && cooldown < 1) {
                throw new IllegalStateException("Un hechizo de área
    ↪ necesita enfriamiento");
            }
            return new Spell(name, manaCost, damage, range, cooldown,
    ↪ element, area);
        }
    }
}
```

Criterios para valorar otras soluciones. Vale otra decisión si la justificas con los criterios: por ejemplo, `Coordinate` con Builder solo se defiende si prevés muchos campos más. Para `Stats`, un Builder sería razonable si el héroe ganara muchos campos opcionales; si mañana `Stats` tuviera diez campos, la decisión cambiaría.

Errores típicos:

- Poner la regla de `area` y `cooldown` en los pasos `area` o `cooldown`: depende del orden en que los llames.
- Dar un Builder a `Coordinate` «por si acaso»: sobrediseño.

Solución del ejercicio 4.4

enunciado en la pág. 102

`Settings.java`, `Welcome.java`, `Reward.java` y `Report.java` (cuatro archivos)

```java
public record Settings(String playerName, int difficulty) {}

public class Welcome {
    private final Settings settings;

    public Welcome(Settings settings) {
        this.settings = settings;
    }

    public String text() {
        return "Hola, " + settings.playerName();
    }
}

public class Reward {
    private final Settings settings;

    public Reward(Settings settings) {
        this.settings = settings;
    }

    public int gold() {
        return 10 * settings.difficulty();
    }
}

public class Report {
    private final Settings settings;

    public Report(Settings settings) {
        this.settings = settings;
    }

    public String text() {
        return settings.playerName() + " juega en dificultad "
                + settings.difficulty();
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        show(new Settings("Aria", 2));
        show(new Settings("Test", 5));
    }

    private static void show(Settings settings) {
        System.out.println(new Welcome(settings).text());
        System.out.println("Recompensa: " + new Reward(settings).gold());
        System.out.println(new Report(settings).text());
    }
}
```

Por qué funciona. Cada clase declara en su constructor lo que necesita, y `main`, el composition root, decide qué `Settings` le da. Como ya no hay acceso global, se puede usar cada clase con una configuración distinta en la misma ejecución, que con `getInstance()` era imposible. Sigue habiendo un solo `Settings` *por uso*, creado en un único sitio.

Errores típicos:

- Dejar un constructor sin argumentos que siga llamando a `getInstance()`.
- Conservar `Settings` como clase con constructor privado: no se puede crear uno distinto.
- Olvidar pasar el `Settings` a una de las tres clases: no compila, y eso es lo que se quiere.

Solución del ejercicio 4.5

enunciado en la pág. 104

Los archivos nuevos o cambiados son seis; `GameSession`, `Combat`, `ConsoleUi`, las fábricas de bioma y el resto no cambian.

engine/character/HeroBuilder.java

HeroBuilder.java

```java
package emberdeep.engine.character;

public class HeroBuilder {
    private String name;
    private Stats stats;

    public HeroBuilder name(String name) {
        this.name = name;
        return this;
    }

    public HeroBuilder stats(Stats stats) {
        this.stats = stats;
        return this;
    }

    public Hero build() {
        if (name == null || name.isBlank()) {
            throw new IllegalStateException("El héroe necesita un nombre");
        }
        if (stats == null) {
            throw new IllegalStateException("El héroe necesita estadísticas
    ↪ ");
        }
        if (stats.maxHp() <= 0 || stats.attack() <= 0 || stats.defense() <
    ↪ 0) {
            throw new IllegalStateException("Estadísticas no válidas: " +
    ↪ stats);
        }
        return new Hero(name, stats);
    }
}
```

engine/world/Floor.java y Dungeon.java

Floor.java

```java
package emberdeep.engine.world;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.world.biome.BiomeFactory;
import java.util.List;

public record Floor(BiomeFactory biome, List<EnemyKind> enemies) {
    public Floor {
        enemies = List.copyOf(enemies);
    }
}
```

Dungeon.java

```java
package emberdeep.engine.world;

import java.util.List;

public record Dungeon(List<Floor> floors) {
    public Dungeon {
        floors = List.copyOf(floors);
    }
}
```

engine/world/DungeonBuilder.java

DungeonBuilder.java

```java
package emberdeep.engine.world;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.world.biome.BiomeFactory;
import java.util.ArrayList;
import java.util.List;

public class DungeonBuilder {
    private final List<Floor> floors = new ArrayList<>();

    public FloorBuilder floor() {
        return new FloorBuilder();
    }

    public Dungeon build() {
        if (floors.isEmpty()) {
            throw new IllegalStateException("La mazmorra necesita al menos una planta");
        }
        for (int i = 0; i < floors.size(); i++) {
            Floor floor = floors.get(i);
            if (floor.biome() == null) {
                throw new IllegalStateException("La planta " + (i + 1) + " no tiene bioma");
            }
            if (floor.enemies().isEmpty()) {
                throw new IllegalStateException("La planta " + (i + 1) + " no tiene enemigos");
            }
        }
        return new Dungeon(floors);
    }

    public class FloorBuilder {
        private BiomeFactory biome;
        private List<EnemyKind> enemies = List.of();

        public FloorBuilder biome(BiomeFactory biome) {
            this.biome = biome;
            return this;
        }

        public FloorBuilder enemies(List<EnemyKind> enemies) {
            this.enemies = enemies;
            return this;
        }

        public DungeonBuilder done() {
            floors.add(new Floor(biome, enemies));
            return DungeonBuilder.this;
        }
    }
}
```

app/GameConfig.java

GameConfig.java

```java
package emberdeep.app;

import emberdeep.engine.combat.damage.DamageFormula;
import java.util.Objects;

public record GameConfig(long seed, int floors, DamageFormula
    ↪ defaultFormula) {
    public GameConfig {
        if (floors < 1) {
            throw new IllegalArgumentException("floors debe ser al menos 1")
    ↪ ;
        }
        Objects.requireNonNull(defaultFormula, "defaultFormula");
    }
}
```

app/Main.java (en tres tramos seguidos, dentro de la misma clase)

Main.java (1/3: imports y arranque)

```java
package emberdeep.app;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.damage.ScaledFormula;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.Dungeon;
import emberdeep.engine.world.DungeonBuilder;
import emberdeep.engine.world.Floor;
import emberdeep.engine.world.biome.BiomeFactory;
import emberdeep.engine.world.biome.CavernBiome;
import emberdeep.engine.world.biome.CryptBiome;
import emberdeep.ui.ConsoleUi;
import java.util.List;
import java.util.Random;
import java.util.Scanner;

public class Main {
    private final GameConfig config;

    private Main(GameConfig config) {
        this.config = config;
    }

    public static void main(String[] args) {
        GameConfig config = new GameConfig(42L, 3, new FlatFormula());
        new Main(config).play(args);
    }
```

Main.java (2/3: la partida)

```
    private void play(String[] args) {
        int floorNumber = chooseFloor(args);
        ConsoleUi ui = new ConsoleUi(new Scanner(System.in), System.out);
        DamageFormula formula = chooseFormula(ui, args);
        Random random = new Random(config.seed());

        BiomeFactory crypt = new CryptBiome(random);
        BiomeFactory cavern = new CavernBiome(random);
        List<EnemyKind> cryptKinds = List.of(
                EnemyKind.SKELETON, EnemyKind.ASH_RAT, EnemyKind.ZOMBIE);
        List<EnemyKind> cavernKinds = List.of(
                EnemyKind.GOBLIN, EnemyKind.BAT, EnemyKind.SPIDER);

        Dungeon dungeon = new DungeonBuilder()
                .floor().biome(crypt).enemies(cryptKinds).done()
                .floor().biome(cavern).enemies(cavernKinds).done()
                .floor().biome(crypt).enemies(cryptKinds).done()
                .build();
        Floor floor = dungeon.floors().get(floorNumber - 1);

        GameSession session = new GameSession(
                heroes(), floor.biome(), floor.enemies(), formula);
        ui.run(session);
    }
```

Main.java (3/3: auxiliares y llave final)

```
    private DamageFormula chooseFormula(ConsoleUi ui, String[] args) {
        if (args.length > 1 && args[1].equals("default")) {
            return config.defaultFormula();
        }
        return ui.askFormula() == 1 ? new FlatFormula() : new ScaledFormula
    ↪ ();
    }

    private int chooseFloor(String[] args) {
        int floor = args.length > 0 ? Integer.parseInt(args[0]) : 1;
        if (floor < 1 || floor > config.floors()) {
            throw new IllegalArgumentException(
                    "La planta debe estar entre 1 y " + config.floors());
        }
        return floor;
    }

    private List<Hero> heroes() {
        return List.of(
                new HeroBuilder().name("Aria")
                        .stats(new Stats(30, 8, 3)).build(),
                new HeroBuilder().name("Borin")
                        .stats(new Stats(40, 6, 5)).build(),
                new HeroBuilder().name("Cael")
                        .stats(new Stats(24, 9, 2)).build());
    }
}
```

Por qué funciona. Los héroes salen de tres builders, con los mismos números que antes, y la mazmorra, de un builder anidado que valida al final. El mismo `CryptBiome` sirve a las plantas 1 y 3: un bioma es una fábrica sin estado propio salvo el `Random` compartido. Como cada ejecución juega una sola planta con `Random(42)` recién creado y el builder no consume números aleatorios, las trazas de las plantas 1 y 2 son las del capítulo anterior, y la 3 repite la de la 1. `GameConfig` se crea en `main` y se pasa al constructor de `Main`; nadie más lo ve. Para comprobar los mensajes de validación, pega temporalmente en `play` el fragmento del enunciado: da los tres textos esperados.

Errores típicos:

- Olvidar `done()`: la planta no se añade, y `build()` dice que no hay plantas.
- Hacer `FloorBuilder` `static`: no puede ver `floors` ni escribir `DungeonBuilder.this`.
- Crear un `new Random(...)` por bioma o usar la semilla en otro sitio: cambia las trazas. Se crea uno, con `config.seed()`.
- Pasar un número de planta fuera de 1 a 3 y esperar la cripta por defecto: ahora es un error.

Estructura final: la del árbol de la sección «En el proyecto». Este es el estado del que parte el capítulo siguiente.

CAPÍTULO 5

Prototype y Flyweight: copiar o compartir

Objetivos del capítulo

- Reconocer el olor que pide un Prototype: un objeto ya montado del que quieres variantes con un retoque.
- Aplicar Prototype con una plantilla mutable, un `copy()` profundo y un registro, y demostrar por qué falla la copia superficial.
- Reconocer el olor que pide un Flyweight y separar el estado intrínseco del extrínseco.
- Aplicar Flyweight con una fábrica de flyweights, `Map.computeIfAbsent`, la identidad con `==` y un contador de instancias.
- Elegir entre Factory, Prototype y Flyweight, y dar los hitos H5 y H6: plantillas de enemigo con élites y un mapa ASCII con casillas compartidas.

Los enemigos de Emberdeep se construyen desde cero cada vez que una fábrica de bioma lo pide, y no tienen habilidades. Pronto querrás un «élite»: un zombi igual que el normal, pero con más vida y un golpe extra. Y querrás un mapa, donde cientos de casillas repiten los mismos tres datos. Son dos formas de pensar un objeto repetido. Con la primera (Prototype) *copias* algo ya montado y lo retocas. Con la segunda (Flyweight) *compartes* lo que nunca cambia, en lugar de copiarlo. Empiezas por copiar.

5.1 El olor que pide Prototype: montar la misma plantilla cien veces

Hoy una fábrica de bioma monta cada enemigo con un `new` y una lista de datos. Supón que los enemigos ya tuvieran habilidades (en este capítulo las tendrán) y que quieres el zombi normal y, además, el zombi élite:

Hipotético: dos zombis montados a mano

```
Enemy zombie = new Enemy("Zombi", EnemyKind.ZOMBIE, new Stats(22, 5, 1),
        new FirstAliveAi(), List.of(new Skill("Abrazo pútrido", 7)));

Enemy eliteZombie = new Enemy("Élite Zombi", EnemyKind.ZOMBIE, new Stats(33, 7,
    ↪ 2),
        new FirstAliveAi(), List.of(new Skill("Abrazo pútrido", 7),
                                    new Skill("Furia", 12)));
```

El élite repite todo el zombi y cambia tres cosas. Si mañana el abrazo pasa de 7 a 8, tienes que acordarte de cambiarlo en los dos sitios, y en los seis tipos de enemigo, y en todas las variantes que se añadan. Este es el olor: **varios objetos casi iguales que se montan repitiendo los mismos datos**. Lo que quieres decir es «dame otro como este, pero con esto distinto».

Prototype hace justo eso. Su intención en el catálogo es indicar qué objetos crear mediante una instancia prototípica y crear los nuevos *copiándola* [7]. No se parte de una clase y un `new`, sino de un objeto ya montado. En Emberdeep, ese objeto será una *plantilla* de enemigo, y la copia se convertirá en un enemigo de verdad. Nystrom dedica un capítulo a este patrón, con monstruos que se crean con «spawners» como ejemplo [12].

5.2 Piezas del patrón Prototype

Rol	Clase en el ejemplo	Propósito	Paquete
Prototipo (clase mutable)	`EnemyTemplate`	Objeto montado que sabe copiarse con `copy()`	`engine.character`
Parte mutable copiada (clase y record)	`List<Skill>` y `Skill` (inmutable)	La lista se duplica; los `Skill` se comparten	`engine.character`
Registro de prototipos (clase)	`TemplateRegistry`	Guarda una plantilla por `EnemyKind`	`engine.character`
Variante (método)	`asElite()`	Copia más ajustes de estadísticas y una habilidad	`engine.character`
Alternativa evitada (interfaz del JDK)	`Cloneable`	No se usa: se explica por qué	`java.lang`
Cliente (clase)	`CryptBiome`, `CavernBiome`	Piden una copia y la convierten en `Enemy`	`engine.world.biome`

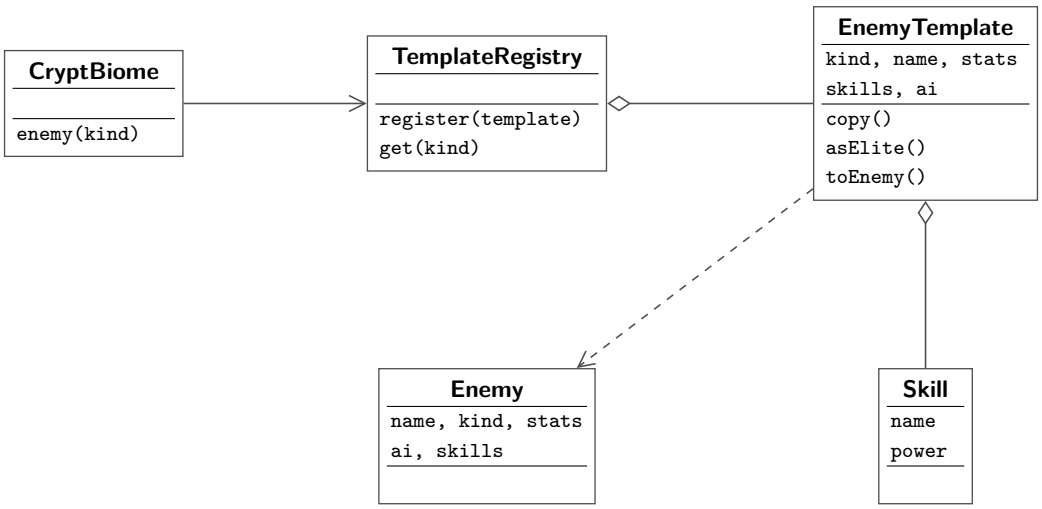


Diagrama UML del patrón Prototype: el cliente pide al registro una plantilla, la copia y la convierte en `Enemy`.

Sin interfaz Prototype

El catálogo original define una interfaz `Prototype` con un método de clonado, porque el cliente debe copiar objetos de clases que no conoce. Aquí solo hay una clase prototípica, `EnemyTemplate`, así que no hace falta la interfaz y el diagrama no la dibuja.

5.3 El ejemplo de Prototype, pieza a pieza: `EnemyTemplate.copy()`

La habilidad. Una `Skill` es un dato: un nombre y una potencia. Hasta el capítulo 7 solo se guarda, no se usa. Es un `record`, y por eso inmutable:

`engine/character/Skill.java`

```
package emberdeep.engine.character;

public record Skill(String name, int power) {}
```

El enemigo. El `Enemy` gana un campo, la lista de habilidades. Sigue siendo un record, y conserva el `kind` del capítulo 3:

`engine/character/Enemy.java`

```
package emberdeep.engine.character;

import emberdeep.engine.combat.ai.EnemyAi;
import java.util.List;

public record Enemy(String name, EnemyKind kind, Stats stats, EnemyAi ai,
                    List<Skill> skills) {}
```

La plantilla. `EnemyTemplate` es una clase *mutable*: puedes cambiarle el nombre, las estadísticas y las habilidades. Con un record inmutable no tendría sentido copiar (compartir el original sería seguro), así que el patrón pide un objeto con estado que cambia. Los campos y los dos constructores:

`engine/character/EnemyTemplate.java (1/2: datos y copia)`

```
package emberdeep.engine.character;

import emberdeep.engine.combat.ai.EnemyAi;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class EnemyTemplate {
    private final EnemyKind kind;
    private String name;
    private Stats stats;
    private final List<Skill> skills;
    private final EnemyAi ai;

    public EnemyTemplate(EnemyKind kind, String name, Stats stats, EnemyAi ai)
    ↪ {
        this(kind, name, stats, new ArrayList<>(), ai);
    }

    private EnemyTemplate(EnemyKind kind, String name, Stats stats,
                          List<Skill> skills, EnemyAi ai) {
        this.kind = kind;
        this.name = name;
        this.stats = stats;
```

```
        this.skills = skills;
        this.ai = ai;
    }

    public EnemyTemplate copy() {
        return new EnemyTemplate(kind, name, stats, new ArrayList<>(skills), ai
    ↪ );
    }

    public Enemy toEnemy() {
        return new Enemy(name, kind, stats, ai, List.copyOf(skills));
    }
```

El constructor privado recibe la lista tal cual; el público crea una vacía. `copy()` es el corazón del patrón. Distingue tres casos de lo que contiene la plantilla:

- **Inmutable: se comparte.** `kind`, `name` (un `String`), `stats` (un record) no pueden cambiar por dentro; si cambias el nombre de la copia, asignas un `String` nuevo y el original no se entera.
- **Mutable: se duplica.** La lista `skills` sí cambia por dentro (`add`). Por eso la copia recibe `new ArrayList<>(skills)`, una lista nueva con los mismos elementos. Como cada `Skill` es inmutable, basta copiar la lista, no sus elementos. Esto es una **copia profunda** de lo que hace falta.
- **Compartido a propósito.** La IA (`ai`) se comparte entre la plantilla y sus copias: es una estrategia del capítulo 2, y la de la cripta lleva dentro el único `Random` del juego. Duplicarla rompería la traza con semilla.

`toEnemy()` cierra el trabajo: crea el `Enemy` con el mismo `kind` y una copia inmutable de las habilidades (`List.copyOf`), para que el enemigo ya no comparta nada con la plantilla. La segunda mitad de la clase añade la variante élite, los modificadores y los accesos:

engine/character/EnemyTemplate.java (2/2: élite y accesos)

```

    public EnemyTemplate asElite() {
        EnemyTemplate elite = copy();
        elite.setName("Élite " + name);
        elite.setStats(new Stats(stats.maxHp() * 3 / 2,
                stats.attack() + 2, stats.defense() + 1));
        elite.addSkill(new Skill("Furia", 12));
        return elite;
    }

    public EnemyTemplate addSkill(Skill skill) {
        skills.add(skill);
        return this;
    }

    public void setName(String name) { this.name = name; }
    public void setStats(Stats stats) { this.stats = stats; }

    public EnemyKind kind() { return kind; }
    public String name() { return name; }
    public Stats stats() { return stats; }
```

```
    public EnemyAi ai() { return ai; }

    public List<Skill> skills() {
        return Collections.unmodifiableList(skills);
    }
}
```

`asElite()` es la «variante» del patrón: una copia a la que se le aplican ajustes (vida por uno y medio, un punto más de defensa, dos de ataque y una habilidad nueva). Lo más importante es que *primero* copia y *después* retoca la copia; la plantilla de partida no cambia. `skills()` devuelve una vista que no se puede modificar desde fuera: el único camino para añadir una habilidad es `addSkill`.

El registro. Una plantilla por cada tipo de enemigo, en un `EnumMap`:

engine/character/TemplateRegistry.java

```
package emberdeep.engine.character;

import java.util.EnumMap;
import java.util.Map;

public class TemplateRegistry {
    private final Map<EnemyKind, EnemyTemplate> templates =
            new EnumMap<>(EnemyKind.class);

    public void register(EnemyTemplate template) {
        templates.put(template.kind(), template);
    }

    public EnemyTemplate get(EnemyKind kind) {
        EnemyTemplate template = templates.get(kind);
        if (template == null) {
            throw new IllegalArgumentException("No hay plantilla para " + kind)
    ↪ ;
        }
        return template;
    }
}
```

`get` devuelve el prototipo registrado, no una copia. Regla de uso: **nunca lo modifiques ni lo conviertas directamente**; pide siempre `copy()`. Si lo retocases, cambiarías todos los enemigos futuros de ese tipo.

El cliente. Cada bioma tiene su propio registro y lo rellena en el constructor. Fíjate en que el constructor sigue recibiendo el `Random` (y por eso `Main` no cambia), y en que el `switch` sobre los seis tipos desaparece: el registro solo conoce los tipos de su bioma, y pedir uno ajeno lanza `IllegalArgumentException`, como antes.

engine/world/biome/CryptBiome.java (fragmento: sin imports ni objetos)

```
public class CryptBiome implements BiomeFactory {
    private final TemplateRegistry registry = new TemplateRegistry();

    public CryptBiome(Random random) {
```

```
        registry.register(new EnemyTemplate(EnemyKind.SKELETON, "Esqueleto",
                new Stats(18, 7, 2), new RandomTargetAi(random))
                .addSkill(new Skill("Golpe de hueso", 9)));
        registry.register(new EnemyTemplate(EnemyKind.ASH_RAT, "Rata de ceniza"
    ↪ ,
                new Stats(8, 5, 0), new LowestHpAi())
                .addSkill(new Skill("Mordisco sucio", 6)));
        registry.register(new EnemyTemplate(EnemyKind.ZOMBIE, "Zombi",
                new Stats(22, 5, 1), new FirstAliveAi())
                .addSkill(new Skill("Abrazo pútrido", 7)));
    }

    @Override
    public Enemy enemy(EnemyKind kind) {
        return registry.get(kind).copy().toEnemy();
    }
    // item(ItemKind) no cambia
}
```

`registry.get(kind).copy().toEnemy()` resume el patrón: del prototipo, una copia; de la copia, el producto. Esto también significa que `CryptBiome` sigue siendo una fábrica del capítulo 3: Prototype no la sustituye, trabaja *por dentro*. El mismo registro puede dar una copia por llamada, y cada `Enemy` resultante tiene su propia lista de habilidades.

5.4 Prototype: el bug de la copia superficial

Una copia *superficial* duplica el objeto pero no lo que este tiene dentro: los campos que son referencias apuntan a los mismos objetos que el original. Si esos objetos cambian, cambian para los dos. Es el fallo clásico de Prototype, y se ve con una sola línea mal escrita en `copy()`:

Roto: copy() que comparte la lista

```
public EnemyTemplate copy() {
    return new EnemyTemplate(kind, name, stats, skills, ai);   // skills, sin
    ↪ copiar
}
```

Compila, y la copia parece correcta. Mira qué pasa al crear un élite y mirar el original:

Comprobación (fragmento)

```
EnemyTemplate zombie = new EnemyTemplate(EnemyKind.ZOMBIE, "Zombi",
        new Stats(22, 5, 1), new FirstAliveAi())
        .addSkill(new Skill("Abrazo pútrido", 7));
EnemyTemplate elite = zombie.asElite();
System.out.println(zombie.name() + " " + zombie.skills().size());
System.out.println(elite.name() + " " + elite.skills().size());
```

Con la copia superficial

```
Zombi 2
Élite Zombi 2
```

El zombi normal ya tiene la *Furia* del élite. `asElite()` añadió la habilidad a «su» lista, que era la lista del original. El nombre sí se separó, porque asignar un `String` nuevo no toca el anterior; la lista no, porque `add` cambia el objeto compartido. Con `new ArrayList<>(skills)` en `copy()` la salida es `Zombi 1` y `Élite Zombi 2`. Se arregla igual en cualquier clase: por cada campo, pregunta si es inmutable (se comparte) o mutable (se duplica). Lo practicas en el ejercicio 5.1.

5.5 Si ya lo conoces: lo nuevo en Prototype es la copia profunda y el registro

Si ya conocías Prototype, lo nuevo aquí es:

- **Qué se copia y qué se comparte.** La copia profunda no es «copiar todo»: se duplica lo mutable (la lista), se comparte lo inmutable (los records) y lo compartido a propósito (la IA con su `Random`).
- **El registro de prototipos.** En lugar de que cada cliente guarde su plantilla, un `TemplateRegistry` la entrega por tipo; es la pieza que lo conecta con las fábricas.
- **Por qué no `Cloneable`.** Es una interfaz sin métodos: marca a una clase como clonable, pero el método `Object.clone()` es protegido, lanza una excepción comprobada y hace una copia superficial. Para hacerlo bien hay que sobrescribirlo, llamar a `super.clone()` y arreglar a mano cada campo mutable, y todo choca con los campos `final`. Bloch recomienda un constructor o un método de copia en su lugar [3], que es lo que hace `copy()`.
- **Convive con Factory.** `CryptBiome` es una fábrica que, por dentro, copia plantillas. Copiar y construir no son rivales.

5.6 El olor que pide Flyweight: un `TileType` por cada casilla

El segundo problema es el mapa. Una planta es una rejilla de casillas: muro, suelo, puerta, escalera y trampa. La forma directa de representarlas es un objeto por casilla:

Hipotético: un objeto por casilla

```
public record Tile(char glyph, String color, boolean walkable) {}

// al leer el mapa, para cada carácter:
grid[y][x] = new Tile('#', "gray", false);
```

Cada muro repite los mismos tres valores, y cada suelo también. En el mapa pequeño de este capítulo son unas pocas decenas de objetos y no importa; en un mapa de cien por cien serían diez mil objetos idénticos, y para cambiar el color de los muros tendrías que recorrerlos todos. El olor es **muchos objetos pequeños con datos idénticos que nunca cambian**.

La idea de Flyweight, en el catálogo, es usar la compartición para manejar de forma eficiente un gran número de objetos de grano fino [7]: si cien casillas son «muro», que sean cien referencias al *mismo* objeto muro. Nystrom lo ilustra también con un terreno hecho de casillas [12]. Para que compartir sea seguro hay que separar dos clases de datos:

- **Estado intrínseco**: lo que es igual para todos los usos y no cambia (el glifo, el color, si se puede pisar). Vive dentro del flyweight, que es inmutable.

- **Estado extrínseco**: lo que depende de cada uso (en qué posición está la casilla, dónde está el grupo). *No* vive en el flyweight: lo aporta quien lo usa.

Si metieras la posición dentro del flyweight, dos casillas distintas ya no podrían ser el mismo objeto.

Aquí el ahorro es de demostración

Con cinco tipos de casilla y mapas de 84 casillas, compartir apenas ahorra nada, y no lo medimos: este curso no hace pruebas de rendimiento. Lo que demuestra el ejemplo es la *estructura* (una fábrica que reparte las mismas instancias) y que lo compartido es el mismo objeto. Con cinco tipos fijos, un `enum` también serviría (Nystrom parte de él antes de llegar al flyweight [12]); usamos un `record` y una fábrica porque así se ve el patrón.

5.7 Piezas del patrón Flyweight

Rol	Clase en el ejemplo	Propósito	Paquete
Flyweight (record)	`TileType`	Estado intrínseco inmutable: glifo, color, transitable	`engine.world.map`
Fábrica de flyweights (clase)	`TileTypes`	Entrega siempre la misma instancia por glifo y cuenta cuántas creó	`engine.world.map`
Estado extrínseco (dato, no clase)	Posición `(x, y)`	Dónde está cada casilla y dónde está el grupo	`engine.world.map`
Contexto (clase)	`GameMap`	Rejilla de referencias a `TileType` y posición del grupo	`engine.world.map`
Cliente (clase)	`MapParser`	Convierte texto en `GameMap` pidiendo los tipos a la fábrica	`engine.world.map`

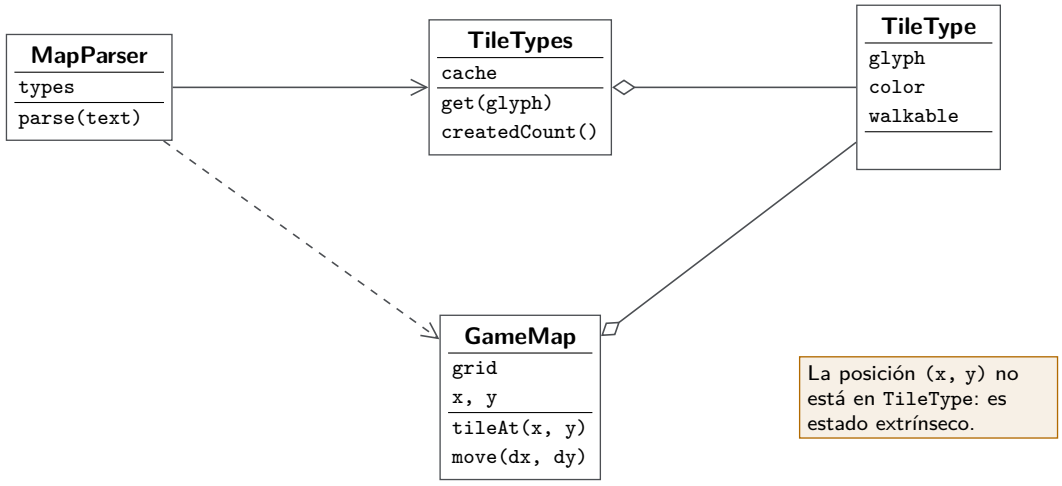


Diagrama UML del patrón Flyweight: la fábrica reparte instancias de `TileType`, y el mapa las referencia muchas veces.

5.8 El ejemplo de Flyweight, pieza a pieza: `TileTypes`

El flyweight. Es un record con los tres datos intrínsecos. El color es un nombre simbólico (`"gray"`, `"red"`): el capítulo 10 decidirá cómo se pinta, y mientras tanto no se usa.

engine/world/map/TileType.java

```
package emberdeep.engine.world.map;

public record TileType(char glyph, String color, boolean walkable) {}
```

Los cinco tipos, con su glifo en el texto del mapa: `#` muro (no transitable), `.` suelo, `+` puerta, `>` escalera y `^` trampa (todas transitables; la trampa aún no hace daño).

La fábrica. Guarda un mapa de glifo a `TileType` y un contador:

engine/world/map/TileTypes.java

```
package emberdeep.engine.world.map;

import java.util.HashMap;
import java.util.Map;

public class TileTypes {
    private final Map<Character, TileType> cache = new HashMap<>();
    private int created = 0;

    public TileType get(char glyph) {
        return cache.computeIfAbsent(glyph, this::create);
    }

    public int createdCount() {
        return created;
    }

    private TileType create(char glyph) {
        TileType type = switch (glyph) {
            case '#' -> new TileType('#', "gray", false);
            case '.' -> new TileType('.', "white", true);
            case '+' -> new TileType('+', "brown", true);
            case '>' -> new TileType('>', "cyan", true);
            case '^' -> new TileType('^', "red", true);
            default -> throw new IllegalArgumentException(
                    "Casilla desconocida: " + glyph);
        };
        created++;
        return type;
    }
}
```

`Map.computeIfAbsent`. Es el método de `Map` que implementa «dame lo que hay; si no hay nada, créalo, guárdalo y dámelo». Recibe la clave y una función que se llama *solo si la clave no está*. Escrito a mano, sería:

A mano: get, comprobar null y put

```
TileType type = cache.get(glyph);
if (type == null) {
    type = create(glyph);
    cache.put(glyph, type);
}
return type;
```

Con `computeIfAbsent(glyph, this::create)` son tres pasos en uno. Si la clave existe, devuelve el valor guardado y *no* llama a `create`; si no, llama una vez, guarda el resultado y lo devuelve. Dos detalles: la función no debe modificar el propio mapa (la nuestra solo incrementa un contador), y si lanza una excepción no se guarda nada, por eso `created++` va después del `switch`: un glifo desconocido no cuenta.

Demostración con == y con el contador. Dos llamadas con el mismo glifo devuelven el mismo objeto; una con otro glifo, otro objeto:

Comprobación (fragmento)

```
TileTypes types = new TileTypes();
TileType a = types.get('#');
TileType b = types.get('#');
TileType c = types.get('.');
System.out.println(a == b);                                  // true
System.out.println(a == c);                                  // false
System.out.println(a == new TileType('#', "gray", false)); // false
System.out.println(a.equals(new TileType('#', "gray", false))); // true
System.out.println(types.createdCount());                    // 2
```

La tercera línea muestra la diferencia: `==` compara *identidad* (¿es el mismo objeto?) y `equals`, en un record, compara *valores*. Flyweight garantiza lo primero. Pedir cien veces `'#'` deja `createdCount()` en 1.

== solo es fiable si la fábrica es la única vía

Con un flyweight, `==` funciona porque nadie más construye `TileType`: si alguien escribe `new TileType('#', ...)`, tendrás dos «muros» distintos. Un caso conocido del propio JDK: `Integer.valueOf(100) == Integer.valueOf(100)` es `true` porque `valueOf` reutiliza los enteros de -128 a 127, pero fuera de ese rango no puedes contar con ello. Moraleja: en código normal, compara con `equals`; usa `==` solo para demostrar la identidad de un flyweight, como aquí.

El contexto y el estado extrínseco. `GameMap` guarda una rejilla de *referencias* a `TileType` y la posición del grupo. Ninguna casilla sabe en qué posición está: la posición es el índice dentro de la rejilla, y es el estado extrínseco. Un fragmento (el archivo entero está en la solución del ejercicio de proyecto):

engine/world/map/GameMap.java (fragmento)

```
public class GameMap {
    private final TileType[][] grid;   // grid[fila][columna]
    private int x;                     // posición del grupo: estado extrínseco
    private int y;

```

```
    public TileType tileAt(int x, int y) {
        return grid[y][x];
    }

    public boolean move(int dx, int dy) {
        int nx = x + dx;
        int ny = y + dy;
        if (nx < 0 || ny < 0 || nx >= width() || ny >= height()
                || !grid[ny][nx].walkable()) {
            return false;
        }
        x = nx;
        y = ny;
        return true;
    }
}
```

El cliente. `MapParser` lee un texto y, para cada carácter, pide el tipo a la fábrica en lugar de crearlo: `grid[y][x] = types.get(c)`. Si el mapa tiene 84 casillas y cinco glifos distintos, termina con 84 referencias y `createdCount()` igual a 5.

5.9 Prototype, Factory y Flyweight: copiar, construir o compartir

Los tres patrones entregan objetos, y se confunden. Mira lo que hace cada uno:

	Prototype	**Factory**	**Flyweight**
Pregunta	¿Cómo obtengo otro como este, con un retoque?	¿Cómo construyo un objeto de este tipo desde cero?	¿Cómo evito tener mil copias de lo mismo?
Objeto nuevo	Sí, una copia por llamada	Sí, uno por llamada	Casi nunca: devuelve el mismo
Estado mutable	Sí, y cada copia tiene el suyo	Sí	No: el flyweight es inmutable
`==` entre dos resultados	`false`	`false`	`true` (mismo tipo)
En Emberdeep	`EnemyTemplate.copy()`	`CryptBiome.enemy(kind)`	`TileTypes.get(glyph)`

Criterio rápido: si lo que tienes es un *objeto ya montado* que quieres duplicar, es Prototype; si lo que tienes es un *tipo* y una receta, es Factory; si lo que tienes es *un dato que se repite y nunca cambia*, es Flyweight. Se combinan sin problema: `CryptBiome` es una Factory que usa Prototype por dentro. Practicas la decisión en el ejercicio 5.4.

5.10 Prototype y Flyweight: cuándo sí y cuándo no

Prototype compensa cuando el objeto es costoso o tedioso de montar y quieres variantes, cuando las variantes se definen como «lo mismo que X pero con esto distinto» (los élites) o cuando el tipo se decide en tiempo de ejecución con plantillas registradas. **No compensa** si el

objeto es un record inmutable (compartir el original ya es seguro, y para cambiar un dato basta un `with`), si tiene pocos campos (un constructor es más claro) o si no puedes decir qué campos son mutables: una copia mal hecha da errores silenciosos, como el de la sección anterior.

Flyweight compensa cuando hay muchísimos objetos con datos idénticos e inmutables y puedes separar el estado extrínseco sin esfuerzo. **No compensa** con pocos objetos, cuando el estado intrínseco cambia (no puedes compartir lo que muta), o cuando no has comprobado que el problema existe: sin medirlo, lo único seguro es que añades una fábrica, un mapa y la regla de que nadie use `new`. En Emberdeep, ya lo has visto, es una demostración. Para elegir con más calma entre los tres, el catálogo de [16] tiene una ficha de cada uno.

5.11 En el proyecto: H5 y H6 con Prototype y Flyweight

Este capítulo avanza dos hitos, cada uno con su patrón. Cambios del **H5** (Prototype):

- **Nuevos** `Skill`, `EnemyTemplate` y `TemplateRegistry` en `engine/character/`.
- **Cambia** `Enemy`: pasa a `Enemy(name, kind, stats, ai, skills)`. `Fighter`, `Combat` y `GameSession` no cambian: de momento nadie lee las habilidades.
- **Cambian** `CryptBiome` y `CavernBiome`: cada una tiene su `TemplateRegistry` y fabrica con `registry.get(kind).copy().toEnemy()`. La partida es la misma: las estadísticas, las IA y el `Random` no se tocan, así que las trazas de los capítulos anteriores se repiten. Los élites existen, pero ninguna planta los usa todavía.

Habilidades de cada tipo (datos del juego, un solo `Skill` por plantilla):

Tipo	Habilidad	Potencia	Bioma
`SKELETON`	Golpe de hueso	9	Cripta
`ASH_RAT`	Mordisco sucio	6	Cripta
`ZOMBIE`	Abrazo pútrido	7	Cripta
`GOBLIN`	Puñalada	8	Caverna
`BAT`	Chillido	5	Caverna
`SPIDER`	Picadura	9	Caverna

Cambios del **H6** (Flyweight):

- **Nuevo paquete** `engine/world/map/` con `TileType`, `TileTypes`, `GameMap`, `MapParser` y `MapData`. Hay un mapa fijo de 12 por 7 casillas por planta, escrito como bloque de texto en `MapData`, con `@` como posición inicial del grupo.
- **Cambia** `Floor`, que gana el texto del mapa: `Floor(biome, enemies, map)`. Es el paso que anunció el capítulo anterior: `FloorBuilder` gana `map(String)`, y `build()` avisa si una planta no tiene mapa («La planta N no tiene mapa»; se comprueba después del bioma y de los enemigos).
- **Cambia** `ConsoleUi`: nuevo método `explore`, que mueve al grupo con `w a s d`, avisa si choca con un muro, baja por la escalera y, con la tecla `t`, muestra cuántos `TileType` se han creado y comprueba con `==` que dos esquinas iguales son el mismo objeto.
- **Cambia** `Main`: con el segundo argumento `map` se explora el mapa desde la planta elegida en lugar de combatir. La lucha sigue empezando por planta, como antes.

Hitos 5 y 6: plantillas de enemigo y mapa con casillas compartidas

Tras estos hitos, cada enemigo sale de una plantilla copiada, existe el élite, y puedes recorrer un mapa de casillas compartidas. La estructura:

Estructura de carpetas tras el capítulo 5

```
emberdeep/
|-- settings.gradle
|-- build.gradle
|-- gradlew
|-- gradlew.bat
|-- gradle/wrapper/
`-- src/main/java/emberdeep/
    |-- app/
    |   |-- Main.java              (composition root)
    |   `-- GameConfig.java
    |-- engine/
    |   |-- character/             Stats, Hero, HeroBuilder, Enemy,
    ↪ EnemyKind, Fighter,
    |   |                          Skill, EnemyTemplate, TemplateRegistry
    |   |-- combat/
    |   |   |-- Combat.java
    |   |   |-- ai/                EnemyAi, FirstAliveAi, LowestHpAi,
    ↪ RandomTargetAi
    |   |   `-- damage/            DamageFormula, FlatFormula,
    ↪ ScaledFormula
    |   |-- item/                  Item, ItemKind, Inventory
    |   |-- session/               GameSession
    |   `-- world/
    |       |-- Dungeon.java, Floor.java, DungeonBuilder.java
    |       |-- biome/             BiomeFactory, CryptBiome, CavernBiome
    |       `-- map/               TileType, TileTypes, GameMap, MapParser,
    ↪  MapData
    `-- ui/
        `-- ConsoleUi.java
```

Lo practicas en el ejercicio 5.5. Antes, cuatro ejercicios más pequeños, en otros dominios.

5.12 Ejercicios

Ejercicio 5.1: El élite que contagia al original ★★☆☆☆

Este programa (no es de Emberdeep; dos archivos sin paquete) crea un lobo y su variante élite. Cada `MonsterTemplate` guarda un nombre, una vida y una lista de habilidades.

MonsterTemplate.java y Main.java

```java
import java.util.ArrayList;
import java.util.List;

public class MonsterTemplate {
    private String name;
    private int hp;
    private final List<String> abilities;

    public MonsterTemplate(String name, int hp, List<String> abilities) {
        this.name = name;
        this.hp = hp;
        this.abilities = abilities;
    }

    public MonsterTemplate copy() {
        return new MonsterTemplate(name, hp, abilities);
    }

    public MonsterTemplate asElite() {
        MonsterTemplate elite = copy();
        elite.name = "Élite " + name;
        elite.hp = hp * 2;
        elite.abilities.add("Furia");
        return elite;
    }

    public String name() { return name; }
    public List<String> abilities() { return abilities; }
}

public class Main {
    public static void main(String[] args) {
        MonsterTemplate wolf = new MonsterTemplate(
                "Lobo", 12, new ArrayList<>(List.of("Mordisco")));
        MonsterTemplate elite = wolf.asElite();
        System.out.println(wolf.name() + " " + wolf.abilities());
        System.out.println(elite.name() + " " + elite.abilities());
        System.out.println(wolf.abilities() == elite.abilities());
    }
}
```

1. Ejecútalo. ¿Qué imprime y qué debería imprimir?
2. Explica por qué el nombre del lobo no se contagió y la lista sí.
3. Corrige solo `MonsterTemplate`.

Está bien si, tras corregirlo, imprime

```
Lobo [Mordisco]
Élite Lobo [Mordisco, Furia]
false
```

y si en tu explicación aparecen las palabras «referencia» y «lista».

Pista: *Mira qué recibe el constructor en* `copy()`*: ¿la lista del original o una lista nueva?*

Depuración ▪ solución en la pág. 139

Ejercicio 5.2: Un SpellBook con copia profunda ★★☆☆☆

Vas a escribir un libro de hechizos que se copia sin compartir su lista. Archivos nuevos, sin paquete, no de Emberdeep. Pasos:

1. Crea el `record Spell(String name, int cost)`.
2. Crea la clase `SpellBook` con un campo `owner` (`String`, `final`) y una lista `spells` (`private final List<Spell>`, `ArrayList` vacía al crearse). Constructor público `SpellBook(String owner)`.
3. Añade `void add(Spell spell)`, `String owner()` y `List<Spell> spells()`, que devuelve `List.copyOf(spells)`.
4. Añade un constructor *privado* `SpellBook(SpellBook other)` que copie el dueño y *añada* cada hechizo de `other.spells` a la lista propia, y un método `SpellBook copy()` que lo use.
5. En `Main`, crea el libro de «Aria» con el hechizo «Bola de fuego» (coste 5), haz `copy()`, añade «Rayo» (coste 8) *solo a la copia* e imprime los tamaños y el dueño de la copia.

Está bien si imprime

```
Original: 1
Copia: 2
Dueña de la copia: Aria
```

y si los hechizos no se copian uno por uno (son un record: se comparten con seguridad).

Guiado ▪ solución en la pág. 140

Ejercicio 5.3: Un estilo de texto compartido ★★★☆☆

Un editor de texto guarda cada carácter con su estilo. Vas a compartir los estilos. Archivos nuevos, sin paquete, no de Emberdeep:

1. `record GlyphStyle(String font, int size, boolean bold)`: el flyweight.
2. `StyleFactory`, con `GlyphStyle get(String font, int size, boolean bold)` que usa `computeIfAbsent` y `int createdCount()`, que dice cuántos estilos creó.
3. `record PlacedChar(char value, int x, GlyphStyle style)`: el valor y la posición `x` son estado extrínseco; el estilo, intrínseco.
4. En `Main`, genera 10 000 `PlacedChar`. El de índice `i` tiene `value` igual a `(char) ('a' + i % 26)`, `x` igual a `i` y el estilo número `i % 4` de esta tabla: 0 = `mono`, 12, sin negrita; 1 = `mono`, 12, negrita; 2 = `serif`, 14, sin negrita; 3 = `serif`, 14, negrita.
5. Imprime el número de caracteres, `createdCount()` y el resultado de comparar con `==` los estilos de los índices 0 y 4, y de los índices 0 y 1.

Está bien si imprime

```
Caracteres: 10000
Estilos creados: 4
Índices 0 y 4, mismo objeto: true
Índices 0 y 1, mismo objeto: false
```

y si `GlyphStyle` no contiene la posición.

Pista: *Como clave del mapa de la fábrica puedes usar un* `String` *que una los tres datos, por ejemplo* `font + ":" + size + ":" + bold`*.*

Práctica ▪ solución en la pág. 142

Ejercicio 5.4: Copiar, construir o compartir ★★★☆☆

Para cada situación del juego, decide si usarías Factory, Prototype o Flyweight y justifícalo en una frase:

1. Crear el enemigo de un tipo dado, que en la cripta y en la caverna tiene datos distintos.
2. Un jefe montado con equipo y seis habilidades, del que quieres tres variantes que cambian solo la vida y una habilidad.
3. Las 10 000 casillas de suelo de un mapa grande, que comparten glifo y color.
4. El icono de cada tipo de objeto (un dato que no cambia), mostrado en cientos de huecos del inventario.
5. Una poción nueva cuya potencia se calcula a partir del número de planta.

Entrega una tabla con situación, patrón y la frase. Está bien si cada frase usa el verbo adecuado (construir desde cero, copiar y retocar, compartir lo que no cambia), si aparece al menos una situación de cada patrón y si ninguna justificación se reduce a «es más eficiente» sin decir en qué.

Diseño ▪ solución en la pág. 144

Ejercicio 5.5: Hitos 5 y 6: plantillas, élites y mapa ★★★★☆

Implementa los hitos H5 y H6 partiendo del estado del capítulo anterior (solución del ejercicio 4.5), en dos partes. Primero H5, y compruébalo antes de pasar a H6.

Parte 1 (H5): plantillas clonables.

1. En `engine/character/`: el record `Skill(String name, int power)`, y `Enemy(name, kind, stats, ai, skills)` con `List<Skill> skills` como último componente.
2. `EnemyTemplate`, clase mutable, con el constructor `(EnemyKind, String, Stats, EnemyAi)`, `addSkill(Skill)` (devuelve `this`), `setName`, `setStats`, los accesos `kind()`, `name()`, `stats()`, `ai()` y `skills()` (vista no modificable), y tres métodos: `copy()` (profunda: lista nueva; la IA se comparte), `toEnemy()` (con `List.copyOf`) y `asElite()` (nombre «Élite » más el nombre; vida por 3 y entre 2, *división entera*; ataque +2, defensa +1 y la habilidad `Skill("Furia", 12)` extra).
3. `TemplateRegistry` con `register(EnemyTemplate)` y `get(EnemyKind)`, que lanza `IllegalArgumentException` si no hay plantilla.
4. `CryptBiome` y `CavernBiome` montan su registro en el constructor (que sigue recibiendo el `Random`), con las habilidades de la tabla de la sección anterior, y fabrican con `registry.get(kind).copy().toEnemy()`. `item` no cambia.

Está bien si:

1. Sin argumentos y con el argumento `2`, el juego da las mismas trazas que al final del capítulo anterior.
2. Si añades temporalmente este fragmento al principio de `play` (con los imports de `Enemy`, `EnemyTemplate`, `Skill` y `FirstAliveAi`), la salida son cuatro líneas; bórralo después:

Comprobación temporal en Main.java

```
EnemyTemplate base = new EnemyTemplate(EnemyKind.ZOMBIE, "Zombi",
        new Stats(22, 5, 1), new FirstAliveAi())
        .addSkill(new Skill("Abrazo pútrido", 7));
EnemyTemplate elite = base.asElite();
System.out.println(base.skills().size() + " " + elite.skills().size())
    ↪ ;
System.out.println(elite.name() + " " + elite.stats());
Enemy enemy = elite.toEnemy();
System.out.println(enemy.kind() + " " + enemy.skills().size());
CryptBiome crypt = new CryptBiome(new Random(1));
Enemy a = crypt.enemy(EnemyKind.ZOMBIE);
Enemy b = crypt.enemy(EnemyKind.ZOMBIE);
System.out.println((a == b) + " " + (a.skills() == b.skills())
        + " " + a.skills().equals(b.skills()));
```

```
1 2
Élite Zombi Stats[maxHp=33, attack=7, defense=2]
ZOMBIE 2
false false true
```

Parte 2 (H6): mapa con casillas compartidas.

1. En `engine/world/map/`: `TileType` y `TileTypes` como en la sección del Flyweight (los cinco glifos de la sección).
2. `GameMap(TileType[][] grid, int startX, int startY)` con `width()`, `height()`, `tileAt(x, y)`, `partyX()`, `partyY()`, `partyTile()`, `onStairs()` (el glifo de la casilla del grupo es `>`), `move(dx, dy)` y `render()`. `move` devuelve `false` y no mueve si la casilla destino está fuera del mapa o no es transitable. `render()` devuelve las filas, cada una terminada en salto de línea, con `@` en la posición del grupo.
3. `MapParser(TileTypes)` con `parse(String)`: separa por líneas, trata `@` como la posición inicial (la casilla es suelo, `.`) y lanza `IllegalArgumentException` si una fila tiene otra longitud que la primera («Fila N con otra longitud») o si falta la `@` («Falta la posición inicial '@'»).
4. `MapData.floor(int)` devuelve el texto de cada una de las tres plantas, de 12 columnas por 7 filas, rodeado de muros, con una sola `@`, una sola `>` y los cinco glifos en cada mapa. El de la planta 1 es este (diseña tú los otros dos con las mismas reglas):

Mapa de la planta 1

```
############
#@...#....>#
#.##.#.##..#
#.#..+.#.^.#
#.#.##.#...#
#...#..+...#
############
```

5. `Floor` gana el componente `String map`; `FloorBuilder` gana `map(String)` (valor por defecto: vacío) y `DungeonBuilder.build()` lanza `IllegalStateException` con «La planta N no tiene mapa» si está vacío, después de comprobar el bioma y los enemigos. `Main` pasa `MapData.floor(n)` a cada planta.
6. `ConsoleUi.explore(GameMap, TileTypes)` devuelve `boolean`. En cada vuelta imprime el mapa, la línea «[w a s d] mover, [t] casillas, [q] salir» y el prompt `> `. Con `w`, `a`, `s` y `d` mueve (si no puede, imprime «No puedes pasar.»); con `t` imprime «Casillas: N, TileType creados: M» (N = ancho por alto del mapa) y «Esquinas opuestas, mismo objeto: B», donde B es el resultado de comparar con `==` las casillas `(0, 0)` y `(ancho-1, alto-1)`; con `q` devuelve `false`. Si tras una tecla el grupo está sobre la escalera, imprime «Bajas por la escalera.» y devuelve `true`.
7. En `Main`, con el segundo argumento `map`, se explora desde la planta del primer argumento: cada planta se parsea con el mismo `TileTypes`; si `explore` devuelve `true` y quedan plantas, se pasa a la siguiente; si no, termina. En ese modo no se pregunta la fórmula ni se combate.

Está bien si:

1. Los combates dan las mismas trazas que antes con los argumentos `1`, `2`, `3` y `1 default`.
2. Esta orden pulsa `a` (choca con el muro), `t` y `q`:

   Explorar la planta 1

   ```
   printf 'a\nt\nq\n' | ./gradlew -q run --args="1 map"
   ```

   y la salida contiene, en este orden (entre ellas se repite el mapa), estas líneas:

   Líneas esperadas

   ```
   > No puedes pasar.
   > Casillas: 84, TileType creados: 5
   Esquinas opuestas, mismo objeto: true
   ```

3. Esta orden recorre el camino hasta la escalera y sale en la planta siguiente:

   Bajar a la planta 2

   ```
   printf '%s\n' d d d s s d d w w d d d d q | ./gradlew -q run --args="1
       ↪ map"
   ```

   Aparece «Bajas por la escalera.» y justo después el mapa de la planta 2.

4. Si añades temporalmente este fragmento al principio de `play` (con los imports de las clases de `engine.world.map`), imprime `252 casillas, 5 TileType`:

Comprobación temporal en Main.java

```
TileTypes types = new TileTypes();
MapParser parser = new MapParser(types);
int cells = 0;
for (int number = 1; number <= 3; number++) {
    GameMap map = parser.parse(MapData.floor(number));
    cells += map.width() * map.height();
}
System.out.println(cells + " casillas, " + types.createdCount() + "
    ↪ TileType");
```

5. Un texto sin `@` y un texto con una fila más corta lanzan `IllegalArgumentException`.

Pista: *Para trocear el texto del mapa:* `text.strip().split("\n")`*. Un* `TileType[][]` *se indexa* `grid[fila][columna]`*, es decir,* `grid[y][x]`*.*

Proyecto ▪ solución en la pág. 144

5.13 Resumen

Resumen

- El olor de Prototype son objetos casi iguales montados repitiendo datos. Prototype copia un objeto ya montado y retoca la copia.
- Una copia profunda no copia todo: duplica lo mutable (la lista de habilidades), comparte lo inmutable (records, `String`) y lo que debe ser compartido (la IA con su `Random`). Una copia superficial comparte la lista y el élite contagia al original.
- `Cloneable` no se usa: `Object.clone()` es superficial y engorroso. Se prefiere un constructor o un método de copia (`copy()`).
- Un registro de prototipos entrega una plantilla por tipo; el cliente copia y nunca modifica el prototipo registrado. Prototype convive con Factory: `CryptBiome` copia por dentro.
- Flyweight comparte los objetos idénticos e inmutables: el estado intrínseco (glifo, color, transitable) vive en el flyweight; el extrínseco (la posición) lo aporta el contexto.
- Una fábrica de flyweights con `Map.computeIfAbsent` devuelve siempre la misma instancia por clave; se demuestra con `==` y con un contador (`createdCount()`), no con rendimiento.
- Copiar (Prototype), construir (Factory) o compartir (Flyweight): objeto montado, tipo y receta, o dato que se repite.
- H5 y H6: `Skill`, `EnemyTemplate`, `TemplateRegistry`, `Enemy` con `skills`, `TileType`, `TileTypes`, `GameMap`, `MapParser`, `MapData` y el modo `map`.

Soluciones de los ejercicios

Solución del ejercicio 5.1

enunciado en la pág. 132

1. Imprime `Lobo [Mordisco, Furia]`, `Élite Lobo [Mordisco, Furia]` y `true`. Debería imprimir `Lobo [Mordisco]`, `Élite Lobo [Mordisco, Furia]` y `false`.
2. En `copy()`, el constructor recibe `abilities`, es decir, la *referencia* a la lista del original. La copia y el original apuntan a la misma lista, y `elite.abilities.add("Furia")` la cambia para los dos; la última línea (`==` entre las dos listas) lo delata con `true`. El nombre, en cambio, es un `String` inmutable: `elite.name = "Élite " + name` asigna otro objeto a la copia y el original conserva el suyo.
3. La corrección es crear una lista nueva en `copy()`:

MonsterTemplate.java (corregido)

```
import java.util.ArrayList;
import java.util.List;

public class MonsterTemplate {
    private String name;
    private int hp;
    private final List<String> abilities;

    public MonsterTemplate(String name, int hp, List<String> abilities) {
        this.name = name;
        this.hp = hp;
        this.abilities = abilities;
    }

    public MonsterTemplate copy() {
        return new MonsterTemplate(name, hp, new ArrayList<>(abilities));
    }

    public MonsterTemplate asElite() {
        MonsterTemplate elite = copy();
        elite.name = "Élite " + name;
        elite.hp = hp * 2;
        elite.abilities.add("Furia");
        return elite;
    }

    public String name() { return name; }
    public List<String> abilities() { return abilities; }
}
```

Main.java (el mismo del enunciado)

```
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        MonsterTemplate wolf = new MonsterTemplate(
                "Lobo", 12, new ArrayList<>(List.of("Mordisco")));
        MonsterTemplate elite = wolf.asElite();
        System.out.println(wolf.name() + " " + wolf.abilities());
        System.out.println(elite.name() + " " + elite.abilities());
        System.out.println(wolf.abilities() == elite.abilities());
    }
}
```

Por qué funciona. `new ArrayList<>(abilities)` crea una lista distinta con los mismos elementos. Como los elementos son `String` (inmutables), no hace falta copiarlos: compartirlos es seguro. La copia profunda es «copiar lo mutable»; aquí lo único mutable es la lista.
Errores típicos:

- Arreglarlo en `asElite()` (por ejemplo, creando allí otra lista): el fallo está en `copy()`, y cualquier otro cliente de `copy()` seguiría contagiando.
- Pensar que el nombre también estaba compartido: lo que se comparte son las referencias, pero un `String` no se puede cambiar por dentro.
- Comprobar el arreglo solo con el tamaño de la lista: la prueba de identidad (`==`) es más directa.

Solución del ejercicio 5.2

enunciado en la pág. 134

Spell.java

```
public record Spell(String name, int cost) {}
```

SpellBook.java

```java
import java.util.ArrayList;
import java.util.List;

public class SpellBook {
    private final String owner;
    private final List<Spell> spells = new ArrayList<>();

    public SpellBook(String owner) {
        this.owner = owner;
    }

    private SpellBook(SpellBook other) {
        this.owner = other.owner;
        this.spells.addAll(other.spells);
    }

    public SpellBook copy() {
        return new SpellBook(this);
    }

    public void add(Spell spell) {
        spells.add(spell);
    }

    public String owner() { return owner; }

    public List<Spell> spells() {
        return List.copyOf(spells);
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        SpellBook book = new SpellBook("Aria");
        book.add(new Spell("Bola de fuego", 5));
        SpellBook copy = book.copy();
        copy.add(new Spell("Rayo", 8));
        System.out.println("Original: " + book.spells().size());
        System.out.println("Copia: " + copy.spells().size());
        System.out.println("Dueña de la copia: " + copy.owner());
    }
}
```

Por qué funciona. El constructor de copia crea su propia lista (por la inicialización del campo) y le añade los hechizos del otro libro: dos listas, los mismos elementos. Añadir «Rayo» a la copia no toca la lista del original. `Spell` es un record, inmutable, así que compartir sus instancias es seguro, y `owner` es un `String`, también inmutable.

Errores típicos:

- Escribir `this.spells = other.spells`: no compila si el campo es `final`; si quitas el `final` para que compile, vuelves a compartir la lista (el bug de la sección del capítulo).

- Devolver la lista interna en `spells()`: quien la reciba podría modificar el libro sin pasar por `add`. Por eso devuelve una copia inmutable.
- Copiar cada `Spell` con `new Spell(...)`: es innecesario con un record.

Solución del ejercicio 5.3

enunciado en la pág. 134

GlyphStyle.java

```java
public record GlyphStyle(String font, int size, boolean bold) {}
```

PlacedChar.java

```java
public record PlacedChar(char value, int x, GlyphStyle style) {}
```

StyleFactory.java

```java
import java.util.HashMap;
import java.util.Map;

public class StyleFactory {
    private final Map<String, GlyphStyle> cache = new HashMap<>();
    private int created = 0;

    public GlyphStyle get(String font, int size, boolean bold) {
        String key = font + ":" + size + ":" + bold;
        return cache.computeIfAbsent(key, k -> {
            created++;
            return new GlyphStyle(font, size, bold);
        });
    }

    public int createdCount() {
        return created;
    }
}
```

Main.java

```java
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        StyleFactory styles = new StyleFactory();
        String[] fonts = {"mono", "mono", "serif", "serif"};
        int[] sizes = {12, 12, 14, 14};
        boolean[] bolds = {false, true, false, true};

        List<PlacedChar> chars = new ArrayList<>();
        for (int i = 0; i < 10_000; i++) {
            int s = i % 4;
            GlyphStyle style = styles.get(fonts[s], sizes[s], bolds[s]);
            chars.add(new PlacedChar((char) ('a' + i % 26), i, style));
        }
        System.out.println("Caracteres: " + chars.size());
        System.out.println("Estilos creados: " + styles.createdCount());
        System.out.println("Índices 0 y 4, mismo objeto: "
                + (chars.get(0).style() == chars.get(4).style()));
        System.out.println("Índices 0 y 1, mismo objeto: "
                + (chars.get(0).style() == chars.get(1).style()));
    }
}
```

Por qué funciona. Los 10 000 caracteres son 10 000 objetos `PlacedChar` distintos (cada uno tiene su valor y su posición), pero el estilo es una referencia a uno de cuatro `GlyphStyle`. `computeIfAbsent` solo llama a la función cuando la clave es nueva, así que `created` sube cuatro veces. Los índices 0 y 4 piden el mismo estilo (`mono`, 12, sin negrita) y comparten objeto; el 0 y el 1 difieren en la negrita. La lambda modifica `created`, que es un campo, y por eso puede hacerlo (una variable local no se podría modificar desde una lambda).

Errores típicos:

- Poner la posición `x` dentro de `GlyphStyle`: ya no habría cuatro estilos, sino diez mil.
- Dejar fuera la negrita de la clave: los estilos 0 y 1 se fusionan y ya no son cuatro.
- Crear estilos con `new GlyphStyle(...)` en `Main`: se salta la fábrica y el `==` deja de ser fiable.

Solución del ejercicio 5.4 enunciado en la pág. 135

N.º	Patrón	Justificación
1	Factory	Se *construye desde cero* a partir de un tipo y de una receta que depende del bioma: eso es una fábrica (Abstract Factory, de hecho).
2	Prototype	Ya hay un objeto *montado*; las variantes son «lo mismo con esto distinto», y se resuelven *copiando y retocando* en lugar de repetir los datos.
3	Flyweight	Hay muchísimas casillas con datos idénticos e inmutables, y se *comparte* una sola instancia; la posición queda fuera, como estado extrínseco.
4	Flyweight	Es un dato que no cambia y se repite en cientos de usos: se *comparte* el icono de cada tipo y cada hueco solo guarda la referencia.
5	Factory	No hay nada montado que copiar: la poción se *construye desde cero* con un valor calculado (la planta), lo que es una fábrica o un método fábrica.

Hay dos matices que se valoran. La situación 2 podría resolverse con una fábrica si las variantes fueran pocas y los datos pocos; lo que inclina hacia Prototype es que el jefe es costoso de montar y las variantes son retoques. La situación 4 solo es Flyweight si el icono es inmutable; si cada hueco pudiera cambiar el suyo, habría que copiarlo o separar lo que cambia. Una respuesta es válida aunque difiera en un caso dudoso si lo justifica con el criterio de montar, copiar o compartir.

Errores típicos:

- Justificar con «es más eficiente» sin decir en qué: no se ha medido nada.
- Elegir Prototype para la poción de la situación 5 porque «hay muchas»: sin un objeto montado que copiar, es una fábrica.
- Elegir Flyweight cuando el dato cambia por uso (por ejemplo, la vida de cada enemigo): lo mutable no se comparte.

Solución del ejercicio 5.5 enunciado en la pág. 135

La solución sigue las dos partes. `Stats`, `Hero`, `HeroBuilder`, `Fighter`, `Combat`, `GameSession`, `GameConfig`, el paquete `ai` y el paquete `damage` no cambian. Aquí aparecen enteros todos los archivos que cambian o se crean. La parte 1 trae seis archivos (`Skill`, `Enemy`, `EnemyTemplate`, `TemplateRegistry`, `CryptBiome` y `CavernBiome`) y la parte 2, nueve (`Floor`, `DungeonBuilder`, `TileType`, `TileTypes`, `GameMap`, `MapParser`, `MapData`, `ConsoleUi` y `Main`).

Parte 1 (H5). engine/character/

Skill.java

```
1 package emberdeep.engine.character;
2
3 public record Skill(String name, int power) {}
```

Enemy.java

```java
package emberdeep.engine.character;

import emberdeep.engine.combat.ai.EnemyAi;
import java.util.List;

public record Enemy(String name, EnemyKind kind, Stats stats, EnemyAi ai,
                    List<Skill> skills) {}
```

EnemyTemplate.java (1/2: datos y copia)

```java
package emberdeep.engine.character;

import emberdeep.engine.combat.ai.EnemyAi;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class EnemyTemplate {
    private final EnemyKind kind;
    private String name;
    private Stats stats;
    private final List<Skill> skills;
    private final EnemyAi ai;

    public EnemyTemplate(EnemyKind kind, String name, Stats stats, EnemyAi
    ↪ ai) {
        this(kind, name, stats, new ArrayList<>(), ai);
    }

    private EnemyTemplate(EnemyKind kind, String name, Stats stats,
                          List<Skill> skills, EnemyAi ai) {
        this.kind = kind;
        this.name = name;
        this.stats = stats;
        this.skills = skills;
        this.ai = ai;
    }

    public EnemyTemplate copy() {
        return new EnemyTemplate(kind, name, stats, new ArrayList<>(skills),
    ↪  ai);
    }

    public Enemy toEnemy() {
        return new Enemy(name, kind, stats, ai, List.copyOf(skills));
    }
```

EnemyTemplate.java (2/2: élite y accesos)

```

    public EnemyTemplate asElite() {
        EnemyTemplate elite = copy();
        elite.setName("Élite " + name);
        elite.setStats(new Stats(stats.maxHp() * 3 / 2,
                stats.attack() + 2, stats.defense() + 1));
        elite.addSkill(new Skill("Furia", 12));
        return elite;
    }

    public EnemyTemplate addSkill(Skill skill) {
        skills.add(skill);
        return this;
    }

    public void setName(String name) { this.name = name; }
    public void setStats(Stats stats) { this.stats = stats; }

    public EnemyKind kind() { return kind; }
    public String name() { return name; }
    public Stats stats() { return stats; }
    public EnemyAi ai() { return ai; }

    public List<Skill> skills() {
        return Collections.unmodifiableList(skills);
    }
}
```

TemplateRegistry.java

```
package emberdeep.engine.character;

import java.util.EnumMap;
import java.util.Map;

public class TemplateRegistry {
    private final Map<EnemyKind, EnemyTemplate> templates =
            new EnumMap<>(EnemyKind.class);

    public void register(EnemyTemplate template) {
        templates.put(template.kind(), template);
    }

    public EnemyTemplate get(EnemyKind kind) {
        EnemyTemplate template = templates.get(kind);
        if (template == null) {
            throw new IllegalArgumentException("No hay plantilla para " +
    ↪ kind);
        }
        return template;
    }
}
```

engine/world/biome/CryptBiome.java (1/2: registro)

```java
package emberdeep.engine.world.biome;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.EnemyTemplate;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Stats;
import emberdeep.engine.character.TemplateRegistry;
import emberdeep.engine.combat.ai.FirstAliveAi;
import emberdeep.engine.combat.ai.LowestHpAi;
import emberdeep.engine.combat.ai.RandomTargetAi;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;
import java.util.Random;

public class CryptBiome implements BiomeFactory {
    private final TemplateRegistry registry = new TemplateRegistry();

    public CryptBiome(Random random) {
        registry.register(new EnemyTemplate(EnemyKind.SKELETON, "Esqueleto",
    ↪
                new Stats(18, 7, 2), new RandomTargetAi(random))
                .addSkill(new Skill("Golpe de hueso", 9)));
        registry.register(new EnemyTemplate(EnemyKind.ASH_RAT, "Rata de
    ↪ ceniza",
                new Stats(8, 5, 0), new LowestHpAi())
                .addSkill(new Skill("Mordisco sucio", 6)));
        registry.register(new EnemyTemplate(EnemyKind.ZOMBIE, "Zombi",
                new Stats(22, 5, 1), new FirstAliveAi())
                .addSkill(new Skill("Abrazo pútrido", 7)));
    }
```

CryptBiome.java (2/2: enemy e item)

```java

    @Override
    public Enemy enemy(EnemyKind kind) {
        return registry.get(kind).copy().toEnemy();
    }

    @Override
    public Item item(ItemKind kind) {
        return switch (kind) {
            case HEALING -> new Item("Poción de cripta", kind, 12);
            case BOMB -> new Item("Bomba de ceniza", kind, 10);
        };
    }
}
```

engine/world/biome/CavernBiome.java (1/2: registro)

```
package emberdeep.engine.world.biome;

import emberdeep.engine.character.Enemy;
import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.EnemyTemplate;
import emberdeep.engine.character.Skill;
import emberdeep.engine.character.Stats;
import emberdeep.engine.character.TemplateRegistry;
import emberdeep.engine.combat.ai.FirstAliveAi;
import emberdeep.engine.combat.ai.LowestHpAi;
import emberdeep.engine.combat.ai.RandomTargetAi;
import emberdeep.engine.item.Item;
import emberdeep.engine.item.ItemKind;
import java.util.Random;

public class CavernBiome implements BiomeFactory {
    private final TemplateRegistry registry = new TemplateRegistry();

    public CavernBiome(Random random) {
        registry.register(new EnemyTemplate(EnemyKind.GOBLIN, "Goblin",
                new Stats(14, 6, 1), new FirstAliveAi())
                .addSkill(new Skill("Puñalada", 8)));
        registry.register(new EnemyTemplate(EnemyKind.BAT, "Murciélago",
                new Stats(10, 6, 0), new LowestHpAi())
                .addSkill(new Skill("Chillido", 5)));
        registry.register(new EnemyTemplate(EnemyKind.SPIDER, "Araña",
                new Stats(16, 7, 2), new RandomTargetAi(random))
                .addSkill(new Skill("Picadura", 9)));
    }
```

CavernBiome.java (2/2: enemy e item)

```

    @Override
    public Enemy enemy(EnemyKind kind) {
        return registry.get(kind).copy().toEnemy();
    }

    @Override
    public Item item(ItemKind kind) {
        return switch (kind) {
            case HEALING -> new Item("Poción de caverna", kind, 10);
            case BOMB -> new Item("Bomba de roca", kind, 14);
        };
    }
}
```

Por qué funciona (parte 1). Cada bioma guarda sus plantillas y entrega, por cada `enemy(kind)`, un `Enemy` nuevo con su propia lista de habilidades. El `switch` sobre los seis tipos ha desaparecido: pedir un tipo ajeno a la cripta cae en `registry.get`, que lanza `IllegalArgumentException` (el mensaje cambia, el tipo de excepción no). Las estadísticas, las IA y el `Random` son los de siempre, y el registro se monta en el constructor sin consumir

números aleatorios, así que las trazas son idénticas a las del capítulo anterior. En el fragmento de comprobación, el élite tiene 2 habilidades y el zombi base sigue teniendo 1; los dos enemigos del mismo tipo no comparten lista (`false`) pero sus listas son iguales (`true`).

Errores típicos (parte 1):

- Convertir `registry.get(kind)` en enemigo sin `copy()`: dos enemigos del mismo tipo compartirían la lista de la plantilla.
- Llamar a `addSkill` sobre el resultado de `get`: cambias el prototipo y todos los enemigos futuros de ese tipo ganan la habilidad.
- Duplicar la IA en `copy()` con `new RandomTargetAi(...)`: rompe la semilla compartida.
- Calcular la vida del élite con `maxHp() * 1.5`: no compila (resultado `double`); se usa `* 3 / 2`.

Parte 2 (H6). engine/world/ (cambios de H4)

Floor.java

```
package emberdeep.engine.world;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.world.biome.BiomeFactory;
import java.util.List;

public record Floor(BiomeFactory biome, List<EnemyKind> enemies, String map)
    ↪ {
    public Floor {
        enemies = List.copyOf(enemies);
    }
}
```

DungeonBuilder.java (1/2: la mazmorra)

```java
package emberdeep.engine.world;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.world.biome.BiomeFactory;
import java.util.ArrayList;
import java.util.List;

public class DungeonBuilder {
    private final List<Floor> floors = new ArrayList<>();

    public FloorBuilder floor() {
        return new FloorBuilder();
    }

    public Dungeon build() {
        if (floors.isEmpty()) {
            throw new IllegalStateException("La mazmorra necesita al menos una planta");
        }
        for (int i = 0; i < floors.size(); i++) {
            Floor floor = floors.get(i);
            String label = "La planta " + (i + 1);
            if (floor.biome() == null) {
                throw new IllegalStateException(label + " no tiene bioma");
            }
            if (floor.enemies().isEmpty()) {
                throw new IllegalStateException(label + " no tiene enemigos");
            }
            if (floor.map() == null || floor.map().isBlank()) {
                throw new IllegalStateException(label + " no tiene mapa");
            }
        }
        return new Dungeon(floors);
    }
```

DungeonBuilder.java (2/2: FloorBuilder)

```

    public class FloorBuilder {
        private BiomeFactory biome;
        private List<EnemyKind> enemies = List.of();
        private String map = "";

        public FloorBuilder biome(BiomeFactory biome) {
            this.biome = biome;
            return this;
        }

        public FloorBuilder enemies(List<EnemyKind> enemies) {
            this.enemies = enemies;
            return this;
        }

        public FloorBuilder map(String map) {
            this.map = map;
            return this;
        }

        public DungeonBuilder done() {
            floors.add(new Floor(biome, enemies, map));
            return DungeonBuilder.this;
        }
    }
}
```

engine/world/map/

TileType.java

```
package emberdeep.engine.world.map;

public record TileType(char glyph, String color, boolean walkable) {}
```

TileTypes.java

```java
package emberdeep.engine.world.map;

import java.util.HashMap;
import java.util.Map;

public class TileTypes {
    private final Map<Character, TileType> cache = new HashMap<>();
    private int created = 0;

    public TileType get(char glyph) {
        return cache.computeIfAbsent(glyph, this::create);
    }

    public int createdCount() {
        return created;
    }

    private TileType create(char glyph) {
        TileType type = switch (glyph) {
            case '#' -> new TileType('#', "gray", false);
            case '.' -> new TileType('.', "white", true);
            case '+' -> new TileType('+', "brown", true);
            case '>' -> new TileType('>', "cyan", true);
            case '^' -> new TileType('^', "red", true);
            default -> throw new IllegalArgumentException(
                    "Casilla desconocida: " + glyph);
        };
        created++;
        return type;
    }
}
```

GameMap.java

```
package emberdeep.engine.world.map;

public class GameMap {
    private final TileType[][] grid;
    private int x;
    private int y;

    public GameMap(TileType[][] grid, int startX, int startY) {
        this.grid = grid;
        this.x = startX;
        this.y = startY;
    }

    public int width() { return grid[0].length; }
    public int height() { return grid.length; }
    public TileType tileAt(int x, int y) { return grid[y][x]; }
    public int partyX() { return x; }
    public int partyY() { return y; }
    public TileType partyTile() { return grid[y][x]; }
    public boolean onStairs() { return partyTile().glyph() == '>'; }

    public boolean move(int dx, int dy) {
        int nx = x + dx;
        int ny = y + dy;
        if (nx < 0 || ny < 0 || nx >= width() || ny >= height()
                || !grid[ny][nx].walkable()) {
            return false;
        }
        x = nx;
        y = ny;
        return true;
    }

    public String render() {
        StringBuilder text = new StringBuilder();
        for (int row = 0; row < height(); row++) {
            for (int col = 0; col < width(); col++) {
                boolean party = row == y && col == x;
                text.append(party ? '@' : grid[row][col].glyph());
            }
            text.append('\n');
        }
        return text.toString();
    }
}
```

MapParser.java

```java
package emberdeep.engine.world.map;

public class MapParser {
    private final TileTypes types;

    public MapParser(TileTypes types) {
        this.types = types;
    }

    public GameMap parse(String text) {
        String[] rows = text.strip().split("\n");
        int width = rows[0].length();
        TileType[][] grid = new TileType[rows.length][width];
        int startX = -1;
        int startY = -1;
        for (int y = 0; y < rows.length; y++) {
            if (rows[y].length() != width) {
                throw new IllegalArgumentException(
                        "Fila " + (y + 1) + " con otra longitud");
            }
            for (int x = 0; x < width; x++) {
                char glyph = rows[y].charAt(x);
                if (glyph == '@') {
                    startX = x;
                    startY = y;
                    glyph = '.';
                }
                grid[y][x] = types.get(glyph);
            }
        }
        if (startX < 0) {
            throw new IllegalArgumentException("Falta la posición inicial '
    ↪ @'");
        }
        return new GameMap(grid, startX, startY);
    }
}
```

MapData.java

```
package emberdeep.engine.world.map;

public final class MapData {
    private static final String FLOOR_1 = """
            #############
            #@...#....>#
            #.##.#.##..#
            #.#..+.#.^.#
            #.#.##.#...#
            #...#..+...#
            #############
            """;
    private static final String FLOOR_2 = """
            #############
            #>..#......#
            #.#.#.####.#
            ###...#..#.#
            #.####.^.+.#
            #@.......#.#
            #############
            """;
    private static final String FLOOR_3 = """
            #############
            #@.#.....>.#
            #..#.###.#.#
            #.##.#^..#.#
            #....#.+##.#
            #.####...^.#
            #############
            """;

    private MapData() {}

    public static String floor(int number) {
        return switch (number) {
            case 1 -> FLOOR_1;
            case 2 -> FLOOR_2;
            case 3 -> FLOOR_3;
            default -> throw new IllegalArgumentException(
                    "No hay mapa para la planta " + number);
        };
    }
}
```

ui/ConsoleUi.java (1/4: imports y fórmula)

```java
package emberdeep.ui;

import emberdeep.engine.character.Fighter;
import emberdeep.engine.combat.Combat;
import emberdeep.engine.item.Inventory;
import emberdeep.engine.item.Item;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.map.GameMap;
import emberdeep.engine.world.map.TileType;
import emberdeep.engine.world.map.TileTypes;
import java.io.PrintStream;
import java.util.List;
import java.util.Scanner;
import java.util.stream.Collectors;

public class ConsoleUi {
    private final Scanner in;
    private final PrintStream out;

    public ConsoleUi(Scanner in, PrintStream out) {
        this.in = in;
        this.out = out;
    }

    public int askFormula() {
        out.println("Fórmula de daño:");
        out.println("  [1] plana: ataque - defensa");
        out.println("  [2] escalada: ataque x 100 / (100 + defensa)");
        return ask(2);
    }
```

ConsoleUi.java (2/4: la partida)

```
    public void run(GameSession session) {
        Combat combat = session.combat();
        out.println("=== Emberdeep ===");
        while (!combat.isOver()) {
            out.println("--- Ronda " + session.round() + " ---");
            for (Fighter hero : session.heroes()) {
                if (!hero.isAlive() || combat.isOver()) {
                    continue;
                }
                out.println("Héroes:   " + describe(session.heroes()));
                out.println("Enemigos: " + describe(session.enemies()));
                out.println("Objetos:  " + describeItems(session.inventory()
    ↪ ));
                out.println("Turno de " + hero.name() + ". [1] atacar");
                ask(1);
                Fighter target = askTarget(session.enemies());
                print(combat.heroAttack(hero, target));
            }
            if (!combat.isOver()) {
                print(combat.enemyPhase());
            }
        }
        out.println(combat.heroesWon() ? "¡Victoria!" : "Derrota...");
    }
```

ConsoleUi.java (3/4: modo mapa)

```
    public boolean explore(GameMap map, TileTypes types) {
        while (true) {
            out.print(map.render());
            out.println("[w a s d] mover, [t] casillas, [q] salir");
            out.print("> ");
            out.flush();
            String key = in.nextLine().trim();
            switch (key) {
                case "w" -> step(map, 0, -1);
                case "a" -> step(map, -1, 0);
                case "s" -> step(map, 0, 1);
                case "d" -> step(map, 1, 0);
                case "t" -> showTiles(map, types);
                case "q" -> { return false; }
                default -> out.println("Tecla no válida.");
            }
            if (map.onStairs()) {
                out.println("Bajas por la escalera.");
                return true;
            }
        }
    }

    private void step(GameMap map, int dx, int dy) {
        if (!map.move(dx, dy)) {
            out.println("No puedes pasar.");
        }
    }

    private void showTiles(GameMap map, TileTypes types) {
        TileType corner = map.tileAt(0, 0);
        TileType opposite = map.tileAt(map.width() - 1, map.height() - 1);
        out.println("Casillas: " + map.width() * map.height()
                + ", TileType creados: " + types.createdCount());
        out.println("Esquinas opuestas, mismo objeto: " + (corner ==
    ↪ opposite));
    }
```

ConsoleUi.java (4/4: auxiliares y llave final)

```
    private String describe(List<Fighter> group) {
        return group.stream()
                .map(f -> f.name() + " " + f.hp() + "/" + f.stats().maxHp())
                .collect(Collectors.joining(", "));
    }

    private String describeItems(Inventory inventory) {
        if (inventory.items().isEmpty()) {
            return "(ninguno)";
        }
        return inventory.items().stream()
                .map(Item::name)
                .collect(Collectors.joining(", "));
    }

    private void print(List<String> messages) {
        messages.forEach(out::println);
    }

    private int ask(int max) {
        while (true) {
            out.print("> ");
            out.flush();
            String line = in.nextLine().trim();
            try {
                int n = Integer.parseInt(line);
                if (n >= 1 && n <= max) {
                    return n;
                }
            } catch (NumberFormatException e) {
                // no es un número: se avisa más abajo
            }
            out.println("Escribe un número entre 1 y " + max + ".");
        }
    }

    private Fighter askTarget(List<Fighter> enemies) {
        List<Fighter> alive = enemies.stream().filter(Fighter::isAlive).toList();
        StringBuilder menu = new StringBuilder("Objetivo:");
        for (int i = 0; i < alive.size(); i++) {
            menu.append("  [").append(i + 1).append("] ").append(alive.get(i).name());
        }
        out.println(menu);
        return alive.get(ask(alive.size()) - 1);
    }
}
```

app/Main.java (1/3: imports y arranque)

```java
package emberdeep.app;

import emberdeep.engine.character.EnemyKind;
import emberdeep.engine.character.Hero;
import emberdeep.engine.character.HeroBuilder;
import emberdeep.engine.character.Stats;
import emberdeep.engine.combat.damage.DamageFormula;
import emberdeep.engine.combat.damage.FlatFormula;
import emberdeep.engine.combat.damage.ScaledFormula;
import emberdeep.engine.session.GameSession;
import emberdeep.engine.world.Dungeon;
import emberdeep.engine.world.DungeonBuilder;
import emberdeep.engine.world.Floor;
import emberdeep.engine.world.biome.BiomeFactory;
import emberdeep.engine.world.biome.CavernBiome;
import emberdeep.engine.world.biome.CryptBiome;
import emberdeep.engine.world.map.MapData;
import emberdeep.engine.world.map.MapParser;
import emberdeep.engine.world.map.TileTypes;
import emberdeep.ui.ConsoleUi;
import java.util.List;
import java.util.Random;
import java.util.Scanner;

public class Main {
    private final GameConfig config;

    private Main(GameConfig config) {
        this.config = config;
    }

    public static void main(String[] args) {
        GameConfig config = new GameConfig(42L, 3, new FlatFormula());
        new Main(config).play(args);
    }
```

Main.java (2/3: la partida y el modo mapa)

```

    private void play(String[] args) {
        int floorNumber = chooseFloor(args);
        ConsoleUi ui = new ConsoleUi(new Scanner(System.in), System.out);
        Random random = new Random(config.seed());

        BiomeFactory crypt = new CryptBiome(random);
        BiomeFactory cavern = new CavernBiome(random);
        List<EnemyKind> cryptKinds = List.of(
                EnemyKind.SKELETON, EnemyKind.ASH_RAT, EnemyKind.ZOMBIE);
        List<EnemyKind> cavernKinds = List.of(
                EnemyKind.GOBLIN, EnemyKind.BAT, EnemyKind.SPIDER);

        Dungeon dungeon = new DungeonBuilder()
                .floor().biome(crypt).enemies(cryptKinds)
                        .map(MapData.floor(1)).done()
                .floor().biome(cavern).enemies(cavernKinds)
                        .map(MapData.floor(2)).done()
                .floor().biome(crypt).enemies(cryptKinds)
                        .map(MapData.floor(3)).done()
                .build();

        if (args.length > 1 && args[1].equals("map")) {
            explore(ui, dungeon, floorNumber);
            return;
        }
        DamageFormula formula = chooseFormula(ui, args);
        Floor floor = dungeon.floors().get(floorNumber - 1);
        GameSession session = new GameSession(
                heroes(), floor.biome(), floor.enemies(), formula);
        ui.run(session);
    }

    private void explore(ConsoleUi ui, Dungeon dungeon, int firstFloor) {
        TileTypes types = new TileTypes();
        MapParser parser = new MapParser(types);
        int number = firstFloor;
        while (true) {
            Floor floor = dungeon.floors().get(number - 1);
            boolean descended = ui.explore(parser.parse(floor.map()), types)
    ↪ ;
            if (!descended || number == dungeon.floors().size()) {
                return;
            }
            number++;
        }
    }
```

Main.java (3/3: auxiliares y llave final)

```
    private DamageFormula chooseFormula(ConsoleUi ui, String[] args) {
        if (args.length > 1 && args[1].equals("default")) {
            return config.defaultFormula();
        }
        return ui.askFormula() == 1 ? new FlatFormula() : new ScaledFormula
    ↪ ();
    }

    private int chooseFloor(String[] args) {
        int floor = args.length > 0 ? Integer.parseInt(args[0]) : 1;
        if (floor < 1 || floor > config.floors()) {
            throw new IllegalArgumentException(
                    "La planta debe estar entre 1 y " + config.floors());
        }
        return floor;
    }

    private List<Hero> heroes() {
        return List.of(
                new HeroBuilder().name("Aria")
                        .stats(new Stats(30, 8, 3)).build(),
                new HeroBuilder().name("Borin")
                        .stats(new Stats(40, 6, 5)).build(),
                new HeroBuilder().name("Cael")
                        .stats(new Stats(24, 9, 2)).build());
    }
}
```

Por qué funciona (parte 2). `TileTypes` reparte una sola instancia por glifo: el mapa de la planta 1 tiene 84 casillas y sus cinco glifos distintos (el `@` se sustituye por suelo) crean 5 `TileType`; las otras plantas piden los mismos y no crean más, por eso el fragmento de comprobación da `252 casillas, 5 TileType`. Las dos esquinas del mapa son muros, así que `tileAt(0, 0) == tileAt(ancho-1, alto-1)` es `true`: es el mismo objeto, no dos iguales. La posición del grupo está en `GameMap` (estado extrínseco); ninguna casilla sabe dónde está. `render()` devuelve el texto y es `ConsoleUi` quien lo imprime, así que el motor sigue sin usar `System.out`. `Main` construye la mazmorra antes de decidir si combate o explora; esto no consume números aleatorios ni imprime nada, y la pregunta de la fórmula (que sí imprime) sigue yendo antes de la primera ronda, por lo que las trazas de combate no cambian. Como `DungeonBuilder` comprueba el mapa *después* del bioma y de los enemigos, la comprobación del capítulo anterior (planta sin enemigos) sigue dando «La planta 1 no tiene enemigos».

Errores típicos (parte 2):

- Poner la posición del grupo dentro de `TileType` o crear un `TileType` por carácter: pierdes la compartición y `createdCount()` sube con el tamaño del mapa.
- Dejar `@` como un sexto glifo: `TileTypes` lanzaría `IllegalArgumentException`. El parser lo trata como posición inicial sobre suelo.
- Hacer `created++` antes del `switch` en `create`: un glifo desconocido contaría aunque no se guarde nada.

- Modificar `cache` dentro de la función de `computeIfAbsent`: el método no lo permite.
- Escribir el mapa con una fila de otra longitud: el parser avisa con «Fila N con otra longitud».
- Olvidar la `q` al final de la orden con `printf`: al acabarse la entrada, `nextLine()` lanza `NoSuchElementException`.

Estructura final: la del árbol de la sección «En el proyecto». Este es el estado del que parte el capítulo siguiente.

Parte III

Ampliar el comportamiento

Parte IV

Comunicar

Parte V

Estados, estructura y cierre

Referencias

[1] Kent Beck y Ward Cunningham. «Using Pattern Languages for Object-Oriented Programs». En: *OOPSLA-87 Workshop on Specification and Design for Object-Oriented Programming*. Technical Report CR-87-43. Sep. de 1987. URL: https://c2.com/doc/oopsla87.html (visitado 24-10-2026).

[2] Gavin Bierman. *JEP 441: Pattern Matching for switch*. Final en Java 21. OpenJDK. 2023. URL: https://openjdk.org/jeps/441 (visitado 24-10-2026).

[3] Joshua Bloch. *Effective Java*. 3.ª ed. Addison-Wesley, 2018. ISBN: 978-0-13-468599-1.

[4] Martin Fowler. *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. 3.ª ed. Copyright 2004 según InformIT. Addison-Wesley, 2003. ISBN: 978-0-321-19368-1.

[5] Eric Freeman y Elisabeth Robson. *Head First Design Patterns: Building Extensible and Maintainable Object-Oriented Software*. 2.ª ed. O'Reilly Media, 2020. ISBN: 978-1-4920-7800-5.

[6] Erich Gamma et al. «Design Patterns: Abstraction and Reuse of Object-Oriented Design». En: *ECOOP'93 — Object-Oriented Programming*. Lecture Notes in Computer Science. Springer, 1993, págs. 406-431. DOI: 10.1007/3-540-47910-4_21.

[7] Erich Gamma et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Publicado el 31-10-1994; copyright 1995 según InformIT. Addison-Wesley, 1994. ISBN: 978-0-201-63361-0.

[8] Gradle. *Building Java & JVM projects*. 2026. URL: https://docs.gradle.org/current/userguide/building_java_projects.html (visitado 24-10-2026).

[9] Gradle. *Gradle User Manual, versión 9.8.2*. 2026. URL: https://docs.gradle.org/current/userguide/userguide.html (visitado 24-10-2026).

[10] Joshua Kerievsky. *Refactoring to Patterns*. Copyright 2005 según InformIT. Addison-Wesley, 2004. ISBN: 978-0-321-21335-8.

[11] Peter Norvig. *Design Patterns in Dynamic Languages*. Presentado el 5-5-1996; publicado en la web el 17-3-1998. 1996. URL: https://norvig.com/design-patterns/ (visitado 24-10-2026).

[12] Robert Nystrom. *Game Programming Patterns*. Versión web gratuita en gameprogrammingpatterns.com. Edición del autor, 2014. ISBN: 978-0-9905829-0-8.

[13] Object Management Group. *Unified Modeling Language (UML), versión 2.5.1*. formal/17-12-05. Dic. de 2017. URL: https://www.omg.org/spec/UML/2.5.1 (visitado 24-10-2026).

[14] Oracle. *Java Platform, Standard Edition: Java Language Updates, Release 21*. Documento F79543-03. Ene. de 2025. URL: https://docs.oracle.com/en/java/javase/21/language/ (visitado 24-10-2026).

[15] Oracle. *Package java.util.function (Java SE 21 & JDK 21 API)*. 2026. URL: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/function/package-summary.html (visitado 24-10-2026).

[16] Refactoring.Guru. *The Catalog of Design Patterns*. 2026. URL: https://refactoring.guru/design-patterns/catalog (visitado 24-10-2026).

[17] Till Tantau y The PGF/TikZ Team. *PGF and TikZ, versión 3.1.12*. CTAN. 2026. URL: https://ctan.org/pkg/pgf (visitado 24-10-2026).

[18] Raoul-Gabriel Urma, Mario Fusco y Alan Mycroft. *Modern Java in Action: Lambdas, Streams, Functional and Reactive Programming*. Manning, 2018. ISBN: 978-1-61729-356-6.